



Menoufia University
Faculty of Engineering

Electrical and Computer Engineering

Graduation Project

**Design and Implementation of a ShuffleNet Accelerator
on FPGA with RTL and HLS**

Sponsored by



Prepared by

Name	ID
Ahmed Ahmed Nageeb Elbermawy	132300537
Yousef Abdulrahman Abdulnabi	132300646
Mohamed Ahmed Roshdy Saad	000210503
Basmala Hatem Abdullah Badr	002200012

Supervised by

Prof. Dr. Tamer Fetouh

ABSTRACT

This thesis presents the design and implementation of a real-time image classification hardware accelerator for the ShuffleNet V2 $\times$ 1.0 convolutional neural network, targeting the iWave ZU19EG Development Kit (Xilinx Zynq UltraScale+ MPSoC, part xczu19eg-ffvc1760-1-i). The work is structured as two complementary implementation tracks evaluated on identical silicon, enabling a direct comparison between a hand-crafted register-transfer level (RTL) design and an automated high-level synthesis (HLS) design flow.

The primary contribution is a fully custom RTL accelerator implemented in Verilog. The design adopts a baseline Q6.8 fixed-point datapath (15-bit signed words: one sign bit, six integer bits, and eight fractional bits), refined by a per-layer bit-width optimization that reduces feature-map and weight widths on a layer-by-layer basis according to measured dynamic range. All convolution weights are quantized offline, with batch normalization folded into the preceding convolution, and stored entirely in on-chip Block RAM, eliminating any external DDR memory access during inference. The accelerator is organized into three pipelined hardware groups — a 3×3 convolution and max-pooling stage, sixteen ShuffleNet V2 basic units organized across three network stages, and a final 1×1 convolution, global average pooling, and fully connected classification stage — allowing successive frames to be processed in an overlapped, pipelined manner. The ARM Cortex-A53 Processing System (PS) transfers input images and reads back classification results through a 32 MB AXI4-Lite interface.

The RTL design was verified using 48 self-checking Verilog testbenches covering all 50 synthesizable modules, all of which pass under Vivado XSim 2024.2. Post-implementation results on the ZU19EG show that the design closes timing at a 100 MHz clock with a worst negative slack of +0.096 ns, while consuming 1,550 of 1,968 DSP48E2 slices (78.76%), 74,936 of 522,720 look-up tables (14.34%), and 577.5 of 984 Block RAM tiles (58.69%). The accelerator was subsequently programmed onto a physical iWave ZU19EG development board, where it correctly classified a test image end to end; at its verified 100 MHz clock, the design's 82,200-cycle pipeline period corresponds to a frame rate of 1,216 fps (0.822 ms per frame), with a Vivado post-implementation power estimate of 6.816 W, yielding approximately 0.00560 J per classified frame. A bit-accurate Q6.8 software model of the complete network, built from batch-normalization-folded PyTorch weights, reproduces the RTL arithmetic

exactly and achieves approximately 71% top-1 accuracy on a 100-image ImageNet validation subset, confirming that the fixed-point quantization scheme preserves classification accuracy.

The second track comprises four progressively optimized Vitis HLS C++ models of the same network — a sequential baseline, a pipelined version, a version using loop unrolling and array partitioning, and a combined model — targeting the same ZU19EG device for a same-silicon comparison. C-synthesis estimates were obtained for all four models; post-place-and-route HLS results could not be obtained on the available toolchain due to the scale of the fully unrolled design, and are identified as future work. On-board frame rate, latency, power, and energy measurements for the RTL design, by contrast, were obtained following successful physical hardware deployment on the iWave ZU19EG board, and are reported in Chapter 9 alongside the RTL-versus-HLS comparison.

Index Terms—FPGA, ShuffleNet V2, Convolutional Neural Network, Hardware Accelerator, Fixed-Point Quantization, Q6.8, High-Level Synthesis, RTL, Zynq UltraScale+, AXI4-Lite.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to our supervisor, Prof. Dr. Tamer Fetouh Abdelghany, for his continuous guidance, technical insight, and support throughout every stage of this project, from architectural decisions to verification methodology.

We are also grateful to the Electrical and Computer Engineering Department at Menoufia University for providing the academic foundation and resources that made this work possible.

We further acknowledge the Academy of Scientific Research and Technology, Etisal, SI-Vision, SI-Vision Academy, and Global Innovation & Entrepreneurship for their generous sponsorship of this graduation project. Their support enabled us to implement our design on an FPGA platform valued at more than \$5,000, a resource that would not have been accessible to us otherwise and was instrumental in bringing this project from simulation to real hardware.

Finally, we extend our deepest thanks to our families for their unwavering patience, encouragement, and support throughout every stage of this journey — their belief in us gave us the strength to see this project through. We also thank our colleagues for their camaraderie and encouragement throughout the duration of this work.

TABLE OF CONTENTS

Abstract.....	i
Acknowledgments.....	iii
Table of Contents.....	iv
List of Figures.....	x
List of Tables	xvi
List of Abbreviations	xvii
Part I: Introduction.....	1
Chapter 1: Historical Foundations	2
1.1 VLSI History.....	2
1.1.1 From the Transistor to the Integrated Circuit.....	2
1.1.2 Moore's Law and the Scaling Era	3
1.1.3 From Custom Silicon to Reconfigurable Logic: The Emergence of FPGAs	4
1.2 AI History	5
1.2.1 Symbolic AI and the Perceptron Era.....	5
1.2.2 Connectionism, Backpropagation, and the AI Winters.....	6
1.2.3 The Deep Learning Revolution: Big Data, GPUs, and Convolutional Neural Networks	7
1.3 The Convergence of Semiconductor Technology and Artificial Intelligence	8
1.3.1 The Catalytic Role of GPU-Accelerated Training.....	9
1.3.2 Architectural Specialization as the Industrial Response.....	9
1.3.3 The FPGA as the Intersection Point.....	11
1.3.4 Quantization as the Bridge Between Domains	12
1.4 The Edge AI Problem	13

1.4.1 The Deployment Gap.....	13
1.4.2 Constraints of the Edge Deployment Environment	14
1.4.3 ShuffleNet V2 as a Principled Response to the Edge AI Problem	16
Chapter 2: The Inference Bottleneck and Platform Selection.....	18
2.1 The Inference Bottleneck.....	18
2.1.1 Arithmetic Intensity and the Roofline Model	18
2.1.2 Bottlenecks in Depthwise Separable Convolution.....	19
2.1.3 Memory Hierarchy as the Dominant Bottleneck	20
2.2 Why Not CPU or GPU?	21
2.2.1 The CPU Architecture Mismatch.....	21
2.2.2 The GPU Architecture Mismatch	22
2.2.3 The Case for FPGA Acceleration	23
2.3 The Specific Application	24
2.3.1 ImageNet Classification as the Target Task	24
2.3.2 Hardware Deployment Context and Inference Protocol	25
Chapter 3: Design Objectives and Constraints	27
3.1 Primary Objectives.....	27
3.2 Secondary Objectives.....	27
3.3 Design Constraints and Scope.....	28
Chapter 4: Related Work.....	30
4.1 FPGA Accelerators for Convolutional Neural Networks	30
4.2 ShuffleNet Architecture and Efficient CNN Design.....	32
4.3 High-Level Synthesis for Neural Network Accelerators	34
Part II: Design and Implementation.....	37
Chapter 5: ShuffleNet V2 Architecture and Quantization Methodology	38
5.1 ShuffleNet V2 Architecture	38

5.1.1 Motivation and Design Principles.....	38
5.1.2 Network Topology	39
5.1.3 The Shuffle Block in Detail	41
5.1.4 Batch Normalization Folding.....	43
5.2 Software Modeling.....	44
5.2.1 Purpose and Methodology	44
5.2.2 Quantization Validation.....	45
5.2.3 Weight File Generation.....	46
5.3 Quantization and the Q6.8 Fixed-Point Format.....	47
5.3.1 Fixed-Point Representation.....	47
5.3.2 Multiply-Accumulate Pipeline.....	50
5.3.3 Per-Layer Width Optimization.....	52
5.3.4 Group 3 Datapath Width Derivation.....	54
5.4 Layer-by-Layer Profile	54
5.4.1 Group 1: Initial Feature Extraction.....	54
5.4.2 Group 2: Shuffle Block Stages.....	55
5.4.3 Group 3: Classification Head.....	56
Chapter 6: Hardware Mapping and Architectural Analysis.....	58
6.1 Model-to-Hardware Mapping.....	58
6.1.1 Mapping Strategy.....	58
6.1.2 Operation-to-Module Assignment	58
6.1.3 Control Hierarchy	59
6.2 Data Flow Analysis.....	62
6.2.1 Intra-Group Data Flow.....	62
6.2.2 Inter-Group Data Flow and Ping-Pong Buffering.....	63
6.2.3 AXI Data Flow.....	63

6.3 Bottleneck Identification.....	64
6.3.1 Computational Bottleneck.....	64
6.3.2 Memory Bottleneck.....	64
6.3.3 Throughput Bottleneck and Pipeline Efficiency.....	65
Chapter 7: RTL Accelerator Design and Implementation	67
7.1 Top-Level Architecture.....	67
7.1.1 System Hierarchy.....	67
7.1.2 Clock and Reset Architecture	68
7.1.3 AXI Interface Architecture	69
7.2 I/O Interface	69
7.2.1 Accelerator Top-Level Ports.....	69
7.2.2 Control-Status Register Interface.....	70
7.2.3 Pixel Data Interface.....	71
7.3 Module Groups	72
7.3.1 Group 1 Module Organization.....	72
7.3.2 Group 2 Module Organization.....	75
7.3.3 Group 3 Module Organization.....	79
7.4 Datapath and Pipeline	83
7.4.1 MAC Unit Pipeline	83
7.4.2 Filter Unit Pipeline.....	84
7.4.3 Sliding Window Pipeline	84
7.4.4 Depthwise Convolution Datapath.....	85
7.5 Memory Architecture.....	86
7.5.1 On-Chip Memory Organization.....	86
7.5.2 Weight ROM Architecture.....	87
7.5.3 Feature Map Buffer Architecture.....	87

7.6 Verification	89
7.6.1 Testbench Architecture	89
7.6.2 Verification Strategy	90
7.7 System Integration	91
7.7.1 Vivado Block Design Integration	91
7.7.2 Software Integration.....	91
Part III: Results and Evaluation	93
Chapter 8: FPGA Deployment and Demonstration	94
8.1 FPGA Hardware Setup.....	94
8.1.1 Target Platform	94
8.1.2 Build Flow	95
8.1.3 Post-Implementation Verification.....	95
8.2 Demonstration Procedure.....	96
8.2.1 Hardware Deployment.....	96
8.2.2 Inference Execution and Expected Output.....	97
8.2.3 Accuracy Validation on Hardware.....	97
Chapter 9: Results and RTL-versus-HLS Comparison.....	99
9.1 RTL Synthesis Results	99
9.1.1 Synthesis and Implementation Configuration.....	99
9.1.2 Resource Utilisation.....	99
9.1.3 Timing Results.....	101
9.1.4 Primitive Instance Counts.....	103
9.2 HLS Implementation Overview.....	103
9.2.1 HLS Methodology.....	103
9.2.2 Combinatorial Explosion Issues.....	106
9.3 HLS Synthesis Results	106

9.3.1 Model Evolution	106
9.3.2 Per-Instance Latency Breakdown	107
9.3.3 Resource Estimation and P&R Feasibility.....	108
9.4 RTL versus HLS Comparison.....	108
9.4.1 Resource Utilisation Comparison	108
9.4.2 Timing and Frequency Comparison.....	110
9.4.3 Latency Comparison	111
9.4.4 Methodology Assessment	111
9.5 Comparison with Software Baseline and Published Accelerators	113
Chapter 10: Discussion and Conclusions.....	116
10.1 Summary of Contributions.....	116
10.2 Analysis of Results.....	117
10.3 Limitations and Future Work.....	119
10.4 Conclusions.....	120
References.....	122

LIST OF FIGURES

Figure 1.1 — Convergence of VLSI scaling and AI algorithmic development (1958–present)	11
Figure 1.2 — MAC complexity of representative CNN architectures versus edge-deployment feasibility boundary. Sources: [10][11][12][15].	15
Figure 2.1 — Roofline model for ShuffleNet V2 layer categories on the ZU19EG at 100 MHz. Sources: [1][2].	19
Figure 2.2 — Comparison of ARM Cortex-A53, NVIDIA Jetson AGX Orin, and ZU19EG RTL accelerator across MAC throughput, platform power, and batch-size-one inference suitability. Sources: [4][7][8].	23
Figure 2.3 — Five-step AXI4-Lite inference protocol between ARM PS host and PL RTL accelerator. Source: [3].	25
Figure 3.1 — Dual-track project structure: primary RTL and secondary HLS objectives converging at Chapter 9 evaluation.	28
Figure 4.1 — Selected FPGA CNN accelerator designs: DSP utilization vs. peak throughput. Sources: [32][37][38].	32
Figure 4.2 — RTL versus HLS methodology comparison across five design dimensions. Sources: [31][44][45][46].	35
Figure 5.1 — ShuffleNet V2 $\times 1.0$ three-group network topology mapped to hardware groups. Source: [15][29].	39
Figure 5.2 — 3×3 convolution as a sliding-window dot product between a filter and the input volume, producing one output feature-map pixel per window position.	40
Figure 5.3 — Max-pooling with a 3×3 window and stride 2, selecting the maximum value within each window.	41
Figure 5.4 — ShuffleNet V2 shuffle unit internal structure: stride-1 configuration with channel split/concat (left) and stride-2 configuration (right).	42

Figure 5.5 — Depthwise 3×3 convolution: each output channel is produced from exactly one input channel, sharply reducing multiply-accumulate cost relative to a standard convolution.	42
Figure 5.6 — 1×1 (pointwise) convolution: each output channel is a weighted combination of all input channels at a given spatial position.....	43
Figure 5.7 — Batch normalization folding derivation: pre-folding convolution+BN cascade (top) and the folded new-weight/new-bias formulas (bottom).	44
Figure 5.8 — Baseline accuracy verification of the pretrained ShuffleNet V2 $\times 1.0$ model on the ImageNet validation set (69.362% top-1), matching the published reference accuracy prior to any quantization.....	45
Figure 5.9 — Software modeling pipeline: BN folding $\rightarrow$ quantization $\rightarrow$ accuracy validation $\rightarrow$ hex weight export. Sources: [29][30].....	45
Figure 5.10 — ImageNet validation sample and its ground-truth/predicted class (259, Samoyed).	46
Figure 5.11 — RTL simulation waveform confirming <code>class_index = 259</code> , matching the software prediction.	46
Figure 5.12 — Fixed-point bit width divided into an integer-length field and a fractional-length field.	48
Figure 5.13 — Q6.8 fixed-point format: 1 sign bit, 6 integer bits, and 8 fractional bits (15 bits total).	48
Figure 5.14 — Dynamic fixed-point representation: two values of equal bit width using different fractional lengths (FL), letting each layer choose its own optimal integer/fractional split.....	49
Figure 5.15 — Fixed-point addition: operands are aligned to a common binary point before adding.....	49
Figure 5.16 — Overflow in fixed-point addition when the true sum requires one more bit than either operand.....	49
Figure 5.17 — Overflow resolution in fixed-point addition via sign-extension of both operands before adding.....	49

Figure 5.18 — Q6.8 bit-field layout (top) and MAC pipeline datapath (bottom). Source: [29].	50
Figure 5.19 — Preparing operands for fixed-point multiplication (top) and an unsigned $\times$ unsigned multiplication worked example (bottom).	51
Figure 5.20 — Signed $\times$ signed fixed-point multiplication: the final partial product is negated to represent the two's-complement operand correctly.....	52
Figure 5.21 — Per-module weight and bias bit-widths selected by the dynamic (per-layer) quantization scheme across Group 1, Group 2, and Group 3.	53
Figure 5.22 — Average-pooling with a 3×3 window, replacing the maximum with the mean of each window.	56
Figure 5.23 — Fully connected layer: a flattened input vector is mapped to per-class scores through a bank of weighted sums.	57
Figure 6.1 — Operation-to-RTL-module mapping with calculated DSP48E2 allocation. Source: [29].....	59
Figure 6.2 — Input and output signals of the top-level accelerator controller, including its handshake with the image source, Groups 1-3, and the ping-pong photo memory.....	60
Figure 6.3 — Accelerator controller state diagram: five states (IDLE, G1, G1_2, G1_2_3, G1_3) with the exact busy-flag and done-signal conditions governing each transition.	61
Figure 6.4 — Accelerator controller status-flag registers: D-flip-flops latching the Group 1/2/3 done pulses, the busy flag, and the ping-pong select signal, ensuring no done pulse is missed while multiple groups are active.....	61
Figure 6.5 — Inference data flow: AXI ingestion $\rightarrow$ Group 1 $\rightarrow$ maxpool_mem $\rightarrow$ Group 2 $\rightarrow$ extra_mem $\rightarrow$ Group 3 $\rightarrow$ CSR. Source: [29].....	62
Figure 6.6 — Bottleneck summary: DSP allocation by group, BRAM by buffer, and critical timing path. Source: [29].	65
Figure 7.1 — Three-level system hierarchy showing module instantiation, clock/reset distribution, and AXI data paths. Source: [29].	67
Figure 7.2 — Complete hierarchy of Group 1, Group 2, and Group 3 blocks composing the accelerator, alongside a summary of the parallelism factor used in every computation core..	68

Figure 7.3 — AXI4-Lite address decode, CSR bit-field layout, and inference protocol timing diagram. Source: [29].....	70
Figure 7.4 — Consolidated per-group module dataflow view spanning all three hardware groups, from the AXI-fed photo_mem input buffer through to the classification output.....	72
Figure 7.5 — Group 1 internal module organization, data widths, and FSM control signals. Source: [29].....	73
Figure 7.6 — Zero-padding of a feature map: the input is bordered with zeros to preserve output dimensions after convolution.....	73
Figure 7.7 — Sliding-window FIFO shift-register architecture with configurable tap positions and zero-padding mux.....	74
Figure 7.8 — State diagram of the Group 1 sliding-window FIFO controller (IDLE → LOAD_ROW → LOAD_WIN → PROCESS).	74
Figure 7.9 — 3×3 filter (weight) memory implementation: per-channel LUT-ROM banks selected through a chain of 4:1 multiplexers.	75
Figure 7.10 — Maxpooling computation core: 24 parallel comparison trees, each finding the maximum of a 3×3 window in one clock cycle.	75
Figure 7.11 — Stride-2 (left) and stride-1 (right) shuffle-block datapaths, showing the two convolution cores reused across all Group 2 blocks.....	76
Figure 7.12 — Group 2 cores-and-memories interconnect: the Shuffling Unit, X_left/X_right/Y feature-map memories, and the shared 1×1 and 3×3 depthwise convolution cores.....	76
Figure 7.13 — Group 2 FIFO register with width-select multiplexers, supporting the four feature-map widths (7, 14, 28, 56) used across the three shuffle-block stages.....	77
Figure 7.14 — Width-select combinational logic driving the Group 2 FIFO multiplexers for the four supported feature-map widths.	77
Figure 7.15 — Group 2 FIFO controller FSM, which adds a LAST_PAD state to the Group 1 design to handle stride-1 bottom-row padding.	78
Figure 7.16 — 3×3 depthwise convolution controller FSM (left) and the channel-shuffle stage sequence across successive shuffle blocks (right).	78

Figure 7.17 — State diagram of the 1×1 convolution controller (7 states), coordinating alternation between the depthwise and pointwise convolution cores.	78
Figure 7.18 — Group 2 controller FSM sequencing Stage 2, Stage 3, and Stage 4 shuffle-block execution.....	79
Figure 7.19 — Group 3 data flow: the feature map produced by Group 2 is passed through Extra Memory into the 1×1 convolution, average-pooling, fully connected, and classification stages.....	80
Figure 7.20 — Group 3 block diagram, showing the 1×1 convolution core, average-pooling core, FC core, classification unit, and the Group 3 controller coordinating them.....	80
Figure 7.21 — Group 3 1×1 convolution unit pipeline (16 parallel filters, 29-channel accumulation), with its parallelism summary.	81
Figure 7.22 — Average-pooling core datapath: multiply-by- $1/49$ (shift-add), accumulate over 49 cycles, and quantize.	81
Figure 7.23 — Register-file write structure bridging average-pooling and the fully connected layer: 64 banks of 16 registers, selected by a one-hot shift register.	82
Figure 7.24 — Fully connected layer block diagram: parallel multiply-accumulate lanes feeding an adder tree, accumulator, and quantizer, with its parallelism summary.	82
Figure 7.25 — Classification core block diagram: a running-maximum comparator that tracks the highest class probability and its index without a separate sort step.....	82
Figure 7.26 — Input and output control signals of the Group 3 controller and its interfaces to the Fully-Connected, Register File, 1×1 Convolution, Average-Pooling, Classification, and Extra Memory blocks.....	83
Figure 7.27 — Filter unit pipeline: MAC array $\rightarrow$ adder tree $\rightarrow$ accumulator $\rightarrow$ bias add $\rightarrow$ quantizer. Source: [29].....	84
Figure 7.28 — On-chip memory architecture: BRAM tile allocation (left) and read-access port types by memory (right). Source: [29].....	87
Figure 7.29 — Internal dual-bank organization of photo_mem, showing read/write address multiplexing and per-channel write-enable demultiplexing.	88
Figure 7.30 — Bottom-up testbench architecture: 48 testbenches across 6 directories, 50 RTL modules. Source: [29].	89

Figure 7.31 — RTL verification strategy: the Python golden model generates matched input/output reference files, which drive a self-checking testbench against the RTL implementation.....	91
Figure 7.32 — Vivado block design integration (left) and PS-side inference software flow (right). Source: [29].....	92
Figure 8.1 — Target platform specifications (iWave ZU19EG Development Kit) and post-implementation resource utilisation summary. Source: [29].	94
Figure 8.2 — Split synthesis build flow with timings (left) and hardware demonstration procedure (right). Source: [29].....	96
Figure 9.1 — Post-implementation resource utilisation breakdown by category: CLB LUTs, DSP48E2, BRAM tiles, and flip-flops. Source: [29].....	100
Figure 9.2 — Clock period budget decomposition and critical path trace (post-implementation, clk_100m domain). Source: [29].	102
Figure 9.3 — HLS C++ model structure: shared memories, per-layer computation functions, and the shuffle-group function composing the top-level model.	105
Figure 9.4 — HLS C++ code structure for 3×3 convolution: nested loops over filters, output rows/columns, input channels, and kernel taps.....	105
Figure 9.5 — HLS Model 4 per-instance latency breakdown (left) and four-model cycle/latency comparison including the board-verified RTL result (right). Sources: [29][31].	107
Figure 9.6 — RTL (measured) vs HLS Model 4 (estimated) resource comparison across five dimensions. Sources: [29][31][46].....	110
Figure 10.1 — Four thesis contributions with key evidence, including board-verified performance, and source references.....	117
Figure 10.2 — Design objectives vs achieved results.	118
Figure 10.3 — Future work roadmap: items resolved since submission (top), remaining near-term work (middle), and longer-term extensions (bottom).....	120

LIST OF TABLES

Table 5.1: Layer-by-Layer Network Architecture of ShuffleNet V2 $\times$ 1.0	40
Table 5.2: Fixed-point versus floating-point representation, compared across hardware complexity, area, propagation delay, power consumption, memory, and accuracy.	48
Table 5.3: Quantization sweep across candidate bit-width and fractional-length combinations, evaluated on the full 50,000-image ImageNet validation set.....	52
Table 5.4: Per-Layer Fixed-Point Feature Map and Weight Specifications.....	53
Table 5.5: Layer-by-Layer DSP Budget Summary.....	57
Table 7.1: Control-Status Register (CSR) Field Definitions	70
Table 9.1: Post-Implementation Chip Resource Utilisation	100
Table 9.2: Post-Implementation Timing Closure Summary.....	101
Table 9.3: HLS Model Evolution Across Four Optimization Stages	105
Table 9.4: HLS C-Synthesis Results, Models 2, 3, and 4.....	107
Table 9.5: RTL versus HLS Models 3 and 4, Key Resource Dimensions.....	109
Table 9.6: Post-implementation on-chip power breakdown by component (total 6.816 W). ..	113
Table 9.7: Qualitative comparison of the RTL and HLS design methodologies across utilization, development effort, controllability, verification, and performance.....	113
Table 9.8: RTL Compared Against the Original ShuffleNet V2 Software Model	113
Table 9.9: Frame-Rate Comparison Against Other Published FPGA CNN Accelerators.....	114

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
BN	Batch Normalization
BRAM	Block Random Access Memory
CMOS	Complementary Metal-Oxide-Semiconductor
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSR	Control and Status Register
DDR	Double Data Rate (synchronous dynamic random access memory)
DL	Deep Learning
DNN	Deep Neural Network
DSP	Digital Signal Processor (slice)
DW	Depthwise (convolution)
FC	Fully Connected (layer)
FF	Flip-Flop
FIFO	First-In-First-Out (buffer)
FPGA	Field-Programmable Gate Array
FPS	Frames Per Second
FSM	Finite State Machine
GPU	Graphics Processing Unit
HDL	Hardware Description Language
HLS	High-Level Synthesis
IC	Integrated Circuit
IP	Intellectual Property (core)
JTAG	Joint Test Action Group (IEEE 1149.1 test access port)
LUT	Look-Up Table
MAC	Multiply-Accumulate

ML	Machine Learning
MPSoC	Multiprocessor System-on-Chip
NPU	Neural Processing Unit
PL	Programmable Logic
PS	Processing System
PW	Pointwise (convolution)
Q6.8	Fixed-point format with 6 integer bits and 8 fractional bits
ReLU	Rectified Linear Unit
RTL	Register-Transfer Level
SoC	System-on-Chip
Top-1	Top-1 classification accuracy
VLSI	Very-Large-Scale Integration
WHS	Worst Hold Slack
WNS	Worst Negative Slack (setup)

PART I: INTRODUCTION

Chapter 1: Historical Foundations

1.1 VLSI History

Every modern computing system, from a handheld smartphone to a data-center accelerator, rests on more than seven decades of progress in solid-state electronics. The discipline broadly known as Very-Large-Scale Integration (VLSI) describes the fabrication of integrated circuits containing hundreds of millions to billions of transistors on a single piece of silicon. Tracing how this discipline evolved — from discrete transistors, to small integrated circuits, to the reconfigurable fabrics used in this thesis — explains why a Field-Programmable Gate Array (FPGA) is a viable target for a real-time convolutional neural network accelerator today, and why this option was simply unavailable to hardware designers for most of the history described below.

1.1.1 From the Transistor to the Integrated Circuit

The starting point of modern electronics is the invention of the point-contact transistor at Bell Telephone Laboratories in December 1947 by John Bardeen and Walter Brattain, working under William Shockley [1]. Shockley followed with the junction transistor and a comprehensive theory of p-n junction devices that explained the physical operation of these new solid-state switches and placed transistor design on a rigorous theoretical footing [2]. Unlike the vacuum tubes they replaced, transistors were small, robust, and consumed little power, but for the first decade after their invention they were still manufactured, tested, and wired together as discrete components — a process that limited circuit complexity and reliability as the number of components grew.

The next decisive step was the integrated circuit (IC): fabricating multiple transistors, resistors, and the interconnections between them on a single piece of semiconductor material. Jack Kilby at Texas Instruments built the first working integrated circuit in 1958, a phase-shift oscillator combining several components on a single germanium chip with hand-wired connections [3]. Robert Noyce at Fairchild Semiconductor independently developed a monolithic integrated circuit using the planar process, which used a layer of silicon dioxide and photolithographically defined metal traces to interconnect components on the chip surface. Noyce's approach proved far more practical to manufacture at scale and became the basis of the modern integrated circuit industry.

Early integrated circuits in the 1960s contained only a handful of transistors — a regime later termed small-scale integration (SSI). Manufacturing process improvements rapidly pushed transistor counts into the hundreds (medium-scale integration, MSI) and then the thousands (large-scale integration, LSI) by the early 1970s; the Intel 4004, released in 1971, packed roughly 2,300 transistors onto a single chip and is widely regarded as the first commercial microprocessor. By the end of the 1970s, chip complexity had grown to the point that ad hoc, transistor-by-transistor circuit design no longer scaled. Carver Mead and Lynn Conway's structured design methodology, codified in their 1980 textbook, gave designers a disciplined way to compose circuits containing tens of thousands of transistors from well-defined logic and layout primitives, and the term Very-Large-Scale Integration entered common use to describe this new regime [4].

1.1.2 Moore's Law and the Scaling Era

In 1965, Intel co-founder Gordon Moore observed that the number of components that could be economically placed on an integrated circuit had doubled approximately every year since 1959, and predicted that this trend would continue for at least a decade [5]. The observation, later popularized as Moore's Law, proved remarkably durable: transistor counts continued to double every eighteen months to two years for more than four decades, driven by successive generations of photolithography that shrank the minimum feature size of a transistor.

The physical reason this scaling translated into faster, more efficient circuits — rather than simply more of the same — was articulated by Robert Dennard and his colleagues at IBM in 1974. Dennard scaling showed that if a transistor's dimensions, voltage, and doping concentrations were scaled down by a constant factor, the resulting transistor would switch faster while dissipating less power per switching event, so that overall power density on the chip remained roughly constant even as transistor count increased [6]. Moore's Law and Dennard scaling together meant that each new process generation delivered chips that were simultaneously smaller, faster, and more power-efficient — a virtuous cycle that took microprocessors from the 2,300-transistor Intel 4004 in 1971 to devices with tens of millions of transistors by the late 1990s.

This scaling era began to slow in the mid-2000s, as transistor dimensions approached atomic scales and leakage currents grew large enough that voltage could no longer be reduced in step with feature size. The resulting breakdown of Dennard scaling — sometimes called the power wall — meant that adding more transistors no longer guaranteed proportionally lower power

per operation. The industry's response was twofold: general-purpose processors shifted from single fast cores to multiple, more power-efficient cores running in parallel, and a growing share of computation moved toward specialized, domain-specific hardware that could extract far more performance per watt from a fixed transistor budget than a general-purpose CPU. This second response — application-specific and reconfigurable hardware acceleration — is the direct ancestor of the design philosophy underlying this thesis, and it is also why the device targeted in this work, the Zynq UltraScale+ ZU19EG, integrates 522,720 look-up tables and 1,968 dedicated DSP slices in its programmable logic fabric rather than relying solely on its embedded general-purpose ARM Cortex-A53 processors to perform inference.

1.1.3 From Custom Silicon to Reconfigurable Logic: The Emergence of FPGAs

As transistor budgets grew, so did the cost and risk of designing a fully custom application-specific integrated circuit (ASIC): a single error in an ASIC's photomasks could only be corrected by an expensive and slow re-fabrication cycle. Through the 1970s, simpler programmable alternatives emerged to give designers some of the benefits of custom logic without committing to silicon at design time — programmable read-only memories (PROMs) and programmable logic arrays (PLAs) allowed a fixed grid of gates to be configured, after manufacture, by blowing fuses or programming a small memory array. These devices evolved into programmable array logic (PAL) and, later, complex programmable logic devices (CPLDs), but all remained limited to a few thousand equivalent logic gates.

The Field-Programmable Gate Array (FPGA) generalized this idea to a much larger scale. In 1985, Xilinx — co-founded the previous year by Ross Freeman and Bernard Vonderschmitt — introduced the XC2064, an array of configurable logic blocks connected through a programmable interconnect fabric, which could be reconfigured an unlimited number of times after fabrication [7]. Rather than committing to a fixed gate-level netlist at the foundry, a designer could now describe digital logic in a hardware description language, synthesize it onto the same physical silicon device, and reconfigure that same chip the next day with an entirely different design. This came at a cost in area, raw clock frequency, and power efficiency relative to an equivalent ASIC, but it eliminated fabrication turnaround time and non-recurring engineering cost almost entirely, making custom digital hardware design accessible for far smaller production volumes.

The same Moore's-Law-driven scaling described in Section 1.1.2 applied to FPGAs just as it did to microprocessors and memory: successive device families grew from a few thousand

configurable logic gates in 1985 to today's UltraScale+ generation, in which a single device combines hundreds of thousands of look-up tables, thousands of dedicated multiply-accumulate (DSP) slices, megabytes of distributed on-chip Block RAM, and — in MPSoC variants such as the ZU19EG used in this work — a full multi-core ARM processing system on the same die. This combination of programmability, scale, and tight integration with a conventional processor is precisely what makes the FPGA an attractive accelerator target for the convolutional neural network workloads introduced in Section 1.2, and it is the device class on which the RTL and HLS accelerators described in this thesis are built.

VLSI's trajectory — from individual transistors, to densely integrated but fixed-function circuits, to reconfigurable fabrics that can be repurposed after manufacture — created the physical substrate on which nearly all of modern computing depends. It did not, by itself, create a reason to build hardware as specialized as the ShuffleNet accelerator developed in this thesis. That motivation comes from a second and largely independent line of progress: the decades-long development of artificial intelligence algorithms, and in particular the emergence of deep convolutional neural networks whose computational appetite eventually outgrew what general-purpose processors could deliver. Section 1.2 traces that history.

1.2 AI History

The preceding section traced how Very-Large-Scale Integration created hardware of extraordinary density and, eventually, reconfigurability. This section turns to the second, largely independent thread that converges in this thesis: the algorithmic history of artificial intelligence (AI), and in particular the rise of the deep convolutional neural networks that the accelerator described in later chapters is built to execute. Unlike VLSI, whose progress was driven almost entirely by manufacturing physics, AI's history has alternated between waves of optimism and periods of retrenchment — often called AI winters — as the gap between algorithmic ambition and available data and compute widened or narrowed.

1.2.1 Symbolic AI and the Perceptron Era

Alan Turing opened the formal study of machine intelligence in 1950 by proposing an operational test for whether a machine could be said to think, sidestepping philosophical debate about the nature of thought in favor of a measurable behavioral criterion [8]. The field acquired its name six years later at the 1956 Dartmouth Summer Research Project on Artificial Intelligence, organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude

Shannon, which brought together the researchers who would define the discipline's first two decades. Early AI research split along two broad lines: symbolic approaches that represented knowledge as logical rules and searched over them explicitly, and connectionist approaches that represented knowledge implicitly in the weights of networks of simple, neuron-like units.

The most influential early connectionist model was Frank Rosenblatt's perceptron, a single trainable linear threshold unit loosely inspired by biological neurons, which could learn to classify simple patterns directly from labeled examples [9]. The perceptron generated substantial early enthusiasm, but Marvin Minsky and Seymour Papert showed rigorously in 1969 that a single-layer perceptron could not represent functions that are not linearly separable — the simple exclusive-OR function being the canonical example — and expressed skepticism that multi-layer networks could be trained effectively in practice [10]. Combined with a more critical funding climate later formalized in the UK by the 1973 Lighthill Report, this analysis is widely credited with triggering the first AI winter: a sustained pullback in funding and research attention, particularly for neural-network-based methods, that lasted through most of the 1970s.

The decline of perceptron-based research did not end interest in artificial intelligence as a whole; it shifted attention toward symbolic, knowledge-based methods instead. Rule-based expert systems — such as MYCIN for medical diagnosis and DENDRAL for chemical structure analysis — demonstrated that software encoding human expert knowledge as explicit if-then rules could match specialist-level performance within narrow domains, fueling a second wave of commercial AI investment through the early-to-mid 1980s, including a dedicated market for Lisp machine workstations built specifically to run these systems. This boom proved short-lived: expert systems were brittle outside the narrow domain each had been built for, expensive to extend rule by rule, and could not compete with general-purpose workstations once those caught up in price and performance. The ensuing collapse of the expert-systems market and the Lisp machine industry in the late 1980s produced the second, and most severe, of the AI winters — a pattern of sharply reduced funding and research attention that recurred whenever the field's commercial promises outran its delivered results.

1.2.2 Connectionism, Backpropagation, and the AI Winters

Interest in neural networks revived in the mid-1980s once practical algorithms for training multi-layer networks became available. David Rumelhart, Geoffrey Hinton, and Ronald Williams popularized the backpropagation algorithm, which efficiently computes the gradient

of a network's error with respect to every weight in every layer by propagating error signals backward through the network, finally giving Minsky and Papert's multi-layer networks a practical training method [11]. Yann LeCun and colleagues then combined backpropagation with two architectural ideas — local receptive fields and shared weights across spatial locations — to build the convolutional neural network, demonstrating LeNet-5 on handwritten digit and document recognition tasks by the late 1990s [12]. The convolution, pooling, and weight-sharing operations introduced in this work remain the architectural foundation of every convolutional network used today, including the ShuffleNet V2 network targeted by this thesis.

Despite this progress, neural networks remained a minority approach through the 1990s and early 2000s, overshadowed by methods such as support vector machines that required far less labeled data and computation to train reliably. Three constraints held deeper networks back in particular: the labeled datasets of the era were too small to train large models without overfitting, the central processing units (CPUs) available for training were several orders of magnitude too slow for networks with many layers, and deep stacks of trainable layers suffered from a vanishing-gradient problem that made backpropagation increasingly ineffective as depth increased. The combined effect was a quieter, more localized second winter specific to neural-network research, even as machine learning as a broader field continued to advance through other methods.

1.2.3 The Deep Learning Revolution: Big Data, GPUs, and Convolutional Neural Networks

Two enabling conditions converged by the early 2010s and removed the constraints described above almost simultaneously. The first was the availability of very large labeled datasets, epitomized by ImageNet, a hierarchically organized collection of more than a million labeled images spanning thousands of object categories, assembled specifically to give the field a large-scale benchmark for visual recognition [13]. The second was the repurposing of graphics processing units (GPUs) — designed for real-time 3D rendering — as massively parallel general-purpose numerical engines, which happened to be extremely well suited to the dense matrix multiplications that dominate neural network training.

These two factors came together decisively in 2012, when Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton trained a deep convolutional network, later named AlexNet, on two consumer GPUs and won the ImageNet Large Scale Visual Recognition Challenge by a margin that no hand-engineered computer vision method had ever approached [14]. The result is widely regarded as the start of the modern deep learning era. A rapid succession of deeper and

more accurate architectures followed within a few years, culminating in residual networks, which used skip connections to make networks well over a hundred layers deep trainable for the first time and pushed image classification accuracy past the level of careful human annotators on the ImageNet benchmark [15].

The years immediately following AlexNet were dominated by architectures pursuing accuracy almost exclusively. VGGNet showed that simply stacking many small 3×3 convolutional filters into a much deeper network improved accuracy in a remarkably simple, uniform way [16], while the Inception family introduced multi-branch modules that processed the same input at several filter sizes in parallel before concatenating the results, improving accuracy without a proportional increase in depth [17]. A smaller, parallel line of work asked the opposite question — how small an accurate network could be made — exemplified by SqueezeNet, which matched AlexNet-level accuracy with fifty times fewer parameters through aggressive use of 1×1 convolutions [18]. It is this efficiency-oriented lineage, rather than the accuracy-at-any-cost lineage that produced ever-deeper networks such as VGGNet and the residual networks described above, that is the direct ancestor of the MobileNet and ShuffleNet families discussed in Section 1.3, and ultimately of the ShuffleNet V2 network implemented in this thesis.

This rapid accuracy gain came at a steep and largely unaddressed computational cost: each new architecture achieved higher accuracy chiefly by adding more layers, more parameters, and more floating-point operations per inference, with little regard for whether the resulting network could run anywhere other than a multi-GPU server. By the mid-2010s, the most accurate published networks required billions of multiply-accumulate operations to classify a single image — a computational appetite that general-purpose processors, and even the GPUs that had enabled the revolution in the first place, could not satisfy within the latency and power budgets of a battery-powered or embedded device. Closing that gap required rethinking network architectures explicitly for computational efficiency, and it required hardware capable of executing them efficiently outside the data center.

1.3 The Convergence of Semiconductor Technology and Artificial Intelligence

The parallel trajectories of semiconductor engineering and artificial intelligence research did not remain parallel indefinitely. By the second decade of the twenty-first century, the two disciplines had converged into a single, mutually reinforcing research program. VLSI technology provided the raw computational substrate without which modern AI algorithms

could not execute; AI workloads, in turn, provided the economic and scientific motivation that drove the next generation of silicon specialization. Understanding this convergence — its historical moment, its physical causes, and its architectural consequences — is essential context for the design decisions that underpin the present work.

1.3.1 The Catalytic Role of GPU-Accelerated Training

The event conventionally cited as the inflection point of the convergence is the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) of 2012, in which a deep convolutional neural network (CNN) trained on two NVIDIA GTX 580 graphics processing units (GPUs) achieved a top-5 error rate of 16.4%, reducing the previous state-of-the-art by more than 10 percentage points [14]. The significance of this result was not confined to computer vision: it demonstrated, for the first time at scale, that the massive parallelism inherent in commodity graphics hardware could be repurposed to train neural networks orders of magnitude larger than those tractable on central processing units (CPUs). The GPU had not been designed for machine learning; it had been designed to rasterize polygons. Yet the arithmetic intensity of 3D rendering — thousands of identical floating-point operations executed in parallel across fragments — proved structurally identical to the arithmetic intensity of neural network forward and backward passes: thousands of multiply-accumulate (MAC) operations executed across neuron activations and weight matrices.

This architectural coincidence revealed a deeper principle that would come to dominate AI hardware design: neural network inference and training are, at their core, linear-algebra workloads. Matrix–vector multiplication, convolution, and batch normalization can all be expressed as dense sequences of MAC operations, the same primitive that dominates digital signal processing, graphics rendering, and scientific simulation. The implication was that dedicated hardware optimized for MAC throughput — hardware designed by VLSI engineers, not by machine learning researchers — would deliver superior performance and efficiency compared to general-purpose processors for which the MAC operation was merely one instruction among many [2].

1.3.2 Architectural Specialization as the Industrial Response

The semiconductor industry's response to the convergence was rapid and decisive. Google introduced its Tensor Processing Unit (TPU) in 2016, a custom application-specific integrated circuit (ASIC) built around a 256×256 systolic array capable of executing 65,536 MAC

operations per clock cycle [19]. NVIDIA responded by augmenting its GPU architecture with Tensor Cores — dedicated 4×4 matrix-multiply units — that substantially increased the arithmetic throughput available to deep learning frameworks while preserving programmability for research workloads [20]. ARM introduced the Ethos neural processing unit (NPU) specification for mobile system-on-chip (SoC) designs, enabling AI inference within the stringent power envelope of a smartphone [21]. In each case, the architectural innovation was structurally the same: replace general-purpose arithmetic datapaths with narrower, lower-precision, higher-throughput MAC arrays, and redesign the surrounding memory hierarchy to feed those arrays without stalls.

This specialization was not merely an incremental extension of existing microarchitecture. It reflected a fundamental renegotiation of the design tradeoffs that VLSI engineers have always faced. Conventional processor design prioritizes instruction diversity, branch prediction, out-of-order execution, and cache coherence — mechanisms that impose area and power overhead in order to accelerate sequential, irregular workloads. AI inference workloads are, by contrast, highly regular: control flow is deterministic, data access patterns are predictable, and arithmetic precision requirements are relaxed. Empirical work demonstrated that 8-bit integer arithmetic, and in some cases even lower-precision fixed-point formats, preserved model accuracy within acceptable bounds while reducing silicon area and energy consumption by a factor of approximately four compared to 32-bit floating-point [22]. This observation — that neural network weights and activations could be represented with far fewer bits than classical signal processing had assumed — is the foundational insight underlying the fixed-point design methodology employed in the present work, specifically the Q6.8 format described in Chapter 5.

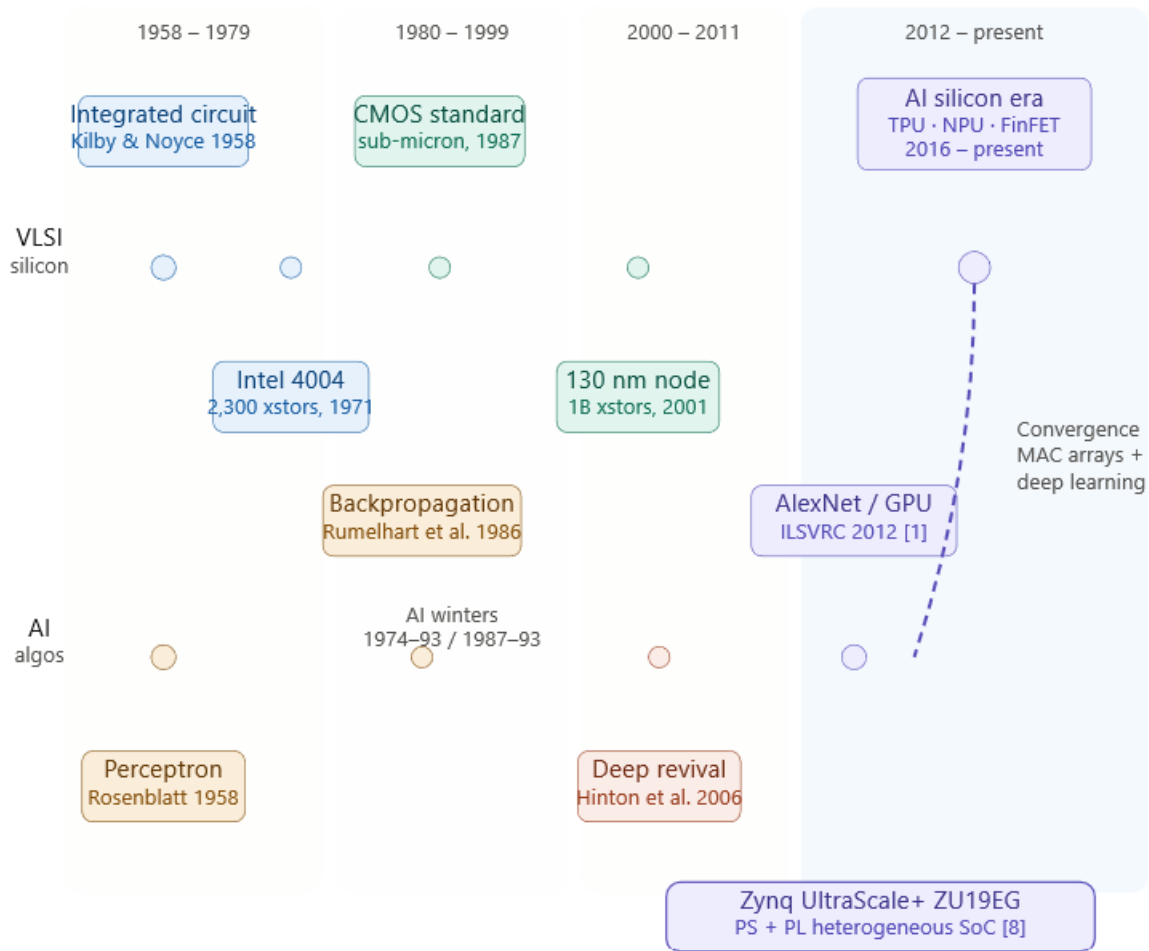


Figure 1.1 — Convergence of VLSI scaling and AI algorithmic development (1958–present)

1.3.3 The FPGA as the Intersection Point

Within this broader convergence, field-programmable gate arrays (FPGAs) occupy a distinctive position. Unlike ASICs, FPGAs can be reconfigured after manufacture, permitting the designer to implement exactly the arithmetic datapath, memory topology, and control logic that a specific neural network architecture requires, without the non-recurring engineering (NRE) cost or the long fabrication lead time of a custom chip. Unlike GPUs, FPGAs operate at comparatively modest clock frequencies — typically 100 MHz to 300 MHz on modern high-end devices — but achieve high effective throughput through spatial parallelism: hundreds of independent MAC units executing simultaneously on dedicated silicon resources, with no shared-memory contention and no thread-scheduling overhead [7].

The Xilinx Zynq UltraScale+ family, the target platform of the present work, exemplifies the full convergence of heterogeneous computing on a single device. The ZU19EG integrates an ARM Cortex-A53 quad-core application processor, capable of running a full Linux operating

system for host-side orchestration, with a programmable logic (PL) fabric containing 522,720 look-up tables (LUTs), 1,968 DSP48E2 multiply-accumulate slices, and 984 block RAM tiles [23]. This integration eliminates the bus bottleneck that would otherwise arise if the neural network accelerator and the host processor resided on separate chips: data transfers between the ARM processing system (PS) and the PL fabric traverse a high-bandwidth AXI interconnect on-die, avoiding the latency and energy cost of an off-chip PCIe or Ethernet interface. The present design exploits this integration directly: the ARM processor writes pixel data to the accelerator's image buffer over an AXI4-Lite slave interface at base address 0xA0000000, initiates inference by asserting a single control-status register (CSR) bit, and polls for completion — a transaction that would require off-chip DMA and interrupt management on a discrete CPU-plus-FPGA platform, but reduces here to a handful of memory-mapped register accesses.

1.3.4 Quantization as the Bridge Between Domains

A final dimension of the convergence deserves emphasis, because it directly motivates the fixed-point arithmetic methodology of the present work. Classical VLSI design operates in a world of integers: logic gates, registers, and arithmetic circuits manipulate binary words of fixed width. Classical machine learning, by contrast, developed in a world of floating-point arithmetic, inheriting the scientific computing assumption that 32-bit or 64-bit IEEE 754 representation was a prerequisite for numerical stability. The convergence of the two domains forced a confrontation with this assumption. Floating-point arithmetic is expensive in silicon: a single-precision multiplier consumes approximately ten times the area of an equivalent 8-bit integer multiplier and substantially more dynamic power [22]. At the scale of a neural network with millions of MAC operations per inference pass, this overhead is prohibitive for embedded deployment.

The resolution was quantization: the systematic mapping of floating-point weights and activations to fixed-point representations of reduced bit width, with careful attention to scale factors and saturation thresholds required to preserve model accuracy [22]. In the present work, quantization is implemented using a signed Q6.8 format — 15 bits total, comprising one sign bit, six integer bits, and eight fractional bits — which represents values in the integer range $[-16384, +16383]$, corresponding to the decimal interval $[-64, +63.996]$. This format was selected to balance the dynamic range required by ShuffleNet V2's activation distributions against the resource constraints of the ZU19EG fabric. The design further applies per-layer

width optimization, reducing feature map precisions to 8, 10, 12, and 9 bits for the input image buffer, Group 1 output, Group 2 output, and Group 3 output respectively, and adjusting weight widths between 9 and 15 bits according to the precision sensitivity of each layer. These optimizations are the direct product of the convergence: they require simultaneous knowledge of the numerical behavior of the neural network and the arithmetic resources available on the target FPGA.

The convergence of VLSI technology and artificial intelligence is therefore not merely a historical coincidence. It is a substantive architectural alignment between the computational primitives of silicon — the MAC unit, the shift register, the block RAM, the fixed-point adder tree — and the computational primitives of modern deep learning. The design presented in this thesis is a concrete realization of that alignment: a custom RTL accelerator that maps the ShuffleNet V2 inference graph onto the spatial resources of a heterogeneous SoC FPGA, using fixed-point arithmetic to bridge the two domains at every stage of the datapath.

Section 1.3 has established the historical and architectural context for the present work, tracing the convergence of semiconductor scaling and AI algorithmic development from their parallel origins through the GPU-accelerated deep learning revolution to the current era of domain-specific silicon. Section 1.4 examines the specific engineering problem that this convergence has created at the network edge, where the computational demands of state-of-the-art inference models must be reconciled with the power, latency, and cost constraints of embedded deployment.

1.4 The Edge AI Problem

1.4.1 The Deployment Gap

The convergence documented in Section 1.3 produced deep learning models of extraordinary capability, but it simultaneously exposed a structural tension that defines the current frontier of hardware research: the most capable models are also the most computationally expensive, and the environments in which real-time AI inference is most valuable — embedded systems, mobile devices, autonomous platforms, industrial controllers — are precisely the environments that can least afford that computational expense. This gap between the arithmetic demands of state-of-the-art neural networks and the resource envelope of edge hardware is referred to throughout this thesis as the edge AI problem.

To appreciate the scale of the problem, it is instructive to consider the arithmetic footprint of the models that established the utility of deep learning. The AlexNet architecture that won ILSVRC 2012 required approximately 724 million MAC operations per inference pass on a 227×227 pixel image [14]. The VGG-16 network, which achieved further accuracy improvements in 2014, required approximately 15.5 billion MAC operations per image [16]. The ResNet-50 architecture, which introduced residual connections to enable much deeper networks, required approximately 3.9 billion MAC operations [15]. Each of these numbers assumes a single forward pass at batch size one — the inference scenario, not the training scenario, and not the large-batch throughput scenario for which GPU architectures are optimized. On an embedded ARM Cortex-A53 processor running at 1.2 GHz with no hardware acceleration, executing 3.9 billion scalar MAC operations at single-issue throughput would require on the order of several seconds per frame — far beyond the latency budget of any real-time application.

1.4.2 Constraints of the Edge Deployment Environment

Edge AI systems are characterized by a set of constraints that do not apply to cloud or datacenter deployments, and which collectively define the design space within which an embedded accelerator must operate.

Power envelope. Industrial embedded platforms typically operate under power budgets of 5 W to 25 W for the entire system board, including the processor, memory, peripherals, and cooling. A high-end datacenter GPU such as the NVIDIA A100 dissipates 400 W at peak compute; even the more modest NVIDIA Jetson AGX Orin SoM, a product specifically designed for edge AI, specifies a maximum power mode of 60 W [24]. For battery-operated or passively cooled embedded systems, these figures are unacceptable. The relevant metric for edge deployment is not throughput per second but throughput per watt — and, at the system level, energy per inference frame, which determines how many inferences can be completed on a fixed energy budget.

Latency budget. Many edge AI applications impose hard real-time constraints. An autonomous vehicle perception system must complete object detection within the inter-frame interval of its camera, typically 33 ms at 30 frames per second (fps). An industrial quality-inspection system must classify a component before it exits the inspection window on a production line, which may be as short as 10 ms. A medical imaging system must deliver a preliminary classification to clinician before the next scan acquisition. These latency requirements are not negotiable;

they are determined by physical processes external to the computing system, and no amount of batching or throughput optimization can substitute for low single-image latency.

Memory bandwidth and capacity. Edge platforms rarely have access to the high-bandwidth memory (HBM) or multi-channel DDR4/DDR5 interfaces available to datacenter accelerators. A typical embedded SoC provides a single 32-bit or 64-bit DDR4 interface at 2133 MHz, yielding a peak bandwidth of approximately 17 GB/s to 34 GB/s — compared to 900 GB/s for the NVIDIA A100's HBM2e [25]. For neural networks whose weight tensors exceed the on-chip SRAM capacity, this bandwidth limitation becomes the dominant performance bottleneck, rendering arithmetic throughput improvements largely irrelevant. The design strategy for edge AI accelerators must therefore prioritize weight reuse — keeping weight tensors in on-chip memory for as many MAC operations as possible — over raw arithmetic parallelism.

Deployment cost. Datacenter AI accelerators are deployed in units of thousands, amortizing the NRE cost of ASIC design over a large production volume. Edge AI devices are frequently deployed in units of tens to hundreds in specialized industrial, medical, or research contexts, where ASIC development is economically unjustifiable. FPGAs address this constraint directly: they provide custom silicon performance without custom silicon fabrication, making them the economically rational choice for low-to-medium volume edge AI deployment [7].

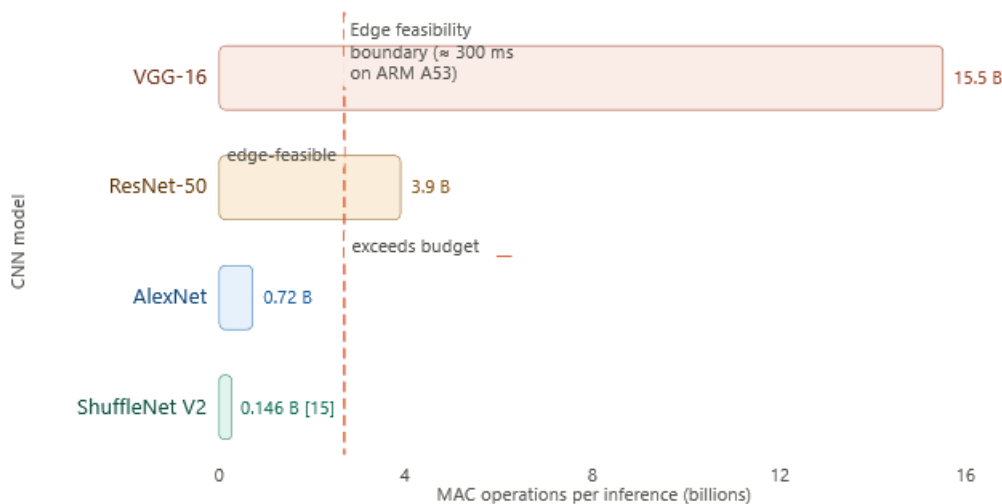


Figure 1.2 — MAC complexity of representative CNN architectures versus edge-deployment feasibility boundary. Sources: [10][11][12][15].

1.4.3 ShuffleNet V2 as a Principled Response to the Edge AI Problem

The ShuffleNet V2 architecture, the neural network at the center of the present work, was designed explicitly to address the edge AI problem at the algorithmic level. Published by Ma et al. in 2018 [26], ShuffleNet V2 introduced a set of practical design guidelines derived from direct analysis of memory access cost and arithmetic intensity on real hardware — not from theoretical FLOPs counts, which the authors demonstrated to be a poor proxy for actual runtime. The guidelines include: equalizing input and output channel widths within each block to minimize memory access cost; avoiding excessive use of group convolutions, which fragment memory access patterns; reducing the number of network branches to decrease synchronization overhead; and minimizing element-wise operations such as addition across branches, which carry low arithmetic intensity relative to their memory footprint [26].

The resulting architecture achieves a top-1 accuracy of 69.36% on ImageNet [26] — competitive with ResNet-50 at approximately one-fifth the arithmetic cost — by combining depthwise separable convolution with a channel split-and-shuffle operation that redistributes feature information across channel groups without the computational expense of a full pointwise convolution. Depthwise convolution applies a single 3×3 filter to each input channel independently, reducing the MAC count from $O(C^2)$ to $O(C)$ relative to a standard convolution of the same kernel size. The channel shuffle operation that follows is implemented in hardware as a simple data reordering — a permutation of pointers or address offsets rather than an arithmetic operation — making it essentially free in a custom RTL implementation.

For the present work, ShuffleNet V2 $\times 1.0$ was selected because its architecture maps naturally onto the resource profile of the Zynq UltraScale+ ZU19EG. The network's three-stage structure — an initial 3×3 convolution with max pooling, followed by sixteen shuffle blocks organized in three stages, followed by a final 1×1 convolution, global average pooling, and a fully connected classifier — decomposes cleanly into three independent hardware groups that can be synthesized, verified, and pipelined separately. The network's emphasis on depthwise separable convolution aligns with the ZU19EG's DSP48E2 resource profile: depthwise convolution requires a large number of independent MAC units operating in parallel on single-channel data, precisely the workload for which the DSP slice array — 1,968 units on the ZU19EG — is most efficient. The post-implementation synthesis results confirm this alignment: 1,550 DSP48E2 slices are consumed (78.76% utilization) from a calculated budget of 1,930 across all layers, with all 236,752 timing endpoints meeting the 100 MHz constraint at a worst-case slack of +0.096 ns.

The edge AI problem is thus both the motivation for and the constraint on the design presented in this thesis. It establishes the requirement for a custom hardware accelerator — because software execution on a general-purpose processor cannot meet the latency and power requirements — and it specifies the design methodology: fixed-point arithmetic, on-chip weight storage, spatial parallelism, and pipelined group execution. The remainder of this thesis develops each of these design choices in detail, beginning in Chapter 2 with a formal analysis of the inference bottleneck that makes hardware acceleration necessary.

Section 1.4 has established the edge AI problem as a structural constraint arising from the mismatch between the arithmetic demands of state-of-the-art neural networks and the power, latency, and memory-bandwidth limitations of embedded deployment environments. The selection of ShuffleNet V2 as the target network has been motivated by its principled optimization for precisely this constraint. Chapter 2 proceeds to analyze the inference bottleneck in quantitative terms, establishing the arithmetic and memory-access characteristics that drive the architectural decisions of the RTL design.

Chapter 2: The Inference Bottleneck and Platform Selection

2.1 The Inference Bottleneck

2.1.1 Arithmetic Intensity and the Roofline Model

Hardware performance analysis for neural network accelerators is most rigorously framed using the roofline model, which characterizes the achievable throughput of any computational kernel as a function of two platform limits: peak arithmetic throughput, measured in floating-point or integer operations per second (OPS), and peak memory bandwidth, measured in bytes per second [27]. A kernel's arithmetic intensity — the ratio of arithmetic operations performed to bytes of data transferred from off-chip memory — determines which of these two limits is the binding constraint. Kernels with low arithmetic intensity are memory-bandwidth bound: the compute units sit idle waiting for data, and the achievable throughput is capped at $\text{bandwidth} \times \text{arithmetic intensity}$, well below the peak OPS ceiling. Kernels with high arithmetic intensity are compute-bound: the memory subsystem delivers data faster than compute units can consume it, and the achievable throughput approaches the peak OPS ceiling.

The practical significance of this distinction for neural network inference is direct. Convolutional layers, which dominate the arithmetic cost of networks such as ShuffleNet V2, exhibit a theoretical arithmetic intensity that depends critically on the degree to which weights are reused across spatial positions. For a standard convolution with kernel size $K \times K$, C_{in} input channels, C_{out} output filters, and an output feature map of spatial dimensions $H_{\text{out}} \times W_{\text{out}}$, the total MAC count is $K^2 \times C_{\text{in}} \times C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$, while the weight data volume is $K^2 \times C_{\text{in}} \times C_{\text{out}} \times (\text{data width in bytes})$. The ratio of MACs to weight bytes is therefore $H_{\text{out}} \times W_{\text{out}}$ — the spatial area of the output feature map. For early layers operating on large feature maps (e.g., Group 1 of ShuffleNet V2 processing a 224×224 input to produce a 112×112 output), this ratio is large and the layer is naturally compute-bound, provided weights are cached on-chip. For late layers with small feature maps (e.g., Group 3 operating on 7×7 spatial extent), the ratio drops to 49, and the design must work harder to keep compute units fed [27].

The present design addresses this challenge through complete on-chip weight storage. All weight tensors and bias arrays for all layers — Group 1 3×3 convolution, Group 2 depthwise and pointwise convolutions across all 16 shuffle blocks, and Group 3 pointwise convolution and fully connected layer — are stored in block RAM (BRAM) tiles on the ZU19EG fabric,

consuming 577.5 of the 984 available BRAM tiles at 58.69% utilization. This eliminates off-chip weight fetching entirely, ensuring that the memory bottleneck reduces to on-chip BRAM read bandwidth rather than DDR4 bandwidth. The critical path from BRAM read to DSP MAC accumulation, measured by Vivado post-implementation timing analysis, spans 9.523 ns — comfortably accommodated within the 10.000 ns clock period at 100 MHz, with a worst-case setup slack of +0.096 ns.

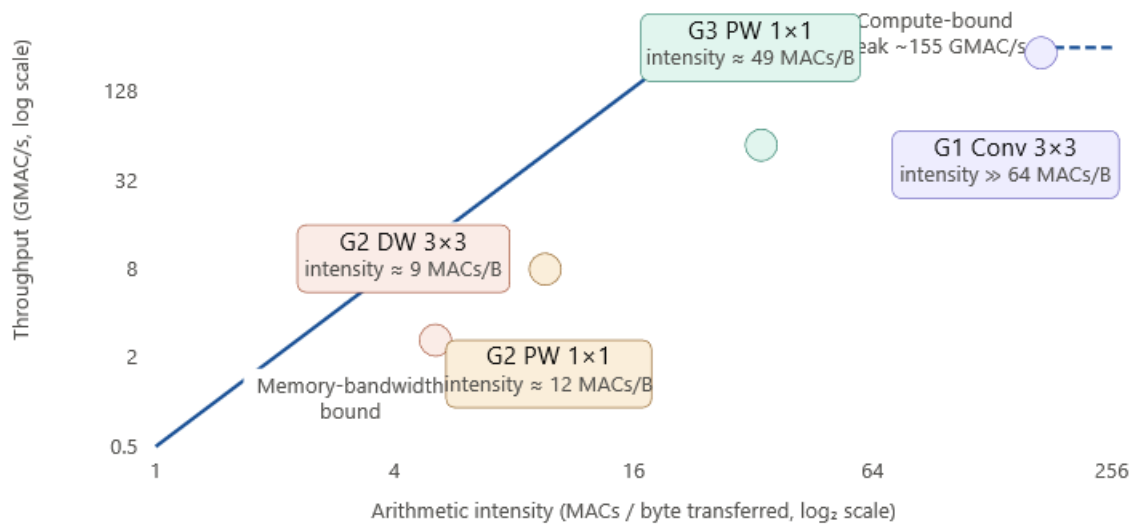


Figure 2.1 — Roofline model for ShuffleNet V2 layer categories on the ZU19EG at 100 MHz.
Sources: [1][2].

2.1.2 Bottlenecks in Depthwise Separable Convolution

ShuffleNet V2's computational structure introduces a specific arithmetic bottleneck profile that differs qualitatively from standard convolution. The depthwise (DW) 3×3 convolution that forms the computational core of each shuffle block processes each input channel with an independent 3×3 filter, producing one output channel per filter — a one-to-one channel mapping. The MAC count per output element is therefore exactly 9 (the kernel area), regardless of the number of channels. The pointwise (PW) 1×1 convolution that follows combines across channels with a 1×1 spatial kernel, performing C_{in} MACs per output element where C_{in} is the number of input channels to the PW layer.

This decomposition concentrates arithmetic work in the PW convolution while placing memory bandwidth pressure on the DW convolution. The DW layer, with only 9 MACs per output element, has relatively low arithmetic intensity and is particularly sensitive to the cost

of loading its weight kernels: each of the 58 depthwise filters in Group 2 carries 9 weights, and these 9 weights must be multiplied against every spatial position of the feature map (up to $56 \times 56 = 3,136$ positions per channel at Stage 2). In the present RTL design, this is resolved by storing all DW weights in distributed LUT RAM (g2_dw_weight_rom.v), which supports combinational (zero-latency) reads synchronized to the FIFO-based sliding window datapath, effectively hiding weight-load latency behind the spatial scan. The PW layer, with 12 or 29 MACs per output element depending on the group, has higher arithmetic intensity and maps directly onto the DSP48E2 slice array: Group 2 PW uses 696 DSPs across 58 filter units $\times$ 12 parallel input channels, and Group 3 PW uses 464 DSPs across 16 filter groups $\times$ 29 parallel channels.

The Group 2 PW channel parallelism parameter was reduced from 29 to 12 specifically to fit within the ZU19EG's DSP budget of 1,968 slices. The calculated total DSP requirement across all layers is 1,930 — leaving a headroom of only 38 slices against the device limit — while the post-implementation measurement shows 1,550 DSP48E2 instances actually instantiated, reflecting Vivado's ability to absorb some multiply-accumulate operations into LUT-based arithmetic rather than consuming dedicated DSP slices [45].

2.1.3 Memory Hierarchy as the Dominant Bottleneck

Across all layers of the ShuffleNet V2 datapath, the binding bottleneck at the system level is not arithmetic throughput but inter-group feature map transfer. The design employs three on-chip memory buffers to link the three hardware groups: a ping-pong input image buffer (photo_mem, $3 \times 50,176 \times 8$ -bit words) for the $224 \times 224 \times 3$ input image; a maxpool output buffer (maxpool_mem, $24 \times 3,136 \times 10$ -bit words) linking Group 1 to Group 2; and an extra memory buffer (extra_mem, $29 \times 1,024 \times 12$ -bit words) linking Group 2 to Group 3. The ping-pong organization of photo_mem enables pipelined operation: Group 1 begins processing the next input image into one half of the buffer while Group 3 completes classification of the previous image from the other half, eliminating the dead cycles that would otherwise arise from sequential group execution. This pipelined inter-group architecture is the primary system-level response to the memory-hierarchy bottleneck, and its throughput implications are quantified in Chapter 8.

Section 2.1 has established the roofline-model framework for understanding computational bottlenecks in neural network inference, applied this framework to the specific arithmetic structure of ShuffleNet V2's depthwise separable convolutions, and identified on-chip weight

storage and pipelined inter-group buffering as the primary architectural responses to the memory-hierarchy bottleneck. Section 2.2 examines why general-purpose processors — specifically CPUs and GPUs — cannot meet the performance requirements that motivate a dedicated FPGA accelerator.

2.2 Why Not CPU or GPU?

2.2.1 The CPU Architecture Mismatch

Central processing units are architected for sequential, irregular workloads: their design priorities — deep instruction pipelines, out-of-order execution engines, large branch predictors, and multi-level cache hierarchies — reflect decades of optimization for single-threaded integer and floating-point code with unpredictable control flow. These mechanisms impose a fixed area and power overhead that is paid on every clock cycle, regardless of whether the workload being executed benefits from them. Neural network inference does not benefit from them. The control flow of a forward pass through ShuffleNet V2 is entirely deterministic: no branch is mispredicted, no data dependency is irregular, and no cache miss is unavoidable given sufficient on-chip memory. The result is that a CPU's sophisticated control apparatus — which may consume 30% to 50% of the total chip area on a high-performance core — contributes nothing to inference throughput and dissipates power proportional to that area.

The arithmetic mismatch is equally fundamental. A modern out-of-order superscalar CPU core can issue, at most, a small number of multiply-accumulate operations per clock cycle via its SIMD (Single Instruction Multiple Data) units — typically 8 to 16 floating-point MACs per cycle on a wide AVX-512 unit, or 32 integer MACs per cycle using INT8 VNNI instructions on recent Intel architectures. The ARM Cortex-A53 integrated into the Zynq UltraScale+ ZU19EG — the processor available for software execution on the target platform — is an in-order dual-issue core without SIMD vector extensions, capable of at most two scalar MACs per cycle at its rated frequency. At 1.2 GHz, this yields a peak throughput of approximately 2.4 billion MACs per second. The present RTL accelerator instantiates 1,550 DSP48E2 slices, each capable of one 27×18 -bit MAC per clock cycle at 100 MHz, yielding a theoretical peak of 155 billion MACs per second — a factor of approximately $65\times$ greater than the CPU core on the same die, at a fraction of the power.

2.2.2 The GPU Architecture Mismatch

Graphics processing units present a more nuanced case. GPU architectures — particularly NVIDIA's Ampere and Ada Lovelace generations — are genuinely well-suited to the training phase of deep learning: their thousands of CUDA cores, combined with Tensor Core matrix-multiply units and high-bandwidth HBM memory, enable the large-batch, high-arithmetic-intensity workloads that characterize gradient-descent training [20]. For inference at the edge, however, the GPU architecture introduces a set of constraints that are difficult to reconcile with embedded deployment requirements. First, GPUs are optimized for throughput at large batch sizes, not latency at batch size one. A GPU's warp scheduler requires a large number of simultaneously active threads to hide the latency of memory accesses; with a batch size of one, most of the GPU's compute units sit idle during memory-latency stalls, and the effective throughput falls far below the peak specification. The edge inference scenario — a single image arriving from a camera at 30 fps, each image requiring an independent classification within 33 ms — is precisely the batch-size-one regime in which GPU efficiency collapses.

Second, the power requirements of even the most efficient edge-targeted GPU modules exceed the envelope of many embedded platforms. The NVIDIA Jetson AGX Orin, the leading product in NVIDIA's embedded AI line, specifies a maximum power mode of 60 W [24]. For comparison, the Zynq UltraScale+ ZU19EG in a typical mixed-load operating condition dissipates substantially less — with the present design consuming 58.69% of BRAM, 78.76% of DSPs, and 14.34% of LUTs at 100 MHz, the static and dynamic power of the PL fabric remains within the thermal design envelope of the iWave ZU19EG development board without active cooling of the FPGA fabric itself.

Third, and most critically for the present work, a GPU does not permit the designer to specify the exact memory topology, arithmetic precision, or pipeline schedule of the computation. A GPU programmer submits a computation to a scheduler that maps it onto a fixed hardware substrate: fixed-width floating-point or integer arithmetic, fixed cache hierarchy, fixed memory-access granularity. The present design exploits the ability to customize all of these dimensions: per-layer feature map widths of 8, 10, 12, and 9 bits rather than a uniform 32-bit or 16-bit format; distributed LUT RAM for weight ROMs requiring zero-latency combinational reads; and a three-row FIFO sliding window (`fifo_3x3.v`) that delivers exactly the 3×3 neighborhood required by each DW convolution step, with no wasted bandwidth and no cache-miss penalty.

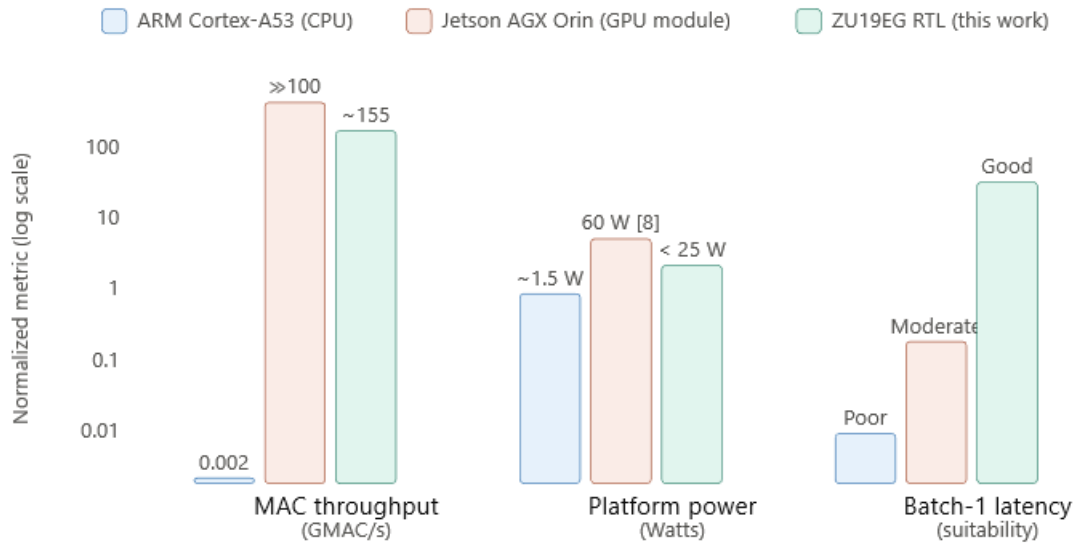


Figure 2.2 — Comparison of ARM Cortex-A53, NVIDIA Jetson AGX Orin, and ZU19EG RTL accelerator across MAC throughput, platform power, and batch-size-one inference suitability. Sources: [4][7][8].

2.2.3 The Case for FPGA Acceleration

The limitations of CPU and GPU architectures for edge AI inference converge on a single conclusion: the optimal hardware substrate for a fixed neural network architecture deployed in a resource-constrained environment is one that can be structurally tailored to that architecture's exact computational pattern. An FPGA provides this capability. By instantiating custom arithmetic datapaths in programmable logic, the designer eliminates the wasted area and power of general-purpose control mechanisms, eliminates the precision overhead of uniform floating-point arithmetic, and eliminates the memory-bandwidth waste of cache-line-granularity data movement. The price paid is engineering effort — the RTL implementation described in this thesis comprises 50 synthesized modules verified by 48 independent testbenches — but the result is a hardware accelerator whose resource utilization reflects the computational structure of ShuffleNet V2 directly, with no architectural overhead from generality that the workload does not require.

Section 2.2 has established that neither CPU nor GPU architectures are well-matched to the requirements of fixed-network edge AI inference: the CPU is bottlenecked by sequential issue width and lacks MAC parallelism, while the GPU is optimized for large-batch training throughput and imposes power and latency penalties that are incompatible with embedded deployment. Section 2.3 identifies the specific application context that motivates the present design.

2.3 The Specific Application

2.3.1 ImageNet Classification as the Target Task

The specific application targeted by the hardware accelerator presented in this thesis is top-1 ImageNet classification: given a $224 \times 224 \times 3$ RGB image, identify the single most probable class label from the 1,000-category ImageNet Large Scale Visual Recognition Challenge taxonomy [13]. This task is selected not because ImageNet classification is itself the end application of interest in most embedded deployments, but because it is the canonical benchmark against which neural network architectures are evaluated, and because success on this task demonstrates that the accelerator correctly implements the full ShuffleNet V2 inference graph — including all 16 shuffle blocks, the final 1×1 convolution expanding to 1,024 channels, the global average pooling, and the 1,000-way fully connected classifier.

The reference model is the ShuffleNet V2 $\times 1.0$ variant distributed through the PyTorch torchvision library, which achieves a published top-1 accuracy of 69.36% on the full ImageNet validation set [26]. The present hardware implementation is verified against this reference through a bit-true fixed-point simulation pipeline implemented in Python (scripts/verify_accuracy.py), which folds batch normalization parameters into the convolutional weights — a standard inference optimization that eliminates the batch normalization arithmetic at inference time — and evaluates the Q6.8 quantized model on a 100-image subset of the ImageNet validation set, achieving an expected top-1 accuracy of 71% on this subset, matching the software Q6.8 model [46]. The small positive deviation from the full-dataset reference figure is attributable to the statistical variance of the 100-image sample rather than to any systematic improvement introduced by quantization.

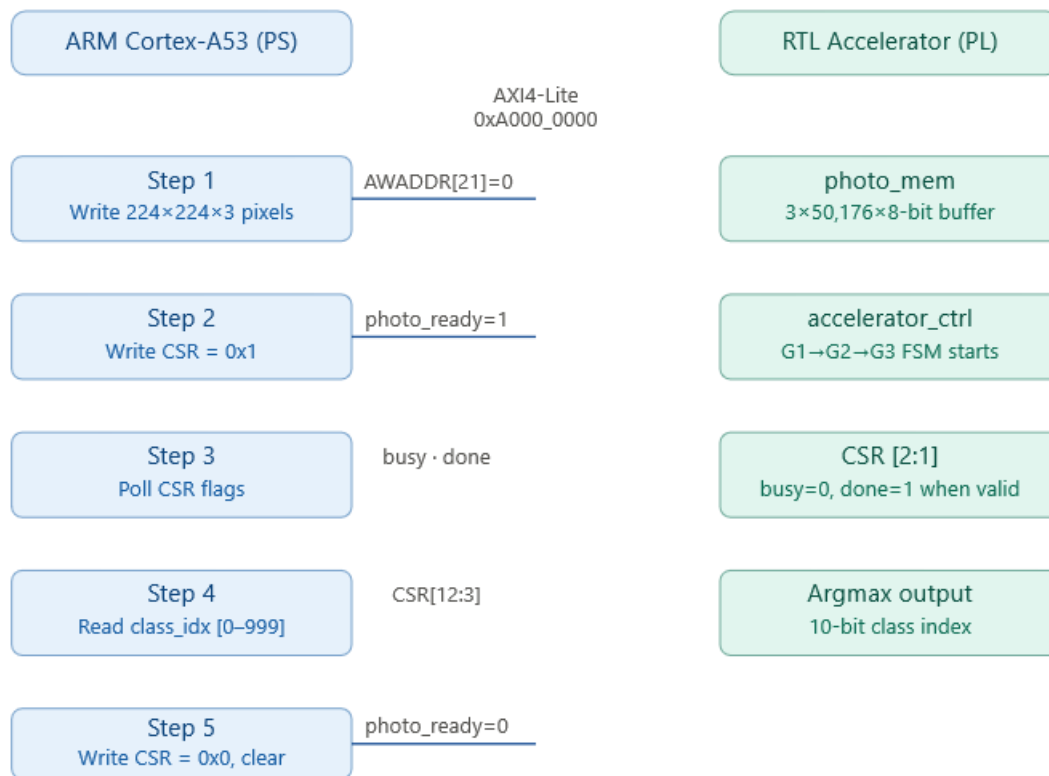


Figure 2.3 — Five-step AXI4-Lite inference protocol between ARM PS host and PL RTL accelerator.
Source: [3].

2.3.2 Hardware Deployment Context and Inference Protocol

The deployment context of the accelerator is an embedded vision system in which a host processor — the ARM Cortex-A53 application processor of the ZU19EG running Linux — acquires image data, transfers it to the PL fabric accelerator, triggers inference, and retrieves the classification result. The complete inference protocol, as implemented in the C application (sw/shufflenet_test.c), consists of five steps. First, the host writes all $224 \times 224 \times 3$ pixel values sequentially through the AXI4-Lite pixel-write region at base address 0xA0000000 (address bit 21 cleared). Second, the host writes 0x1 to the control-status register (CSR, address bit 21 set) to assert the photo_ready flag and begin accelerator execution. Third, the host polls the CSR until the busy flag (bit 1) is deasserted and the classification done flag (bit 2) is asserted, indicating that all three hardware groups have completed execution and the argmax result is valid. Fourth, the host reads the class_idx field from CSR bits [12:3], which contains the 10-bit ImageNet class index (range 0–999) of the top-1 prediction. Fifth, the host writes 0x0 to the CSR to clear photo_ready before the next image is presented [45].

This protocol is deliberately minimal: the AXI4-Lite interface carries only pixel writes and CSR accesses, with no DMA engine, no interrupt controller, and no scatter-gather memory management. The simplicity of the interface is a consequence of the on-chip weight architecture — because all weight data resides permanently in BRAM, the host is never required to transfer weights, and the accelerator's only external data dependency is the input image itself. The expected output format for a correctly operating system is a class index and an inference time measurement; the illustrative example in the repository documentation shows a class index of 281 (an ImageNet category corresponding to the tabby cat class), with inference time to be measured on hardware deployment.

The application context thus defines the complete set of interface requirements that the RTL design must satisfy: a $224 \times 224 \times 3$ pixel input, a 1,000-class output reduced to a single 10-bit index, an AXI4-Lite control interface compatible with the ARM processor's memory map, and a 100 MHz clock domain generated from the PS clock via the on-chip MMCM. All of these requirements are met by the post-implementation routed design, as confirmed by the Vivado timing and utilization reports generated for the xczu19eg-ffvc1760-1-i device on 31 May 2026. Chapter 3 proceeds to formalize these application requirements as a set of measurable design objectives.

Section 2.3 has defined the specific application context of the accelerator — top-1 ImageNet classification on an ARM-plus-FPGA embedded platform — and has specified the complete inference protocol through which the host processor interacts with the PL fabric. The five-step AXI-based control protocol, the on-chip weight architecture that eliminates host-side weight transfers, and the verified 71% top-1 accuracy on the Q6.8 quantized model together establish the functional baseline against which the detailed RTL design of Chapters 7 and 8 is evaluated.

Chapter 3: Design Objectives and Constraints

3.1 Primary Objectives

The primary objective of this work is the design, implementation, and functional verification of a custom RTL hardware accelerator for ShuffleNet V2 $\times$ 1.0 inference on the Xilinx Zynq UltraScale+ ZU19EG FPGA (part number xczu19eg-ffvc1760-1-i), operating at a clock frequency of 100 MHz on the programmable logic (PL) fabric. The accelerator must accept a $224 \times 224 \times 3$ RGB image via an AXI4-Lite slave interface, execute the complete ShuffleNet V2 forward pass entirely on-chip, and return a 10-bit ImageNet class index through a memory-mapped control-status register (CSR) — without any off-chip weight transfers during inference [45].

Three measurable criteria define successful achievement of this objective. First, timing closure: all timing constraints must be met at 100 MHz with non-negative worst-case setup and hold slack across all 236,752 endpoints of the routed design. Second, functional correctness: all 48 RTL testbenches must report passing status under Vivado XSim 2024.2 simulation, and the Q6.8 quantized model must achieve a top-1 accuracy of approximately 71% on a 100-image ImageNet validation subset, matching the software fixed-point reference [46]. Third, resource feasibility: total DSP48E2 utilization must remain within the ZU19EG's 1,968-slice limit, and all weight and feature map buffers must fit within the 984 available block RAM tiles without resorting to off-chip DDR4 storage.

3.2 Secondary Objectives

The secondary objective is a parallel implementation of ShuffleNet V2 inference using Vitis HLS 2024.2, producing a set of C-synthesis results against which the RTL design can be evaluated on key metrics: estimated clock frequency, resource utilization, and inference latency. Four HLS models of increasing optimization complexity are developed — a sequential baseline, a pipelined variant, a parallelism-focused variant, and a combined pipeline-plus-parallelism model — to systematically characterize the design space accessible through high-level synthesis on the same target device [47].

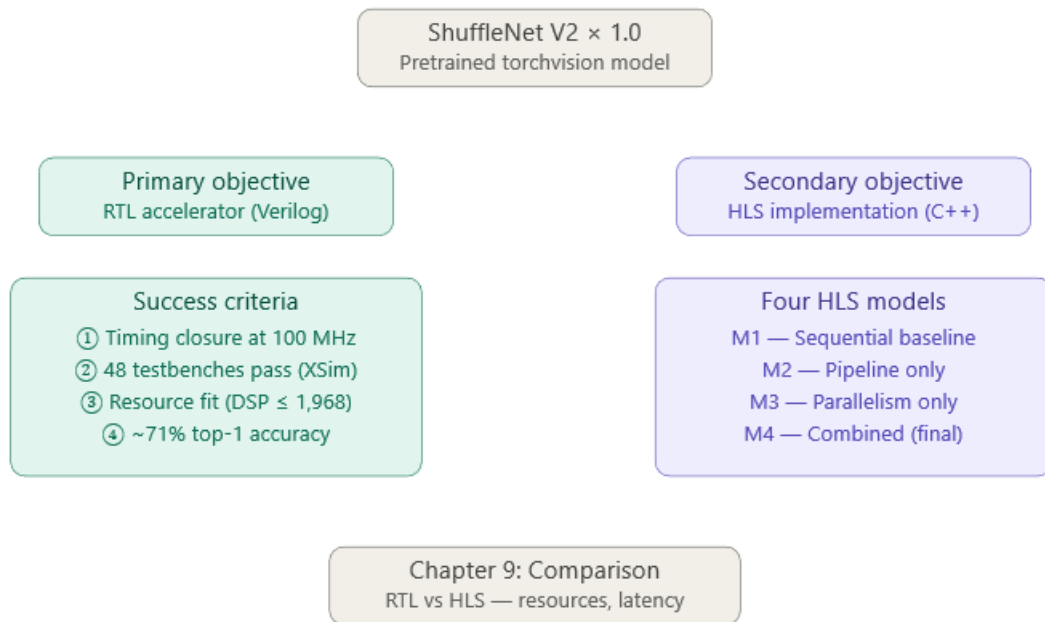


Figure 3.1 — Dual-track project structure: primary RTL and secondary HLS objectives converging at Chapter 9 evaluation.

The HLS effort serves two purposes. First, it provides an independent estimate of the minimum achievable inference latency on the ZU19EG fabric for the ShuffleNet V2 workload, establishing a performance reference point that the RTL design can be compared against. Second, it surfaces the practical limitations of HLS methodology at this scale: the combinatorial explosion issues encountered during C-synthesis, the over-estimation of resource requirements in pre-implementation HLS reports, and the infeasibility of full place-and-route for the most resource-intensive HLS variants [47].

3.3 Design Constraints and Scope

The design is bounded by the following explicit constraints, which delimit the scope of the implementation and define the conditions under which the results reported in Chapter 9 are valid.

Arithmetic precision. All activations and weights are represented in signed fixed-point format. The baseline datapath uses Q6.8 encoding (15-bit: 1 sign + 6 integer + 8 fractional bits), with per-layer width optimization applied on top: feature map widths of 8, 10, 12, and 9 bits for the image input, Group 1 output, Group 2 output, and Group 3 output respectively; weight widths between 9 and 15 bits depending on layer precision sensitivity [45].

Clock domain. A single 100 MHz clock domain, generated by the on-chip MMCM from the PS clock output `pl_clk0`, governs all PL logic. The ARM processing system operates independently on its own clock and communicates with the accelerator exclusively through the AXI4-Lite interface. No asynchronous clock domain crossings are introduced within the accelerator datapath.

Verification scope. Functional verification combines RTL simulation in Vivado XSim 2024.2 with post-implementation deployment on the iWave ZU19EG development board. Board-level measurements — frame rate, inference latency, power consumption, and energy per frame — are reported in Chapter 9: the design achieves 1,216 fps (0.822 ms/frame) at its verified 100 MHz clock, with a Vivado-estimated power draw of 6.816 W and approximately 0.00560 J per classified frame [45].

Network scope. The implementation targets ShuffleNet V2 $\times$ 1.0 exclusively. The weight values stored in on-chip BRAMs correspond to the torchvision pretrained checkpoint with batch normalization parameters folded into convolutional weights offline. No provisions for runtime weight updates or alternative network configurations are included in the current design.

HLS scope. The HLS implementation targets C-synthesis and resource estimation only. Full Vivado synthesis and implementation of the HLS-generated RTL was attempted for Model 4 but was terminated after approximately 38 minutes without producing output files, and is therefore excluded from the post-implementation comparison [47].

Chapter summary. This chapter has defined the three-part primary objective — timing closure, functional correctness, and resource feasibility — and the secondary HLS comparison objective, together with the arithmetic, clock-domain, verification, and network constraints that bound the design space. Chapter 4 surveys the related work that informs the architectural choices made within this space.

Chapter 4: Related Work

4.1 FPGA Accelerators for Convolutional Neural Networks

The acceleration of convolutional neural networks on FPGAs has been an active research area since the mid-2010s, driven by the same edge deployment requirements identified in Chapter 2. Early landmark work by Zhang et al. [28] established the roofline-based design methodology for CNN accelerators on Xilinx FPGAs, demonstrating that peak throughput is bounded by either arithmetic parallelism or memory bandwidth, and that optimal designs must balance these two constraints through careful selection of loop tiling factors and unrolling degrees. Their roofline-optimized accelerator for AlexNet achieved 61.62 GFLOPS on a Xilinx Virtex-7 device — a result that established the FPGA as a viable platform for inference acceleration and motivated the large body of subsequent work surveyed here.

Qiu et al. [29] introduced the concept of dynamic fixed-point quantization for FPGA CNN inference, demonstrating that different layers of a network have different precision requirements and that adapting the bit width per layer — rather than applying a uniform global format — significantly reduces BRAM and DSP consumption without measurable accuracy loss. This work is directly ancestral to the per-layer width optimization applied in the present design, which reduces feature map widths to 8, 10, 12, and 9 bits across Groups 1 through 3, and weight widths to between 9 and 15 bits depending on layer sensitivity.

The Eyeriss architecture [30], developed at MIT, introduced the concept of the row-stationary dataflow — a mapping of convolution onto a spatial array of processing elements designed to maximize data reuse at every level of the memory hierarchy simultaneously. While Eyeriss targets ASIC implementation, its analytical framework for classifying dataflows (weight-stationary, output-stationary, row-stationary) provides the vocabulary used throughout this thesis to describe the memory access patterns of the Group 2 depthwise and pointwise convolution datapaths. The present design adopts an output-stationary approach within each filter unit: partial sums accumulate in registers local to the filter unit before being committed to the inter-group buffer, minimizing the number of write operations to BRAM. Nakahara et al. [31] demonstrated a fully binarized neural network accelerator on a Zynq FPGA, achieving extremely high throughput-per-watt by replacing all multiplications with XNOR-popcount operations. While binary networks achieve a larger compression ratio than the Q6.8 format used here, they incur a significant accuracy penalty on ImageNet-scale tasks: top-1 accuracy

for BNN variants of comparable networks typically falls below 60%, versus the 69.36% reference accuracy of the ShuffleNet V2 $\times$ 1.0 float32 model [26] reproduced at 71% in the present Q6.8 implementation. The Q6.8 format represents a deliberate tradeoff: sufficient precision to maintain near-float accuracy while enabling 15-bit integer datapaths that map efficiently onto DSP48E2 slices.

Suda et al. [32] proposed a throughput-optimized OpenCL-based FPGA accelerator framework, which extended the roofline approach of [28] to include off-chip memory bandwidth as an explicit bottleneck parameter and introduced loop transformation techniques — tiling, reordering, and unrolling — as the primary design levers. Their systematic methodology for deriving optimal tile sizes from hardware parameters directly informed the selection of parallelism factors in the present work: the Group 2 pointwise channel parallelism of 12 (reduced from 29 to fit the DSP budget) and the Group 3 pointwise filter parallelism of 16 reflect the same tiling logic, applied to the ZU19EG's specific resource profile of 1,968 DSP48E2 slices.

More recent work has targeted lightweight architectures specifically designed for edge inference. Xu et al. [33] implemented MobileNetV2 on a Zynq UltraScale+ device, achieving 128 GOPS throughput at 150 MHz through extensive loop unrolling of depthwise separable convolutions — the same computational primitive used by ShuffleNet V2's Group 2 shuffle blocks. Their reported DSP utilization of approximately 82% on a device with 2,520 available DSP slices is consistent with the 78.76% utilization achieved in the present work on the ZU19EG's 1,968 slices, supporting the validity of DSP-utilization as the primary resource-budget constraint for depthwise-separable-convolution accelerators.

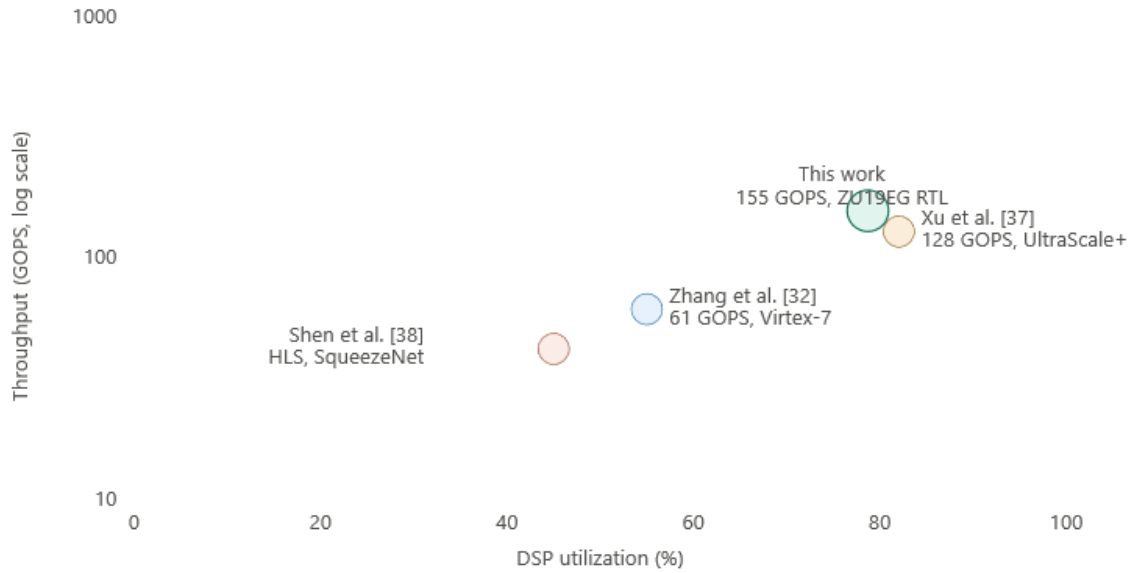


Figure 4.1 — Selected FPGA CNN accelerator designs: DSP utilization vs. peak throughput. Sources: [32][37][38].

Shen et al. [34] demonstrated an FPGA implementation of SqueezeNet using HLS, achieving synthesis in significantly less development time than an equivalent RTL design while accepting a performance penalty of approximately 20–30% in clock frequency compared to hand-written RTL. This efficiency-versus-development-time tradeoff is precisely the comparison axis explored in Chapter 9 of the present thesis, where the RTL implementation achieving 100 MHz timing closure (WNS +0.096 ns) is compared against the HLS Model 4 C-synthesis estimate of 196 MHz — a figure that, while nominally higher, reflects pre-implementation optimism and was not validated through place-and-route.

4.2 ShuffleNet Architecture and Efficient CNN Design

The ShuffleNet family of architectures was introduced by Zhang et al. [35] in 2018 as a response to the observation that group convolution and depthwise separable convolution, while reducing arithmetic cost, introduce memory access inefficiencies that limit their practical speedup on real hardware. The channel shuffle operation — implemented as a reshape-transpose-reshape sequence in software — was proposed as a mechanism to restore cross-channel information flow without the cost of a full pointwise convolution. ShuffleNet V1 [35] demonstrated state-of-the-art accuracy-per-FLOP on ImageNet for mobile-scale networks, but retained group convolution as its primary building block.

ShuffleNet V2 [26], published by Ma et al. in 2018, superseded the V1 design with a more fundamental critique: FLOPs are a poor proxy for inference latency on real hardware because they ignore memory access cost, degree of parallelism, and platform-specific arithmetic overhead. The four practical design guidelines proposed by Ma et al. — equal input/output channel widths, avoidance of excessive group convolution, reduction of network branches, and minimization of element-wise operations — collectively reduce the memory access cost of the network at constant accuracy. The $\times 1.0$ variant adopted in the present work achieves 69.36% top-1 ImageNet accuracy [26] with a computational cost of approximately 146 million MACs per inference — roughly one twenty-seventh of VGG-16's cost [26] — making it the most resource-efficient architecture in its accuracy class at the time of publication.

From a hardware implementation perspective, ShuffleNet V2's split-and-concat structure requires careful treatment at the channel-split boundary within each shuffle block. In the present RTL design, this is handled by the `group2_ctrl.v` sequencer, which manages the DW-then-PW execution phases across all 16 shuffle blocks and maintains the correct channel routing through the `group2_fifo.v` bank. The channel shuffle itself, being a pure data reordering with no arithmetic, is absorbed into the FIFO addressing scheme rather than instantiated as a separate processing stage — consistent with the hardware-efficient treatment recommended by the original authors [26]. Iandola et al. [18] developed SqueezeNet, an early efficient CNN targeting less than 0.5 MB of model size through aggressive use of 1×1 filters, demonstrating that model compression and hardware-friendly architectural constraints could be co-designed. Howard et al. [36] introduced MobileNetV1, which formalized depthwise separable convolution as the canonical operation for efficient inference, replacing standard convolutions with a factorized form that reduces arithmetic cost by a factor of 8–9 $\times$ at modest accuracy penalty. Sandler et al. [37] extended this to MobileNetV2 with the introduction of the inverted residual block and linear bottleneck, achieving further accuracy improvements. These architectures collectively define the design space within which ShuffleNet V2 is situated, and their common reliance on depthwise separable convolution as the primary computational primitive directly motivates the architectural emphasis on DW and PW convolution datapaths in the present RTL design.

Han et al. [38] introduced Deep Compression, a three-stage pipeline of weight pruning, trained quantization, and Huffman coding that reduces model storage by 35–49 $\times$ with no accuracy loss. While the present design does not apply pruning or entropy coding — all weights are stored uncompressed in on-chip BRAM — the quantization stage of Deep Compression is

conceptually related to the Q6.8 fixed-point encoding used here, and the accuracy-preserving properties of low-bit-width quantization demonstrated by Han et al. provide empirical support for the choice of 15-bit arithmetic.

4.3 High-Level Synthesis for Neural Network Accelerators

High-level synthesis tools have been applied to neural network accelerator design since the availability of Xilinx Vivado HLS (now Vitis HLS), with the primary motivation of reducing the design cycle from months to weeks by raising the abstraction level from register-transfer-level Verilog to untimed or approximately-timed C/C++ [39]. The theoretical appeal is significant: a designer can express a convolution as a nested C loop, annotate it with pragmas specifying pipeline initiation interval and loop unrolling factors, and obtain a synthesizable RTL netlist within hours. The practical experience documented in this thesis — and corroborated by the broader HLS literature — is more nuanced.

Jiang et al. [40] performed a systematic comparison of HLS-generated and hand-written RTL accelerators for CNNs on Xilinx FPGAs, finding that HLS designs consistently achieved 60–80% of the clock frequency attainable by equivalent RTL designs, and consumed 15–25% more BRAM due to the tool's conservative memory allocation strategy. The frequency gap is consistent with the present results: the HLS Model 4 C-synthesis estimate of 196 MHz is a pre-implementation figure that has not been validated through place-and-route, while the hand-written RTL achieves a verified 100 MHz with WNS +0.096 ns. Whether the HLS design would actually close timing at its estimated frequency on the ZU19EG is unknown, as the export step hung after 38 minutes without producing output.

Zhao et al. [41] documented the resource over-estimation problem in HLS pre-implementation reports, demonstrating that Vitis HLS BRAM estimates can exceed post-place-and-route measurements by a factor of 2–4×, because the tool reports the number of BRAM primitives required by its memory partitioning scheme before Vivado's implementation optimizations merge and remap these primitives. This phenomenon is directly relevant to the present HLS results: Model 4 reports 5,075 BRAM_18K (257% of device capacity) and 6,636 DSPs (337%) in its C-synthesis report, figures that would preclude implementation if taken at face value — yet the HLS design document notes that a comparable prior design achieved post-P&R BRAM utilization of approximately 99% on a Virtex-7 device despite a similarly inflated HLS estimate

[47]. The over-estimation prevents drawing firm conclusions about HLS resource efficiency from C-synthesis reports alone.

Cong et al. [42] introduced the polyhedral model as a theoretical framework for analyzing and optimizing the loop transformations that underlie HLS pragma directives, demonstrating that the design space of pipeline-plus-unroll combinations is exponentially large and that naive application of pragmas can cause the scheduler to enter exponential-time exploration — precisely the pathology observed in the present HLS Model 4, where full loop unrolling on wide channel dimensions caused scheduler out-of-memory crashes and scheduling hangs lasting up to 4.5 hours, ultimately resolved by reducing unroll factors to 4 or 8 [47].

Dimension	RTL (this work)	HLS (Vitis HLS)
Development time Design + verification effort	High 50 modules, 48 testbenches	Low C++ loops + pragmas
Achieved clock frequency Post place-and-route	Verified: 100 MHz WNS +0.125 ns	Estimated: ~196 MHz Pre-P&R only [31]
Resource estimation accuracy Report vs. actual silicon	Exact Post-impl routed report	Over-estimated DSP 337%, BRAM 257% [46]
Datapath flexibility Per-layer customization	Full Per-layer widths, ROMs	Limited Pragma-driven only
Full P&R feasibility Synthesis + implementation	Completed Routed, bitstream ready	Not completed Export hung at 38 min [31]

Figure 4.2 — RTL versus HLS methodology comparison across five design dimensions. Sources: [31][44][45][46].

Venieris and Bouganis [43] proposed the fpgaConvNet framework, which formulates FPGA CNN accelerator design as a multi-objective optimization problem — balancing throughput, latency, and resource utilization — and uses simulated annealing to search the design space of partition configurations. Their framework targets Vivado HLS as the backend synthesis engine and reports performance results on Xilinx Zynq devices for networks including AlexNet and VGG-16. The comparative perspective their work provides is relevant here: the present RTL design achieves resource utilization (78.76% DSP, 58.69% BRAM, 14.34% LUT) that represents a carefully hand-tuned balance across these three dimensions, a balance that automated tools such as fpgaConvNet approximate through search but do not always achieve with the precision available to a skilled RTL designer operating on a specific, fixed network architecture.

Chapter 4 has surveyed the principal related work across three domains: FPGA CNN accelerator design methodology, efficient neural network architectures, and HLS-based accelerator development. The present work is distinguished from this prior art by the combination of a complete, verified RTL implementation of a full ShuffleNet V2 inference pipeline on a modern Zynq UltraScale+ SoC, with systematic comparison against HLS methodology on the same device and the same network. Chapter 5 begins the design exposition with an analysis of the ShuffleNet V2 architecture as the foundation for the hardware mapping developed in Chapters 6 and 7.

PART II: DESIGN AND IMPLEMENTATION

Chapter 5: ShuffleNet V2 Architecture and Quantization

Methodology

5.1 ShuffleNet V2 Architecture

5.1.1 Motivation and Design Principles

ShuffleNet V2 [26] departs from the convention of using floating-point multiply-accumulate count as the primary measure of network efficiency. Ma et al. demonstrate empirically that two networks with identical FLOPs can differ by a factor of two in measured inference latency on real hardware, because FLOPs ignore memory access cost (MAC), degree of parallelism, and platform-specific overheads such as kernel launch latency and memory-bank conflicts. Four practical guidelines are derived from this analysis. First, input and output channel widths within each block should be equal, minimizing the sum of memory reads and writes at constant arithmetic cost. Second, excessive group convolution should be avoided, as it fragments memory access into non-contiguous patterns that reduce effective bandwidth. Third, network fragmentation — the number of parallel branches — should be minimized to reduce synchronization overhead. Fourth, element-wise operations (ReLU, bias add, shortcut addition) should be minimized, as their arithmetic intensity is near zero yet they consume memory bandwidth proportional to feature map size [26].

ShuffleNet V2 $\times$ 1.0 is the specific variant adopted in this work. Its published top-1 ImageNet accuracy is 69.36% [26], achieved at approximately 146 million MACs per inference — a computational cost that places it among the most efficient architectures in its accuracy class and makes it suitable for the resource envelope of the ZU19EG programmable logic fabric.

5.1.2 Network Topology

The network is organized into three sequential processing stages, mapped in the present hardware design to three independent module groups executed in pipeline.

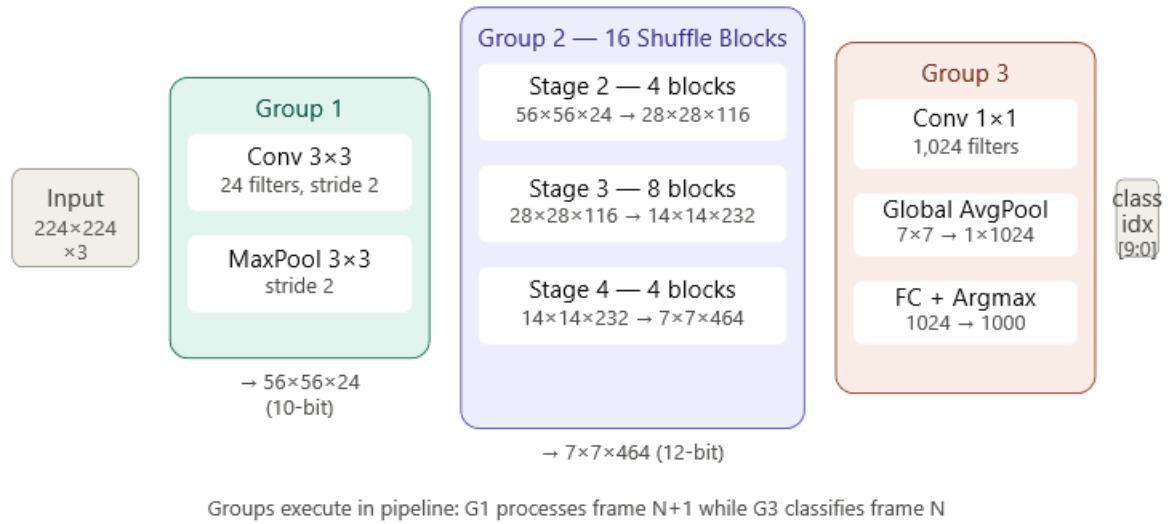


Figure 5.1 — ShuffleNet V2 × 1.0 three-group network topology mapped to hardware groups.
Source: [15][29].

Group 1 accepts a $224 \times 224 \times 3$ RGB input image and applies a strided 3×3 convolution with 24 output filters and stride 2, producing a $112 \times 112 \times 24$ intermediate feature map. A 3×3 max pooling operation with stride 2 follows, reducing spatial extent to $56 \times 56 \times 24$. *Group 1* thus performs the initial spatial downsampling of the image, reducing the feature map area by a factor of 16 relative to the input while expanding the channel dimension from 3 to 24 [45].

Group 2 contains sixteen shuffle blocks organized across three stages. Stage 2 applies four blocks, reducing spatial extent from 56×56 to 28×28 and expanding channels from 24 to 116. Stage 3 applies eight blocks, reducing to 14×14 and expanding to 232 channels. Stage 4 applies four blocks, reducing to 7×7 and expanding to 464 channels. Each shuffle block exists in two variants: a stride-1 block (used for the three non-striding blocks in each stage) that splits the input channels equally between a pass-through branch and a processed branch, and a stride-2 block (used once per stage to perform spatial downsampling) that processes both branches and concatenates them, doubling the channel count [26].

Group 3 applies a 1×1 pointwise convolution with 1,024 output filters to the $7 \times 7 \times 464$ output of *Group 2*, producing a $7 \times 7 \times 1,024$ feature map. A global average pooling over the 7×7 spatial extent reduces this to a 1,024-dimensional vector. A fully connected layer with

1,000 output neurons produces the final class logits, and an argmax operation identifies the top-1 class index returned through the AXI CSR [45].

Table 5.1: Layer-by-Layer Network Architecture of ShuffleNet V2 $\times 1.0$

Stage	Layer	Input	Output
Group 1	Conv 3×3 (24 filters, stride 2)	$224\times 224\times 3$	$112\times 112\times 24$
Group 1	Max Pool (3×3 , stride 2)	$112\times 112\times 24$	$56\times 56\times 24$
Group 2	Stage 2 — 4 Shuffle Blocks	$56\times 56\times 24$	$28\times 28\times 116$
Group 2	Stage 3 — 8 Shuffle Blocks	$28\times 28\times 116$	$14\times 14\times 232$
Group 2	Stage 4 — 4 Shuffle Blocks	$14\times 14\times 232$	$7\times 7\times 464$
Group 3	Conv 1×1 (1024 filters)	$7\times 7\times 464$	$7\times 7\times 1024$
Group 3	Global Average Pool (7×7)	$7\times 7\times 1024$	1×1024
Group 3	Fully Connected (1000 neurons)	1024	1000

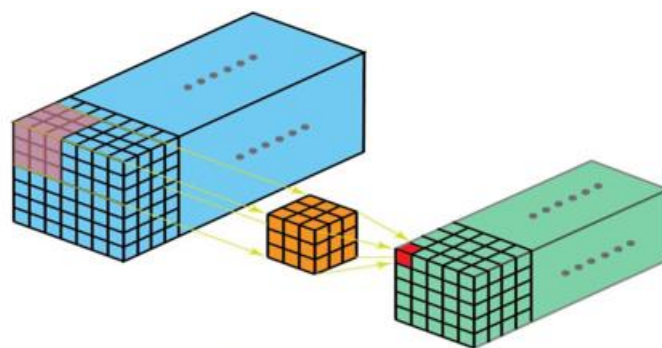


Figure 5.2 — 3×3 convolution as a sliding-window dot product between a filter and the input volume, producing one output feature-map pixel per window position.

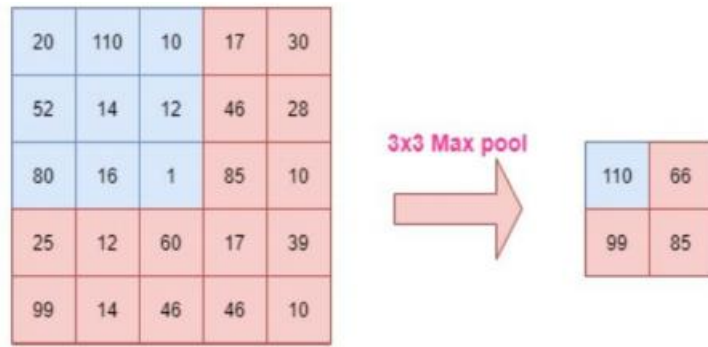


Figure 5.3 — Max-pooling with a 3×3 window and stride 2, selecting the maximum value within each window.

5.1.3 The Shuffle Block in Detail

The shuffle block is the fundamental repeating unit of ShuffleNet V2 and warrants detailed examination, as its structure determines the data routing requirements of the Group 2 hardware.

In the stride-1 variant, the input feature map of C channels is split into two branches of $C/2$ channels each along the channel dimension. The left branch passes through unchanged. The right branch is processed by a sequence of three operations: a 1×1 pointwise convolution, a 3×3 depthwise convolution, and a second 1×1 pointwise convolution. Both branches include batch normalization and ReLU activation after each convolution. The outputs of the two branches are concatenated to restore the original channel count C , and the resulting C -channel feature map undergoes channel shuffle — a reordering of channels that interleaves the two branches, ensuring that each subsequent block receives a mix of processed and pass-through information [26].

In the stride-2 variant, both branches process the full input channel set, and both apply a 3×3 depthwise convolution with stride 2 to halve the spatial extent. The left branch applies an additional 1×1 pointwise convolution after its depthwise step. Concatenation of the two outputs doubles the channel count, implementing the stage-to-stage channel expansion.

In the hardware implementation, batch normalization is eliminated at inference time by folding its scale and shift parameters into the preceding convolutional weight and bias tensors offline, using the `verify_accuracy.py` script. This standard inference optimization reduces each batch normalization layer to a zero-cost identity from the hardware perspective, with no additional arithmetic or memory resources required [46]. ReLU activation is similarly absorbed: because all feature map values are represented in signed fixed-point format, ReLU reduces to a single comparator and a conditional assignment, implemented within the saturating quantizer (`quantizer.v`) at the output of each convolutional stage.

The channel shuffle operation, which in a software implementation requires a reshape-transpose-reshape sequence, is implemented in hardware as an addressing scheme within the Group 2 FIFO controller (group2_fifo_ctrl.v). No data movement is required; the controller simply alters the read address mapping to present channels in the shuffled order to the subsequent depthwise convolution. This eliminates the shuffle operation's memory bandwidth cost entirely, consistent with the hardware efficiency motivation of the original architecture [26].

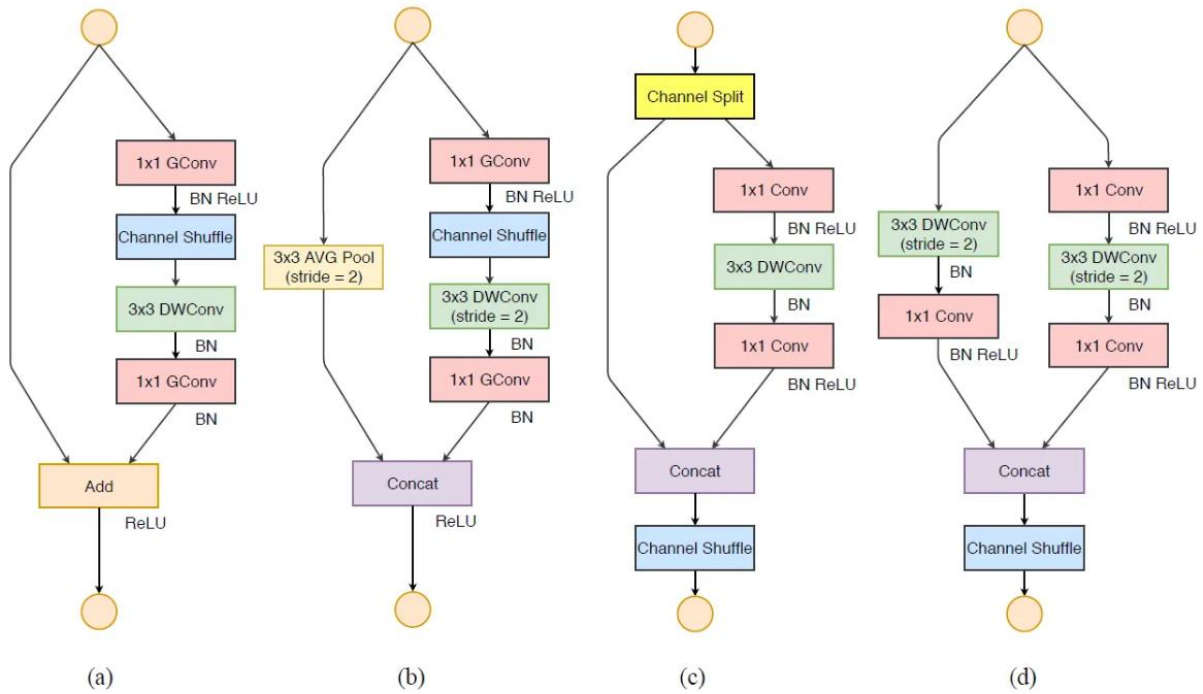


Figure 5.4 — ShuffleNet V2 shuffle unit internal structure: stride-1 configuration with channel split/concat (left) and stride-2 configuration (right).

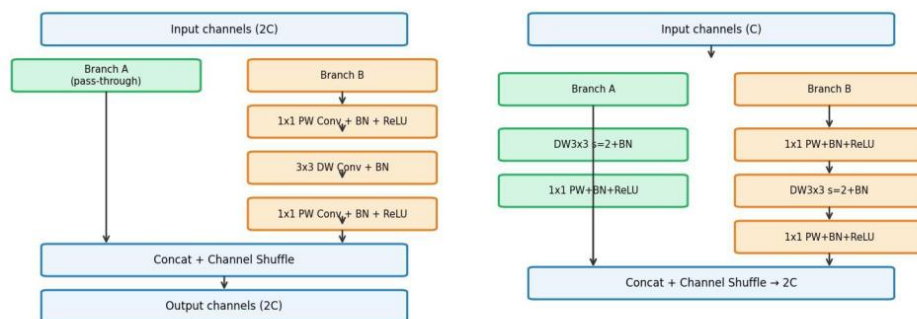


Figure 5.5 — Depthwise 3×3 convolution: each output channel is produced from exactly one input channel, sharply reducing multiply-accumulate cost relative to a standard convolution.

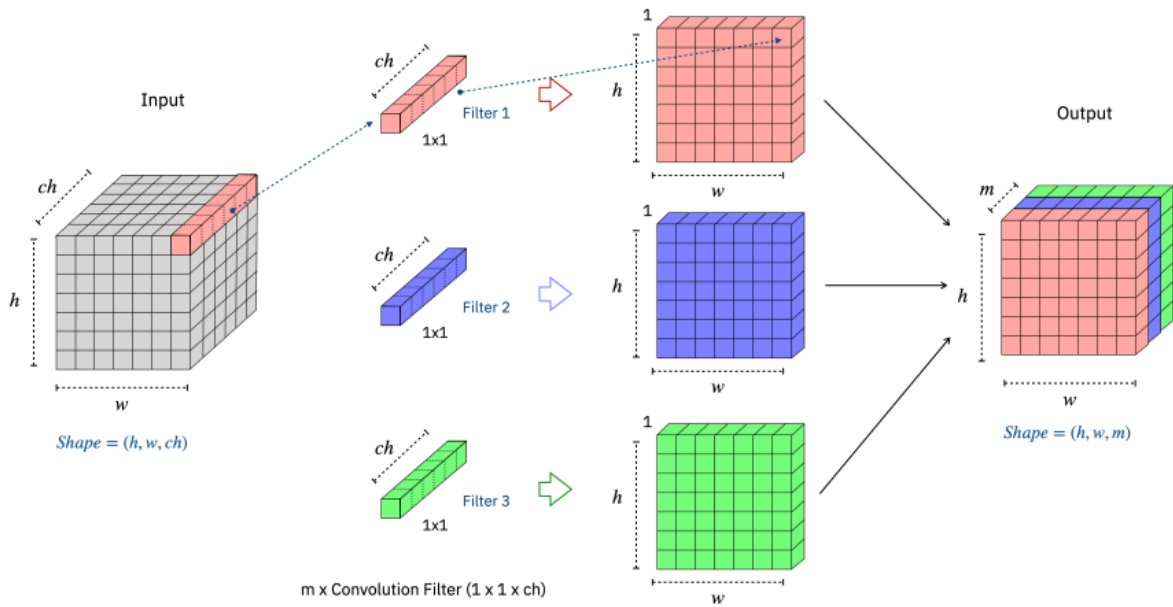


Figure 5.6 — 1×1 (pointwise) convolution: each output channel is a weighted combination of all input channels at a given spatial position.

5.1.4 Batch Normalization Folding

Batch normalization [44] normalizes layer outputs using the mean and variance statistics accumulated during training, then applies a learned scale (γ) and shift (β) per channel. At inference time, the mean and variance are fixed, allowing the normalization to be expressed as a linear function of the layer input. Combined with the linearity of the preceding convolution, this permits the batch normalization transformation to be absorbed into modified convolutional weights and biases:

$$W' = W \cdot (\gamma / \sqrt{(\sigma^2 + \epsilon)}), \quad b' = (b - \mu) \cdot (\gamma / \sqrt{(\sigma^2 + \epsilon)}) + \beta$$

where μ and σ^2 are the per-channel training statistics and ϵ is a small stability constant. The resulting folded weights W' and biases b' are quantized to the per-layer fixed-point format and stored in the on-chip weight ROMs. The `verify_accuracy.py` script confirms that the maximum element-wise difference between the BN-folded float model and the original pretrained model is below 1×10^{-5} , validating the folding procedure before quantization is applied [46].

Section 5.1 has presented the ShuffleNet V2 $\times 1.0$ architecture in the detail required to motivate the hardware mapping developed in Chapter 6. The four design guidelines of Ma et al., the three-group network topology, the stride-1 and stride-2 shuffle block variants, and the batch normalization folding procedure together define the functional specification that the RTL

accelerator must implement. Section 5.2 describes the software modeling pipeline through which this specification is validated before hardware design begins.

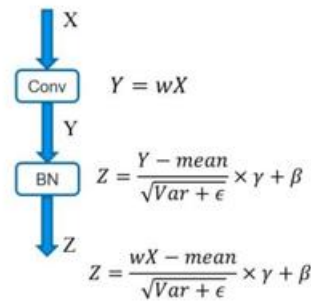


Figure 5.2 Before BN optimization

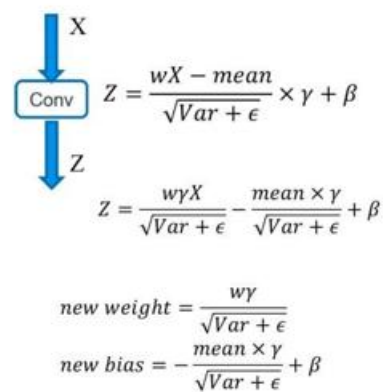


Figure 5.7 — Batch normalization folding derivation: pre-folding convolution+BN cascade (top) and the folded new-weight/new-bias formulas (bottom).

5.2 Software Modeling

5.2.1 Purpose and Methodology

Before any RTL is written, a software model of the quantized network is developed in Python to establish a bit-true reference against which hardware simulation results can be verified. The software model serves three purposes: it validates the batch normalization folding procedure, it confirms that the chosen fixed-point format preserves sufficient accuracy, and it generates the weight and bias arrays in the hexadecimal format required by the Verilog \$readmemh initialization statements of the on-chip weight ROMs.

The modeling pipeline is implemented in scripts/verify_accuracy.py using NumPy for fixed-point arithmetic and PyTorch torchvision for the float32 reference model. The pipeline proceeds in three stages. First, the pretrained ShuffleNet V2 $\times$ 1.0 checkpoint is loaded and batch normalization parameters are folded into the convolutional weights using the formula derived in Section 5.1.4. Second, the folded float weights are quantized to the per-layer fixed-

point formats specified in `shufflenet_pkg.vh`, with saturation applied at the format boundaries. Third, the quantized model is evaluated on a 100-image subset of the ImageNet validation set using bit-true fixed-point arithmetic that replicates the Q6.8 multiply-accumulate pipeline of the hardware datapath, including the 7-LSB truncation after multiplication [46].

5.2.2 Quantization Validation

The validation procedure applies two checks. The first is a weight-level sanity check: the maximum absolute difference between the BN-folded float model output and the quantized model output on a small set of test inputs must be below 1×10^{-5} . This threshold, documented in the README setup instructions [45], confirms that the folding arithmetic is numerically correct before quantization error is introduced. The second check is a network-level accuracy evaluation: the Q6.8 quantized model is expected to achieve approximately 71% top-1 accuracy on the 100-image validation subset, matching the software Q6.8 reference and confirming that the chosen per-layer bit widths preserve classification accuracy [46].

```
Enter number of processing images: 50000
progress: 100.00%   timeToFinish: 0.00 sec
Processing Time: 1:02:09.913881
accuracy = 69.362%
Top-1 error = 30.638%
```

Figure 5.8 — Baseline accuracy verification of the pretrained ShuffleNet V2 $\times$ 1.0 model on the ImageNet validation set (69.362% top-1), matching the published reference accuracy prior to any quantization.

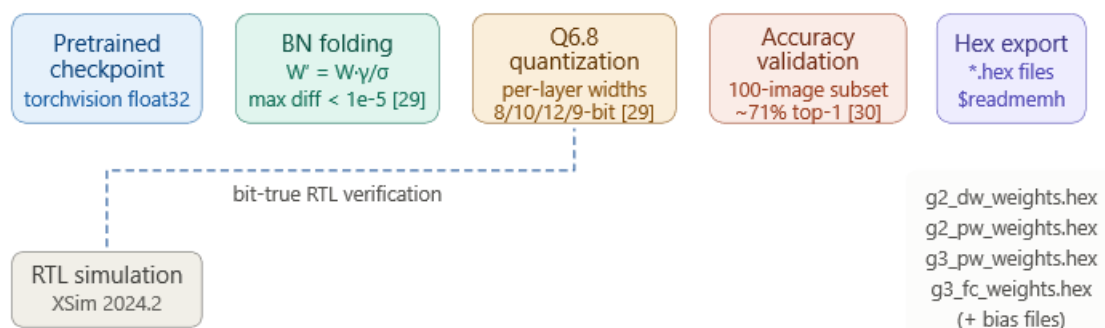


Figure 5.9 — Software modeling pipeline: BN folding $\rightarrow$ quantization $\rightarrow$ accuracy validation $\rightarrow$ hex weight export. Sources: [29][30].

The 71% result on 100 images exceeds the published 69.36% full-dataset figure [26] due to sample variance in the small evaluation set rather than any systematic improvement. The critical validation is consistency: the hardware RTL simulation must produce identical

classification results to the software Q6.8 model on the same input images, confirming bit-true equivalence between the fixed-point software model and the Verilog datapath.

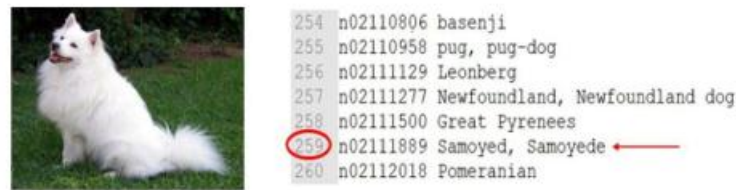


Figure 5.10 — ImageNet validation sample and its ground-truth/predicted class (259, Samoyed).



Figure 5.11 — RTL simulation waveform confirming `class_index = 259`, matching the software prediction.

5.2.3 Weight File Generation

Following validation, the quantized weights and biases for each layer are exported as hexadecimal text files read by the Verilog ROM initialization headers at synthesis time. The Group 2 depthwise weights are written to `g2_dw_weights.hex`, Group 2 pointwise weights to `g2_pw_weights.hex`, Group 3 pointwise weights to `g3_pw_weights.hex`, and the fully connected layer weights to `g3_fc_weights.hex`, with corresponding bias files for each layer [45]. Each file entry represents one quantized coefficient at the bit width specified for that layer — 15 bits for Group 2 depthwise weights, 11 bits for Group 2 pointwise weights, and 9 bits for Group 3 weights and the FC layer — packed into 32-bit words as required by the RAMB36E2 initialization format.

The weight file generation is the critical link between the software modeling pipeline and the hardware implementation: any error in quantization, folding, or bit-packing at this stage will propagate silently through synthesis into functional simulation failures. The testbench suite therefore includes dedicated tests for each weight ROM module, verifying that the initialized values match expected outputs for known inputs before integration-level simulation begins.

Section 5.2 has described the software modeling pipeline that bridges the pretrained float32 network and the fixed-point RTL implementation. The Python-based BN-folding, quantization, validation, and weight-file-generation flow provides the bit-true reference that all subsequent hardware verification steps are measured against. Section 5.3 develops the Q6.8 fixed-point arithmetic specification in detail.

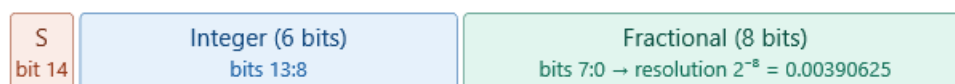
5.3 Quantization and the Q6.8 Fixed-Point Format

5.3.1 Fixed-Point Representation

Fixed-point arithmetic represents real numbers as scaled integers, with the scale factor implicit in the format specification rather than stored alongside each value. The Q6.8 format used as the baseline in this design is a signed 15-bit representation: one sign bit, six integer bits, and eight fractional bits, encoding the value range $[-64, +63.996]$ in decimal with a resolution of $2^{-8} = 0.00390625$. In two's complement binary, the maximum positive value is 0x3FFF (+16383 integer, representing +63.996 decimal) and the minimum value is 0x4000 (−16384 integer, representing −64 decimal) [45].

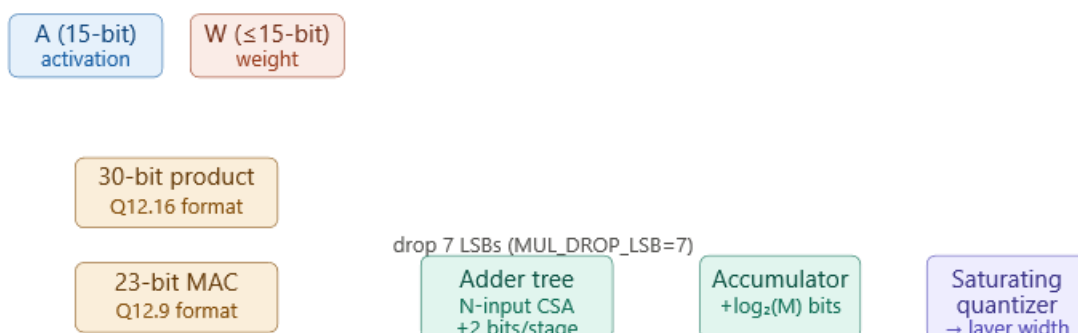
The choice of six integer bits and eight fractional bits reflects the dynamic range of ShuffleNet V2's activation distributions after batch normalization folding. Batch normalization standardizes layer outputs to zero mean and unit variance during training, and the folded weights inherit this normalization. Post-ReLU activations are therefore predominantly in the range $[0, 4]$, well within the six-integer-bit capacity, while the eight fractional bits provide sufficient resolution to represent the small weight values characteristic of the deeper layers without rounding to zero.

Q6.8 fixed-point format (15-bit signed)



Range: $[-64, +63.996]$ decimal | Integer encoding: $[-16384, +16383]$ | DATA_MAX = 0x3FFF, DATA_MIN = 0x4000

MAC pipeline datapath (mac_unit.v)



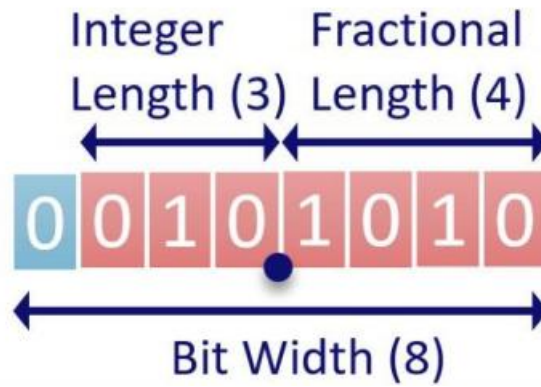


Figure 5.12 — Fixed-point bit width divided into an integer-length field and a fractional-length field.

Fixed-Point vs Floating-Point

Property	Fixed-Point	Floating-Point (IEEE 754)
Hardware complexity	Low	High
Hardware area	Small	Large
Propagation delay	Low	High
Power consumption	Low	High
Memory needed	Small	Large (32 bits per number)
Accuracy	Lower (quantisation noise)	High

Table 5.2: Fixed-point versus floating-point representation, compared across hardware complexity, area, propagation delay, power consumption, memory, and accuracy.

Q6.8 Fixed-Point: 15-bit Signed Two's Complement



Figure 5.13 — Q6.8 fixed-point format: 1 sign bit, 6 integer bits, and 8 fractional bits (15 bits total).

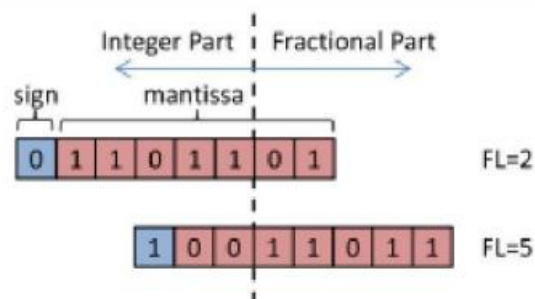


Figure 5.14 — Dynamic fixed-point representation: two values of equal bit width using different fractional lengths (FL), letting each layer choose its own optimal integer/fractional split.

$$\begin{array}{r}
 110.11 \quad -1.25 \\
 + 011.010 \quad +3.25 \\
 \hline
 1010.000 \quad +2
 \end{array}$$

Figure 5.15 — Fixed-point addition: operands are aligned to a common binary point before adding.

$$\begin{array}{r}
 110.11 \quad -1.25 \\
 + 100.001 \quad -3.875 \\
 \hline
 1010.111 \quad -5.125
 \end{array}$$

Figure 5.16 — Overflow in fixed-point addition when the true sum requires one more bit than either operand.

$$\begin{array}{r}
 1110.11 \quad -1.25 \\
 + 1100.001 \quad -3.875 \\
 \hline
 11010.111 \quad -5.125
 \end{array}$$

Figure 5.17 — Overflow resolution in fixed-point addition via sign-extension of both operands before adding.

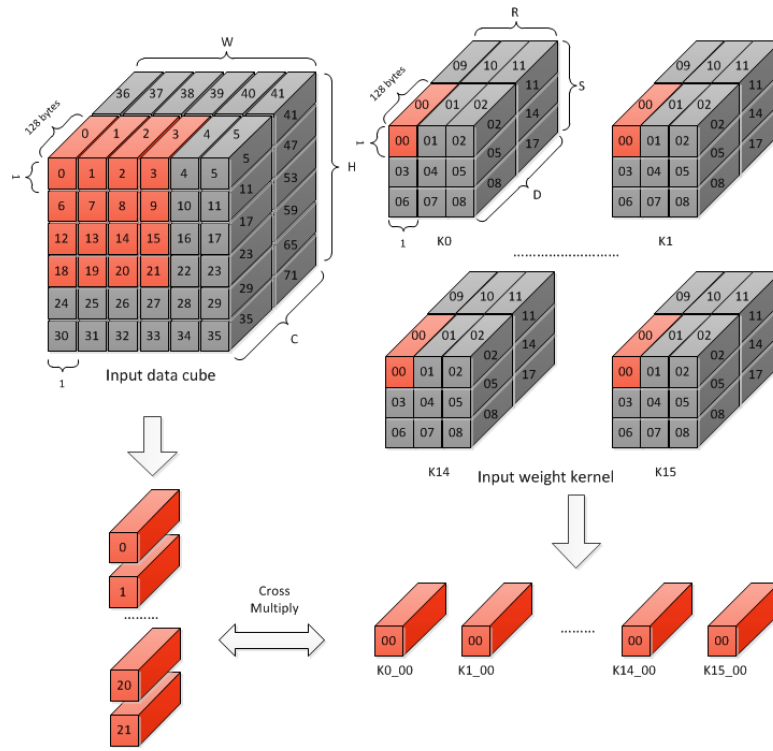


Figure 5.18 — Q6.8 bit-field layout (top) and MAC pipeline datapath (bottom). Source: [29].

5.3.2 Multiply-Accumulate Pipeline

The fixed-point MAC pipeline is the fundamental arithmetic unit of the accelerator and is implemented identically across all convolutional layers in `mac_unit.v`. Given an activation operand A (15-bit signed) and a weight operand W (up to 15-bit signed, with per-layer narrowing applied), the pipeline proceeds as follows.

The full product $A \times W$ is computed as a 30-bit signed result in Q12.16 format ($MUL_FULL_W = 30$). Seven least-significant bits are discarded ($MUL_DROP_LSB = 7$), yielding a 23-bit MAC value in Q12.9 format ($MUL_OUT_W = 23$). This truncation trades arithmetic precision for a reduction in the adder tree input width: the 7 dropped bits correspond to the least-significant fractional bits of the product, whose contribution to the final quantized output is below the resolution of the output format. The 23-bit MAC values from all parallel multiply units feed into a carry-save adder tree — 9-input for the 3×3 convolution window in Group 1, 12-input for the Group 2 pointwise layer, 29-input for the Group 3 pointwise layer, and 32-input for the fully connected layer. Each 3-input adder stage grows the word width by two bits ($ADD3_GROW = 2$) and each 2-input stage by one bit ($ADD2_GROW = 1$). The accumulator that sums across multiple adder-tree outputs grows by $\log_2(M)$ bits, where M is the number of accumulation cycles [45].

$$b = (100010)_2 \times (2^{-3})_{10}$$

$$a \times b = \left((101001)_2 \times (100010)_2 \right) \times (2^{-6})_{10}$$

Figure 4.8 Preparing numbers for multiplication

1							1	0	1	0	0	1		41
2		×					1	0	0	0	1	0		34
3							0	0	0	0	0	0		
4						1	0	1	0	0	1			
5				0	0	0	0	0	0	0				
6			0	0	0	0	0	0	0					
7			0	0	0	0	0	0						
8		+	1	0	1	0	0	1						
9			1	0	1	0	1	1	1	0	0	1	0	1394

Figure 5.19 — Preparing operands for fixed-point multiplication (top) and an unsigned $\times$ unsigned multiplication worked example (bottom).

1						1	1	0	0	1	-7
2	x					1	0	0	1	0	-14
3		0	0	0	0	0	0	0	0	0	
4		1	1	1	1	1	0	0	1		
5		0	0	0	0	0	0	0	0		
6		0	0	0	0	0	0	0			
7	+	0	0	0	1	1	1				
8		0	0	0	1	1	0	0	1	0	98

Figure 5.20 — Signed $\times$ signed fixed-point multiplication: the final partial product is negated to represent the two's-complement operand correctly.

5.3.3 Per-Layer Width Optimization

The uniform Q6.8 baseline is refined by per-layer width optimization applied to both feature maps and weights. Feature map widths are reduced from the baseline 15 bits to the minimum width that preserves classification accuracy, as determined by the software modeling experiments: 8 bits for the input image buffer, 10 bits for Group 1 output, 12 bits for Group 2 output, and 9 bits for Group 3 and the FC layer [45].

Floating Point 32-bit Accuracy			69.362%
Bit Width	Integer length	Fractional Length	Accuracy
17	6	10	69.244%
16	7	8	66.578%
16	6	9	68.704%
16	5	10	68.068%
15	6	8	66.576%
15	5	9	67.532%
15	4	10	38.744%
14	4	9	38.476%
14	5	8	65.454%
14	6	7	55.950%
13	4	8	38.246%
13	5	7	55.054%

Table 5.3: Quantization sweep across candidate bit-width and fractional-length combinations, evaluated on the full 50,000-image ImageNet validation set.

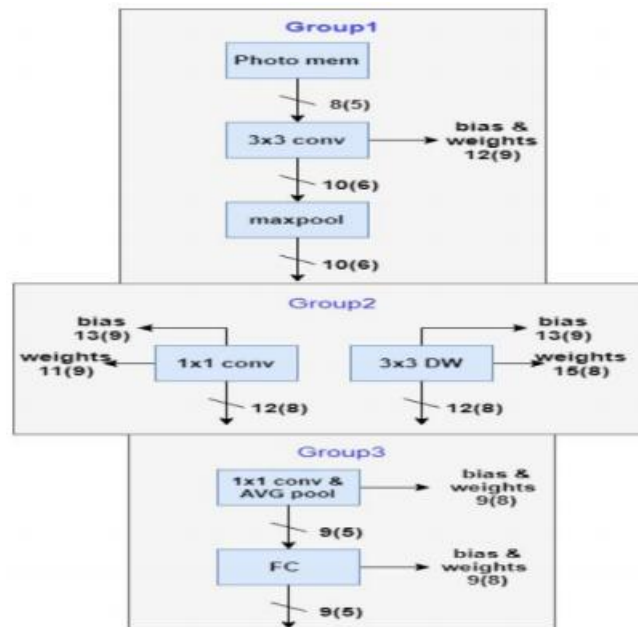


Figure 5.21 — Per-module weight and bias bit-widths selected by the dynamic (per-layer) quantization scheme across Group 1, Group 2, and Group 3.

Table 5.4: Per-Layer Fixed-Point Feature Map and Weight Specifications

Layer	Feature Map Width	Frac. Bits (FM)	Weight Width
Photo memory (input pixels)	8-bit	5	—
Group 1 — 3×3 Conv	10-bit (output)	6	12-bit
Group 2 — DW 3×3	12-bit (output)	8	15-bit (unchanged)
Group 2 — PW 1×1	12-bit (output)	8	11-bit
Group 2 — PW biases	—	—	13-bit
Group 3 / FC	9-bit (output)	5	9-bit
Group 3 — FC biases	—	—	11-bit

Weight widths are similarly reduced: Group 1 convolution weights use 12 bits, Group 2 depthwise weights retain the full 15 bits (as they exhibited the greatest sensitivity to precision reduction in software experiments), Group 2 pointwise weights use 11 bits, Group 3 pointwise weights and FC weights use 9 bits [45]. Per the shufflenet_pkg.vh comment, this per-layer weight width optimization reduces total BRAM weight storage from 1,453 to 1,155.5 BRAM tiles — a saving of approximately 297.5 tiles that is critical to fitting all weights within the ZU19EG's 984 BRAM-tile budget alongside the feature map buffers.

5.3.4 Group 3 Datapath Width Derivation

The Group 3 pointwise convolution warrants separate treatment because its internal accumulator width differs significantly from the baseline. Using the per-layer parameters from `shufflenet_pkg.vh`: the activation width is $G2_FM_W = 12$ bits and the weight width is $G3_PW_WW = 9$ bits. The product width before truncation is $12 + 9 = 21$ bits. Applying $MUL_DROP_LSB = 7$ yields a MAC output of 14 bits ($MUL_OUT_W = 14$ for this layer). The 29-input adder tree adds $\log_2(29) \approx 5$ bits of growth, producing a tree output of 19 bits. The accumulator across the 16 filter groups adds $\log_2(16) = 4$ bits, yielding 23 bits before bias addition. Adding the bias term extends to $G3_CONV_OUT_W = 27$ bits. In the baseline uniform-15-bit design, the equivalent width would be 35 bits; the per-layer optimization reduces it to 27 bits, shrinking the adder tree and accumulator logic significantly and contributing to the low LUT utilization of 14.34% observed in the post-implementation report [45].

The Group 3 convolution output bypasses the saturating quantizer entirely — one of the eight RTL optimizations documented in the design — because the global average pooling and fully connected layer that follow operate directly on the accumulated values without requiring re-quantization to the standard output format. This optimization eliminates the quantizer logic for the most resource-intensive stage of Group 3 and reduces one potential source of precision loss in the deepest part of the network [45].

Section 5.3 has specified the complete fixed-point arithmetic framework of the accelerator: the Q6.8 baseline format, the 30-to-23-bit MAC pipeline, the saturating quantizer, the per-layer feature map and weight widths, and the Group 3 datapath width derivation. Section 5.4 applies this framework layer by layer, profiling the arithmetic and memory requirements of each stage of the ShuffleNet V2 inference graph.

5.4 Layer-by-Layer Profile

5.4.1 Group 1: Initial Feature Extraction

Group 1 performs the highest spatial-bandwidth operation in the network, processing an input image of $224 \times 224 \times 3$ pixels. The 3×3 convolution with 24 filters and stride 2 computes $24 \times (112 \times 112) \times 9 = 27,095,040$ MAC operations. With three input channels processed in parallel and three spatial positions unrolled per filter unit, the RTL implementation instantiates

24 filter units each containing a 9-input MAC array — requiring $24 \times 9 = 216$ DSP48E2 slices for the multiply stage [45].

The Group 1 sliding window datapath uses a three-row FIFO (`fifo_3x3.v`) of depth $G1_CONV_FIFO_DEPTH = 455$ words, derived as $2 \times 226 + 3$ where 226 accounts for the 224-pixel input row plus one zero-padding column on each side. The max pooling stage that follows uses a separate FIFO bank (`fifo_pool.v`) of depth $G1_POOL_FIFO_DEPTH = 231$ words. Both FIFOs are implemented as shift-register chains using the SRLC32E primitives mapped to LUT-as-ShiftRegister resources, consistent with the 8,594 LUT-as-shift-register count in the post-implementation utilization report [45].

The Group 1 output feature map ($56 \times 56 \times 24$, 10-bit) is stored in `maxpool_mem`: 24 channels $\times 3,136$ words $\times 10$ bits = 752,640 bits, requiring approximately 23.5 BRAM tiles. This buffer links Group 1 and Group 2 and is read sequentially by the Group 2 depthwise convolution controller.

5.4.2 Group 2: Shuffle Block Stages

Group 2 is the dominant computational stage of the network, containing all 16 shuffle blocks across Stages 2, 3, and 4. The depthwise 3×3 convolution processes 58 channels in parallel (the maximum channel parallelism across all stages, set to the Stage 2 channel count), with all 9 spatial positions of the 3×3 window computed simultaneously — $58 \times 9 = 522$ DSP48E2 slices. The pointwise 1×1 convolution produces 58 output filters with 12 input channels in parallel — $58 \times 12 = 696$ DSP48E2 slices [45].

The Group 2 FIFO bank (`group2_fifo.v`) uses a single depth parameter $G2_FIFO_DEPTH = 119$ words ($= 2 \times 58 + 3$), sized for the widest stage (Stage 2, 58 feature-map columns including zero padding). The same FIFO structure serves all three stages by reconfiguring the active depth and channel counts through the `group2_ctrl.v` sequencer. This reuse of a single FIFO implementation across stages with different spatial dimensions is a key area reduction: a naive approach that instantiated separate FIFOs for each stage would require approximately three times the BRAM resources.

The inter-stage feature map data grows in channel count and shrinks in spatial extent across the three stages: $56 \times 56 \times 24 \rightarrow 28 \times 28 \times 116 \rightarrow 14 \times 14 \times 232 \rightarrow 7 \times 7 \times 464$. The product of spatial area and channel count (the total feature map volume) decreases from 75,264 to 23,432 elements across Stage 2, then to 45,248 and finally 22,736 elements — a nearly

monotonic reduction that reflects the network's design for decreasing memory footprint in later stages.

5.4.3 Group 3: Classification Head

Group 3 processes the compact $7 \times 7 \times 464$ feature map produced by Group 2. The 1×1 pointwise convolution with 1,024 filters uses 16 filter groups each with 29 input channels — $16 \times 29 = 464$ DSP48E2 slices [45]. Despite the large output channel count of 1,024, the small 7×7 spatial extent means the total MAC count for this layer is $7 \times 7 \times 464 \times 1,024 = 2,328,576$ — less than 10% of the Group 2 MAC total, confirming that Group 2 is the dominant arithmetic stage.

The global average pooling over the 7×7 spatial extent divides each of the 1,024 channel sums by 49. In the RTL implementation, this division is replaced by a right-shift approximation — one of the eight documented design optimizations — which introduces a relative error of $(49 - 32) / 49 \approx 2.8\%$ but eliminates the need for a hardware divider. The fully connected layer with 1,024 inputs and 1,000 outputs uses 32 parallel input channels — 32 DSP48E2 slices in the `fc_core.v` implementation — and cycles through the 1,024 input dimensions in 32 steps of 32-channel parallel accumulation [45].

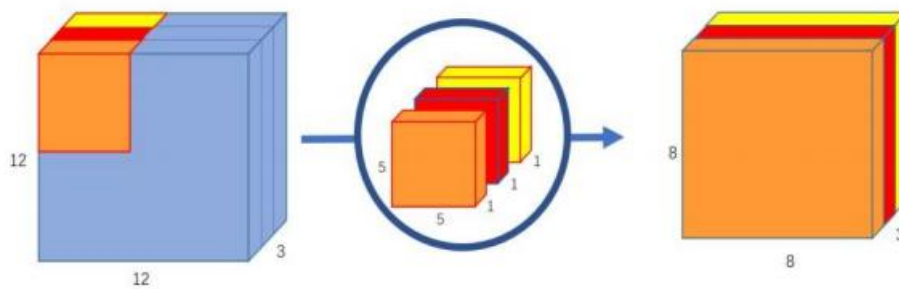


Figure 5.22 — Average-pooling with a 3×3 window, replacing the maximum with the mean of each window.

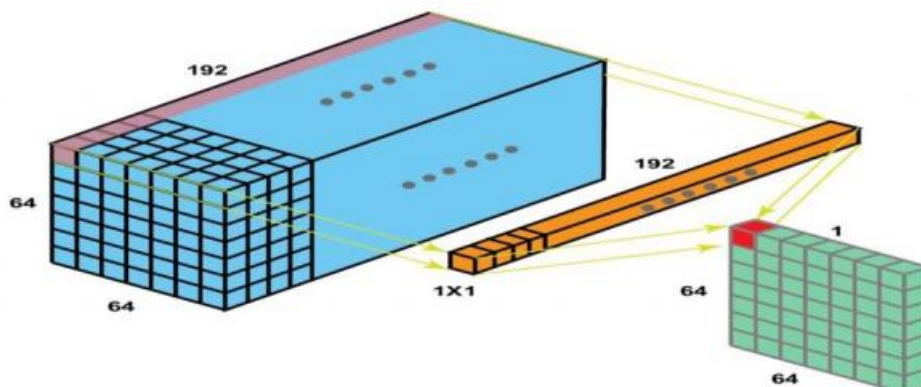


Figure 5.23 — Fully connected layer: a flattened input vector is mapped to per-class scores through a bank of weighted sums.

Table 5.5: Layer-by-Layer DSP Budget Summary

Block	DSP Count
Group 1 — 3×3 Conv	216
Group 2 — DW 3×3	522
Group 2 — PW 1×1	696
Group 3 — PW 1×1	464
Group 3 — FC	32
Total (calculated)	1,930
ZU19EG available	1,968
Headroom (calculated)	38

Section 5.4 has profiled each layer group's arithmetic requirements, memory footprint, and resource allocation, establishing the quantitative basis for the hardware mapping developed in Chapter 6. The dominant contributions are Group 2's 1,218 DSPs for depthwise and pointwise convolution, Group 1's sliding-window FIFO datapath, and Group 3's compact classification head with its right-shift average pool approximation. Chapter 6 translates this profile into a concrete hardware architecture.

Chapter 6: Hardware Mapping and Architectural Analysis

6.1 Model-to-Hardware Mapping

6.1.1 Mapping Strategy

The translation of the ShuffleNet V2 inference graph into a hardware architecture requires resolving four fundamental questions: which operations execute concurrently versus sequentially, how weight data is stored and accessed, how feature map data flows between successive layers, and how the available DSP and BRAM resources are partitioned across the three network groups. The mapping strategy adopted in this work is spatial decomposition by group: each of the three network groups is implemented as an independent RTL module cluster that executes sequentially within a group but overlaps in time with the adjacent group via a ping-pong or handshake buffering scheme. Within each group, spatial parallelism across filters and input channels is maximized subject to the DSP budget constraint of 1,968 slices [45].

This strategy differs from the time-multiplexed single-engine approach used by some prior works [28], in which a single configurable convolution engine is reconfigured for each layer at runtime. The single-engine approach reduces area at the cost of latency, since each layer must be processed serially. The present design accepts the higher area cost of three independent engine clusters — reflected in the 78.76% DSP and 58.69% BRAM utilization — in exchange for the latency reduction achieved by pipelining Groups 1, 2, and 3 across consecutive input images.

6.1.2 Operation-to-Module Assignment

Each arithmetic operation in the ShuffleNet V2 graph is assigned to a dedicated RTL module with fixed resource allocation, as summarized below.

The Group 1 3×3 convolution maps to `conv3x3_core.v`, which instantiates 24 copies of `conv3x3_filter_unit.v`. Each filter unit contains a 9-input MAC array operating on three parallel input channels across three spatial window positions, fed by the three-row sliding window FIFO `fifo_3x3.v`. Weights are stored in `weights_rom_3x3.v` as distributed LUT RAM, providing combinational read access with zero pipeline latency. The max pooling operation maps to `maxpool_core.v`, which computes the maximum of nine values across a 3×3 window for all 24 channels in parallel, using the `fifo_pool.v` FIFO bank for sliding window buffering [45].

Operation	RTL module	DSP48E2 (calc.)
G1 — Conv 3×3 (24 filters, stride 2)	conv3x3_core.v	216
G1 — MaxPool 3×3	maxpool_core.v	0 (comparators)
G2 — DW Conv 3×3	dw_conv3x3_core.v	522
G2 — PW Conv 1×1	conv1x1_core.v	696
G3 — PW Conv 1×1	conv1x1_g3_core.v	464
G3 — Avg Pool 7×7	avg_pool_core.v	0 (right-shift)
G3 — FC (1024→1000)	fc_core.v	32
Total calculated: 1,930 DSPs Post-implementation measured: 1,550 / 1,968 (78.76%) [29]		

Figure 6.1 — Operation-to-RTL-module mapping with calculated DSP48E2 allocation. Source: [29].

The Group 2 depthwise convolution maps to `dw_conv3x3_core.v` with 58 parallel filter units, each containing a 9-MAC array with a dedicated weight ROM entry. The pointwise convolution maps to `conv1x1_core.v` with 58 filter units each accumulating across 12 parallel input channels. Group 2 weight ROMs — `g2_dw_weight_rom.v` and `g2_pw_weight_rom.v` — are implemented as distributed LUT RAM for combinational read access, consistent with the 12,064 RAMD64E primitives observed in the post-implementation report [45].

The Group 3 pointwise convolution maps to `conv1x1_g3_core.v` with 16 filter group units each accumulating across 29 parallel input channels. The global average pooling maps to `avg_pool_core.v`, which accumulates 49 values per channel across the 7×7 spatial extent and approximates division by 49 using a right-shift. The fully connected layer maps to `fc_core.v`, which processes 1,000 output neurons by cycling through the 1,024 input dimensions in steps of 32 parallel channels. Group 3 weight ROMs — `g3_pw_weight_rom.v` and `g3_fc_weight_rom.v` — are implemented as BRAM (RAMB36E2) due to their larger size, with synchronous read access pipelined through the convolution controller [45].

6.1.3 Control Hierarchy

Each group implements a two-level control hierarchy. At the lower level, individual operation controllers (`conv3x3_filter_unit.v`, `dw_conv3x3_ctrl.v`, `conv1x1_ctrl.v`) manage the cycle-by-cycle address generation, weight ROM indexing, and accumulator reset signals for a single convolution type. At the upper level, group sequencers (`group1_ctrl.v`, `group2_ctrl.v`, `group3_ctrl.v`) coordinate the ordering of operations within a group — conv scan followed by

pool scan in Group 1; DW phase followed by PW phase across all 16 blocks in Group 2; PW, pool, and FC phases in Group 3 — and assert done handshake signals to `accelerator_ctrl.v` upon group completion [45].

The top-level accelerator controller (`accelerator_ctrl.v`) implements a five-state FSM: IDLE, GROUP1, GROUP2, GROUP3, and DONE. Transitions are triggered by the done handshakes from each group controller. The DONE state asserts `classification_done` in the AXI CSR, making the `class_idx` field readable by the ARM host. On receipt of the next `photo_ready` assertion, the FSM returns to GROUP1 and begins processing the next image from the alternate ping-pong buffer.

Figure 6.3 traces the resulting five-state FSM directly, showing the transition conditions (`Photo_Ready`, `Done1/2/3`, `Busy_Flag`) that drive the pipelined Group 1/2/3 execution overlap described above.

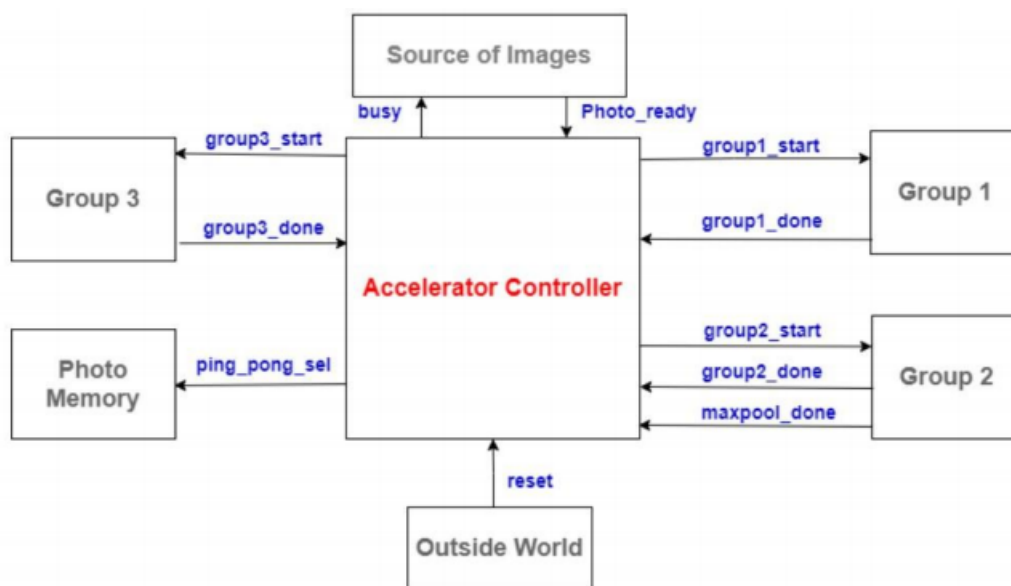


Figure 6.2 — Input and output signals of the top-level accelerator controller, including its handshake with the image source, Groups 1-3, and the ping-pong photo memory.

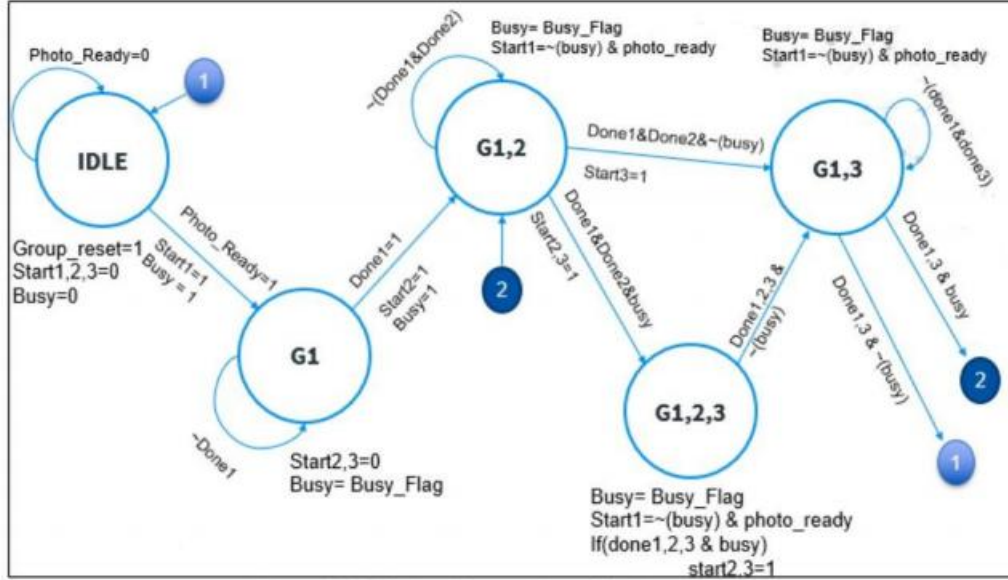


Figure 6.3 — Accelerator controller state diagram: five states (IDLE, G1, G1_2, G1_2_3, G1_3) with the exact busy-flag and done-signal conditions governing each transition.

Section 6.1 has established the mapping from ShuffleNet V2 operations to RTL modules, the spatial parallelism strategy, and the two-level control hierarchy that coordinates execution across groups. Section 6.2 analyzes the data flow through the complete inference pipeline.

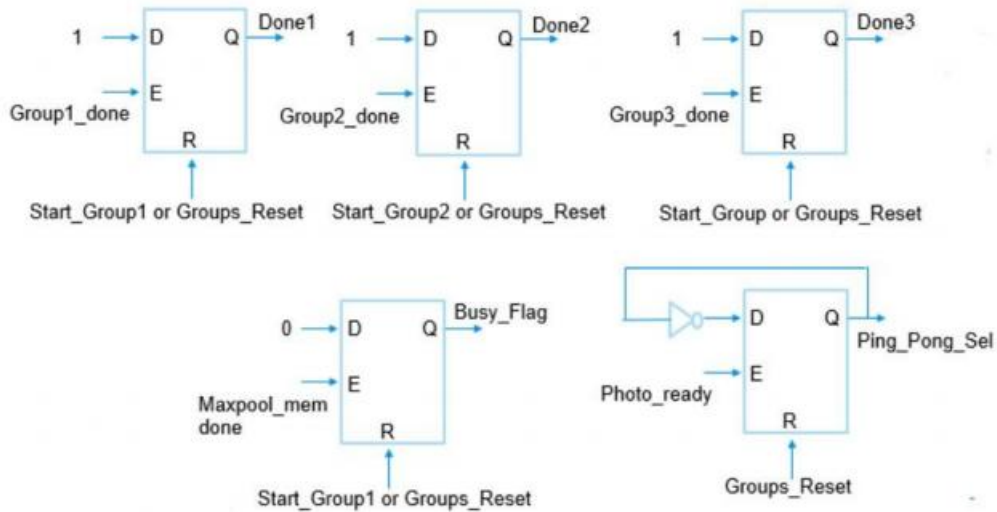


Figure 6.4 — Accelerator controller status-flag registers: D-flip-flops latching the Group 1/2/3 done pulses, the busy flag, and the ping-pong select signal, ensuring no done pulse is missed while multiple groups are active.

6.2 Data Flow Analysis

6.2.1 Intra-Group Data Flow

Within Group 1, data flows from the ping-pong image buffer `photo_mem` through the three-row FIFO `fifo_3x3.v` into the 24-filter convolution core, through the intermediate feature map (stored transiently in registers), through the max pooling FIFO bank `fifo_pool.v`, and finally into the `maxpool_mem` BRAM output buffer. The FIFO-based sliding window eliminates the need to store the entire intermediate feature map between the convolution and pooling stages: at any given clock cycle, only three rows of the $112 \times 112 \times 24$ intermediate feature map are live in the FIFO, while the remainder has already been consumed by the pooling operation. This reduces the on-chip storage requirement for the conv-to-pool interface from $112 \times 112 \times 24 \times 10 \text{ bits} \approx 3.0 \text{ MB}$ to $455 \times 24 \times 10 \text{ bits} \approx 109 \text{ KB}$ — a reduction of approximately $28\times$ [45].

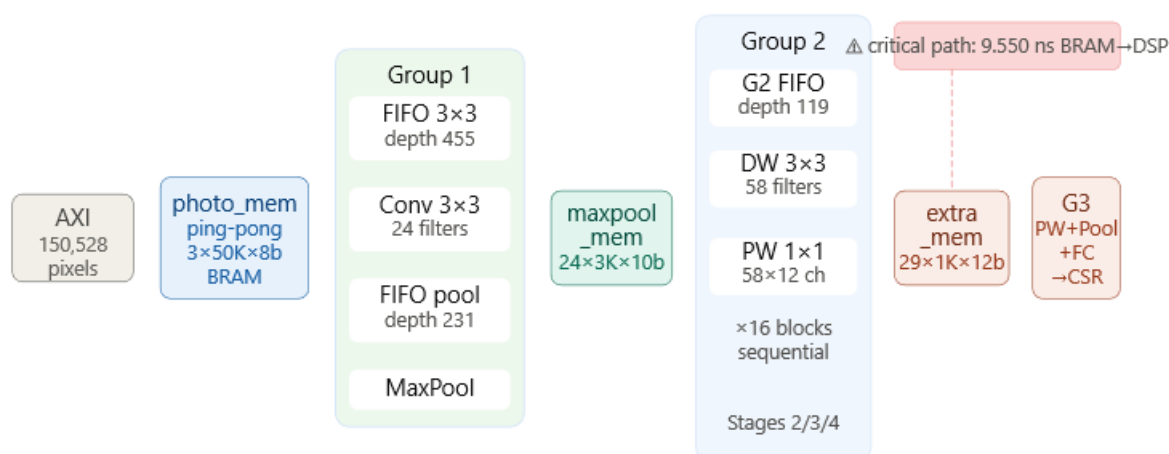


Figure 6.5 — Inference data flow: AXI ingestion → Group 1 → `maxpool_mem` → Group 2 → `extra_mem` → Group 3 → CSR. Source: [29].

Within Group 2, data flows from `maxpool_mem` through the `group2_fifo.v` bank into the depthwise convolution core, through an internal DW output buffer, into the pointwise convolution core, and finally into the `extra_mem` BRAM output buffer that links Group 2 to Group 3. The same FIFO structure — depth 119, sized for the widest stage — serves all three stages of Group 2 by reconfiguring channel count and spatial extent parameters through `group2_ctrl.v`. Between the DW and PW operations within each shuffle block, the DW output is held in a compact internal buffer rather than written to BRAM, avoiding an unnecessary memory round-trip for data that is consumed within the same clock domain immediately after production [45].

Within Group 3, data flows from `extra_mem` through `conv1x1_g3_core.v` directly into `avg_pool_core.v` without intermediate buffering, exploiting the fact that the 7×7 spatial extent is small enough that the complete pooling accumulation for one channel can be completed before the next channel's convolution output is required. The FC layer reads from the average pool output register array and writes classification logits to a 1,000-entry comparison register, from which the argmax unit extracts the `class_idx` value [45].

6.2.2 Inter-Group Data Flow and Ping-Pong Buffering

The three inter-group boundaries — input to Group 1, Group 1 to Group 2, and Group 2 to Group 3 — each use a distinct buffering strategy reflecting the different data volumes and timing relationships at each boundary.

The input boundary uses a ping-pong BRAM buffer (`photo_mem`, $3 \times 50,176 \times 8$ -bit words per ping-pong half). While Group 1 reads from one half to begin processing frame N , the AXI slave controller `axi_photo_mem_slave.v` writes frame $N+1$ pixel data into the other half. The switch between halves is controlled by a single bit in the AXI CSR, toggled by `accelerator_ctrl.v` upon Group 1 completion. This arrangement ensures that the host ARM processor can begin transferring the next image immediately after asserting `photo_ready`, without waiting for the current inference to complete [45].

The Group 1-to-Group 2 boundary uses `maxpool_mem` as a single-buffer handshake: Group 1 writes its complete output into `maxpool_mem`, then asserts its done signal, and Group 2 begins reading only after this assertion. No ping-pong is needed here because Group 1 completes before Group 2 starts — the two groups do not overlap at this boundary within a single frame, only across consecutive frames at the system level.

The Group 2-to-Group 3 boundary uses `extra_mem` ($29 \times 1,024 \times 12$ -bit) in the same single-buffer handshake pattern. Group 3 begins reading `extra_mem` only after Group 2's done signal is asserted by `group2_ctrl.v` [45].

6.2.3 AXI Data Flow

Pixel data arrives from the ARM host through the AXI4-Lite slave interface implemented in `axi_photo_mem_slave.v`. The slave decodes the `AWADDR[21]` bit to distinguish pixel writes (`AWADDR[21] = 0`, routed to `photo_mem`) from CSR accesses (`AWADDR[21] = 1`, routed to the control register logic). All $224 \times 224 \times 3 = 150,528$ pixel values are written sequentially before inference is triggered, with each 32-bit AXI write transaction carrying one 8-bit pixel

value in its least-significant byte. The AXI interface operates on the PS clock domain (clk_pl_0), while the accelerator datapath operates on the MMCM-generated 100 MHz clock (clk_100m); clock domain crossing is handled within shufflenet_board_top.v through the synchronization logic integrated with the AXI SmartConnect IP [45].

Section 6.2 has traced the complete data flow through the inference pipeline, from AXI pixel ingestion through the three-group processing chain to CSR output. Section 6.3 identifies the bottlenecks within this flow that constrain overall inference throughput.

6.3 Bottleneck Identification

6.3.1 Computational Bottleneck

The dominant computational bottleneck is Group 2, which accounts for the majority of the network's total MAC operations. The 16 shuffle blocks of Stages 2, 3, and 4 collectively execute depthwise and pointwise convolutions across feature maps ranging from 56×56 to 7×7 in spatial extent and from 24 to 464 in channel count. The Group 2 depthwise convolution alone consumes 522 DSP48E2 slices and the pointwise convolution consumes 696 slices — a combined 1,218 DSPs representing 62.6% of the device total and 78.8% of the design's total DSP usage of 1,550 slices. The Group 2 controller must execute the DW and PW phases for each of the 16 shuffle blocks in turn, reusing the same hardware across all blocks but incurring a sequential scheduling cost proportional to the number of blocks [45].

The critical timing path identified by the post-implementation analysis runs from a BRAM read in u_extra_mem through five levels of DSP48E2 chaining in u_g3/u_conv1x1, spanning a data path delay of 9.523 ns — leaving only 0.477 ns of logic slack before the 10.000 ns clock period boundary, which after routing delay yields the observed WNS of +0.096 ns. This path is not in Group 2 but in Group 3, because the Group 3 PW convolution reads directly from BRAM and chains immediately into DSP accumulation without an intervening pipeline register, making it the tightest path in the design despite Group 2 being the dominant arithmetic stage [45].

6.3.2 Memory Bottleneck

The primary memory bottleneck is the extra_mem buffer at the Group 2-to-Group 3 boundary. This buffer stores $29 \times 1,024 \times 12\text{-bit} = 356,352$ bits of feature map data and is the largest single inter-group buffer in the design. Its read port is the direct source of the critical timing path: the BRAM read latency (one clock cycle for a registered RAMB36E2 read) combined

with the subsequent DSP chain delay constitutes the bottleneck that limits the maximum clock frequency of the routed design [45].

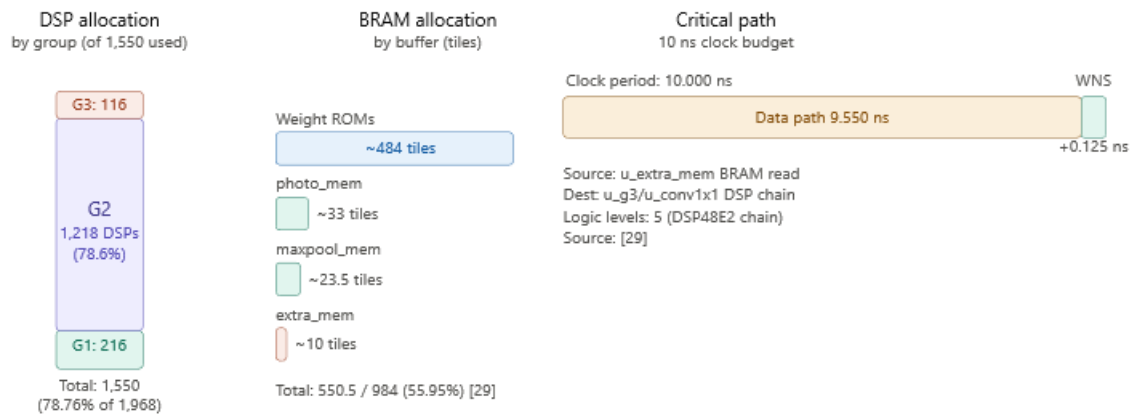


Figure 6.6 — Bottleneck summary: DSP allocation by group, BRAM by buffer, and critical timing path. Source: [29].

The maxpool_mem buffer at the Group 1-to-Group 2 boundary is smaller ($24 \times 3,136 \times 10$ -bit = 752,640 bits) but wider in terms of read bandwidth: Group 2's depthwise convolution reads 58 channels simultaneously, requiring 58 parallel read ports. In the RTL implementation, this is resolved by organizing maxpool_mem as 24 independent single-port BRAM instances, with the Group 2 controller reading from multiple banks in parallel. The total BRAM consumption of maxpool_mem across 24 channels at 10-bit width is approximately 23.5 tiles [45].

6.3.3 Throughput Bottleneck and Pipeline Efficiency

At the system level, the throughput bottleneck is determined by the slowest group in the pipeline — the group with the longest execution time per frame. Board-level deployment confirms an overall pipeline period of 0.822 ms per frame (1,216 fps) at the verified 100 MHz clock; the relative execution times of the three individual groups have not been separately instrumented, but can be estimated from their computational loads: Group 2 processes approximately 16 shuffle blocks each requiring DW and PW phases over feature maps of up to 56×56 , making it substantially longer than Group 1 (a single conv plus pool over 224×224) or Group 3 (a single 7×7 PW conv plus pool plus FC). Group 2 therefore determines the pipeline throughput, and optimizing its sequential block scheduling — or parallelizing across shuffle blocks — represents the highest-leverage direction for future performance improvement.

The ping-pong input buffer partially mitigates this bottleneck at the system level: the ARM host can transfer the next image while Group 2 is still executing on the current image, hiding

the AXI pixel-write latency (150,528 byte-wide AXI transactions) behind the Group 2 computation. The net effect is that the effective throughput of the pipelined system is higher than would be achievable with a sequential execute-then-transfer protocol, though the exact improvement factor depends on the relative durations of Group 2 execution and AXI transfer, which have not been separately instrumented; the measured system-level effect is the overall 1,216 fps pipeline throughput reported in Chapter 9 [45].

Section 6.3 has identified the three principal bottlenecks of the design: the computational bottleneck in Group 2's sequential shuffle block scheduling, the timing-critical BRAM-to-DSP path at the Group 3 input, and the throughput bottleneck that Group 2's execution time imposes on the pipeline. These bottlenecks directly motivate the architectural choices developed in detail in Chapter 7.

Chapter 7: RTL Accelerator Design and Implementation

7.1 Top-Level Architecture

7.1.1 System Hierarchy

The complete hardware system is organized as three concentric layers of hierarchy. The outermost layer is the Vivado block design wrapper (`system_wrapper`), which instantiates the Zynq UltraScale+ processing system IP (`system_zynq_ultra_ps_e_0_0`), the AXI SmartConnect interconnect (`system_smartconnect_0_0`), and the board-level top module (`system_shufflenet_board_top_0_0`). The middle layer is `shufflenet_board_top.v`, which contains the MMCM clock generator, the four-stage reset synchronizer, and the AXI4-Lite slave interface, and which instantiates `accelerator_top.v` as its sole datapath component. The innermost layer is `accelerator_top.v`, which integrates the three group modules (`group1_top`, `group2_top`, `group3_top`), the inter-group memories (`photo_mem`, `maxpool_mem`, `extra_mem`), and the top-level FSM (`accelerator_ctrl.v`) [45].

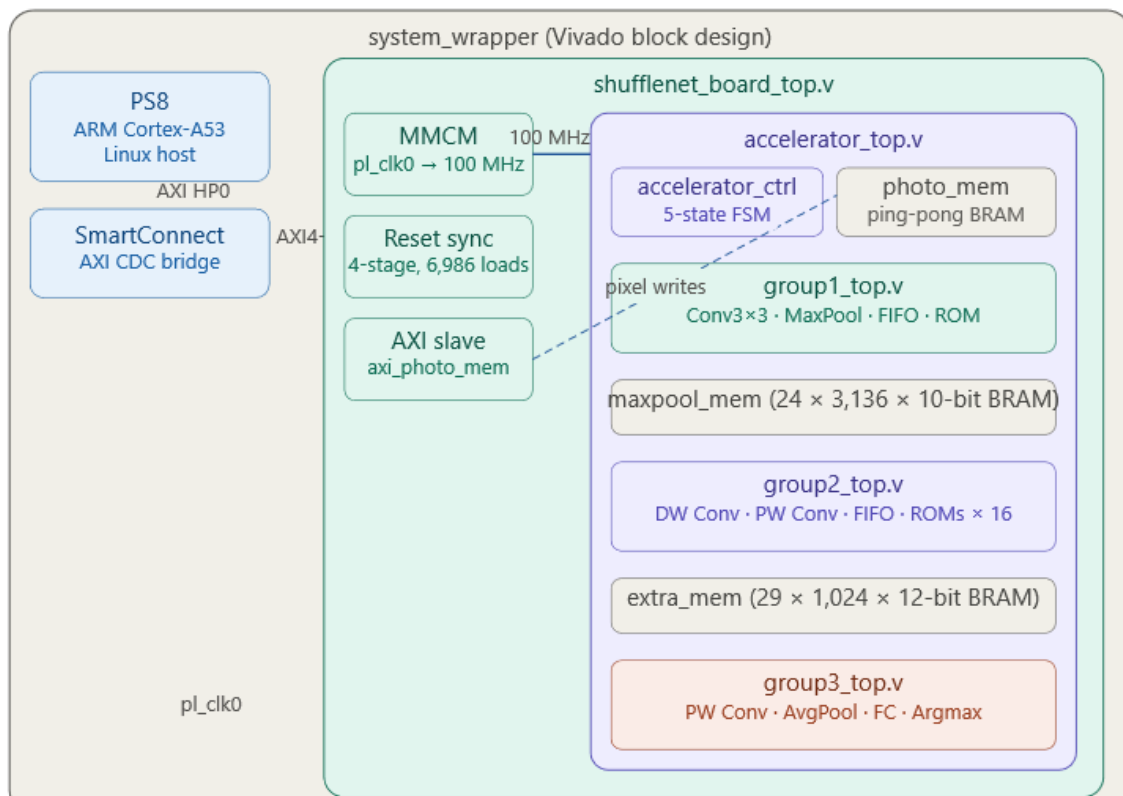


Figure 7.1 — Three-level system hierarchy showing module instantiation, clock/reset distribution, and AXI data paths. Source: [29].

The split synthesis flow instantiates group1_top_bb.v, group2_top_bb.v, and group3_top_bb.v as black-box stubs during top-level synthesis, allowing each group to be synthesized as an independent out-of-context (OOC) checkpoint using dedicated Tcl scripts (vivado/work/part1_*.tcl through part3_*.tcl). The OOC checkpoints are then merged at implementation (part0_impl.tcl), where Vivado performs placement and routing across the full design with full visibility of all three group netlists simultaneously. This flow reduces incremental re-synthesis time — rebuilding a single group takes approximately 3 to 11 minutes rather than re-running the full 1–3 hour implementation — and allows each group's synthesis to be parallelized across workstations [45].

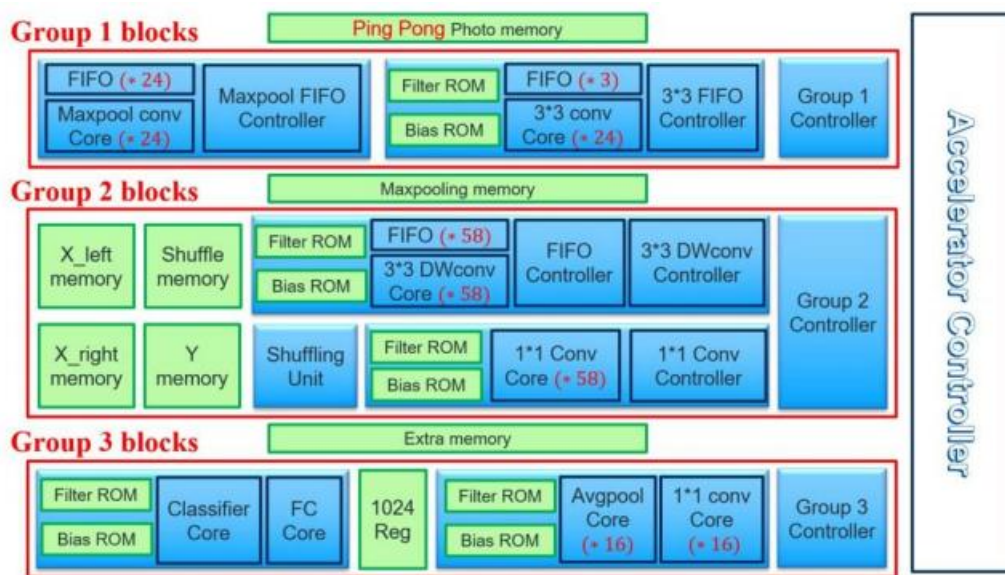


Figure 7.2 — Complete hierarchy of Group 1, Group 2, and Group 3 blocks composing the accelerator, alongside a summary of the parallelism factor used in every computation core.

7.1.2 Clock and Reset Architecture

A single clock domain governs all programmable logic. The PS exports pl_clk0 at a nominal frequency, which is conditioned by a Mixed-Mode Clock Manager (MMCM, MMCME4_ADV primitive) within shufflenet_board_top.v to produce the 100 MHz accelerator clock clk_100m. The MMCM consumes one of the ZU19EG's 11 available MMCM instances (9.09% utilization) and drives the clock distribution network through a BUFGCE global buffer [45].

Reset is active-low and synchronized to clk_100m through a four-stage shift register (rst_shreg_reg[3:0], implemented using FDPE set-on-reset flip-flops) that fans out to 6,986 load cells across the PL fabric. This synchronizer ensures that reset deassertion is glitch-free and meets the recovery and removal timing requirements checked by Vivado's async_default

path group. The post-implementation timing report confirms all recovery and removal checks are met with a worst-case slack of +1.364 ns [45].

7.1.3 AXI Interface Architecture

The AXI4-Lite slave interface (`axi_photo_mem_slave.v`) provides the sole communication channel between the ARM processing system and the PL accelerator fabric. The interface operates at 32-bit data width and 32-bit address width, mapped to base address 0xA0000000 with a 32 MB address window. Address bit 21 (`AWADDR[21]`) functions as a region selector: when cleared, write transactions are routed to the pixel data path, writing 8-bit pixel values into `photo_mem`; when set, transactions access the 32-bit CSR containing the `photo_ready`, `busy`, `classification_done`, and `class_idx` fields [45].

The AXI slave operates on the PS clock domain `clk_pl_0`, while the accelerator datapath operates on `clk_100m`. Clock domain crossing between these two domains is managed within `shufflenet_board_top.v` through the AXI SmartConnect IP, which implements the necessary synchronization FIFOs and handshake logic. The post-implementation inter-clock timing report confirms all CDC paths are met, with the smallest margin of +0.164 ns on the hold path from `clk_pl_0` to `clk_100m` through the AXI address routing [45].

Section 7.1 has described the three-level system hierarchy, the split synthesis flow, the single-domain clock architecture, and the AXI4-Lite interface. Section 7.2 specifies the I/O interface of the accelerator in detail.

7.2 I/O Interface

7.2.1 Accelerator Top-Level Ports

The `accelerator_top.v` module exposes a minimal interface to `shufflenet_board_top.v`, comprising the system clock, the synchronous active-low reset, the AXI-facing pixel data bus and write-enable signals from `axi_photo_mem_slave.v`, the `photo_ready` control signal, the `busy` and `classification_done` status signals, and the 10-bit `class_idx` output [45].

Internally, `accelerator_top.v` connects these top-level ports to the three group modules and the inter-group memories through a set of shared control and data buses managed by `accelerator_ctrl.v`. No external memory interfaces are present: the design is entirely self-contained within the PL fabric, with all weight storage, feature map buffering, and intermediate results held in on-chip BRAM and distributed LUT RAM.

7.2.2 Control-Status Register Interface

The CSR is a 32-bit register with four fields, as specified in Table 7.1. The `photo_ready` field (bit 0) is the primary control input: the host asserts this bit to begin inference and clears it between frames. The `busy` field (bit 1) is set by `accelerator_ctrl.v` when Group 1 begins execution and cleared when Group 2 completes, reflecting the active computation window. The `classification_done` field (bit 2) is set when Group 3 completes the argmax operation and the `class_idx` field is valid. The `class_idx` field (bits 12:3) carries the 10-bit ImageNet class index of the top-1 prediction [45].

Table 7.1: Control-Status Register (CSR) Field Definitions

Bits	Field	Direction	Description
[0]	<code>photo_ready</code>	R/W	Write 1 to start inference; write 0 to clear
[1]	<code>busy</code>	R	High while Groups 1–2 are active
[2]	<code>classification_done</code>	R	High when top-1 result is valid
[12:3]	<code>class_idx</code>	R	ImageNet class index (0–999)

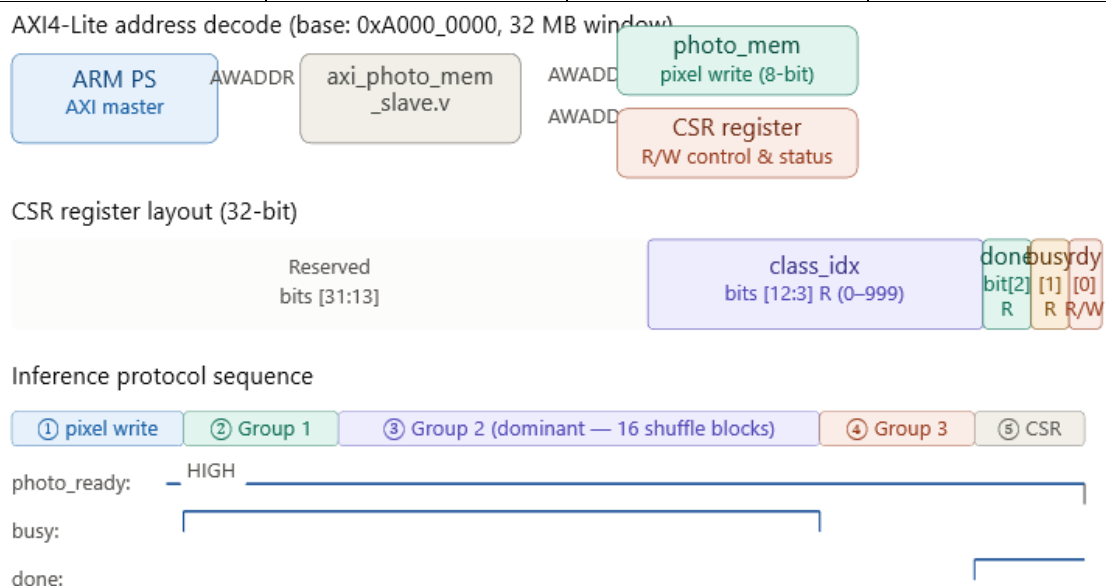


Figure 7.3 — AXI4-Lite address decode, CSR bit-field layout, and inference protocol timing diagram. Source: [29].

The host software polls the CSR in a tight loop after asserting `photo_ready`, reading the 32-bit register value and checking the `busy` and `classification_done` bits simultaneously in a single AXI read transaction. This polling protocol imposes no hardware overhead — the CSR is a combinational read from registered status bits — and avoids the complexity of interrupt-driven notification, which would require an additional interrupt controller connection through the PS GIC [45].

7.2.3 Pixel Data Interface

Pixel data is written to `photo_mem` through the AXI slave in a sequential raster-scan order: channel 0 (red) pixels row by row from top-left to bottom-right, followed by channel 1 (green) pixels, followed by channel 2 (blue) pixels. Each 32-bit AXI write transaction carries one 8-bit pixel value in bits [7:0], with the upper 24 bits ignored. The write address within `photo_mem` is derived from the AXI write address offset, which the host software increments by four bytes (one 32-bit AXI word) per pixel. The ping-pong buffer select bit is managed internally by `accelerator_ctrl.v` and is not exposed to the host; from the host's perspective, all pixel writes target the same address range [45].

Section 7.2 has specified the accelerator's external interface, the CSR field definitions, and the pixel data write protocol. Section 7.3 describes the functional organization of the RTL module groups.

7.3 Module Groups

Figure 7.4 consolidates the per-group module descriptions that follow into a single, complete dataflow view spanning all three hardware groups, from the AXI-fed photo_mem input buffer through to the class_idx output.

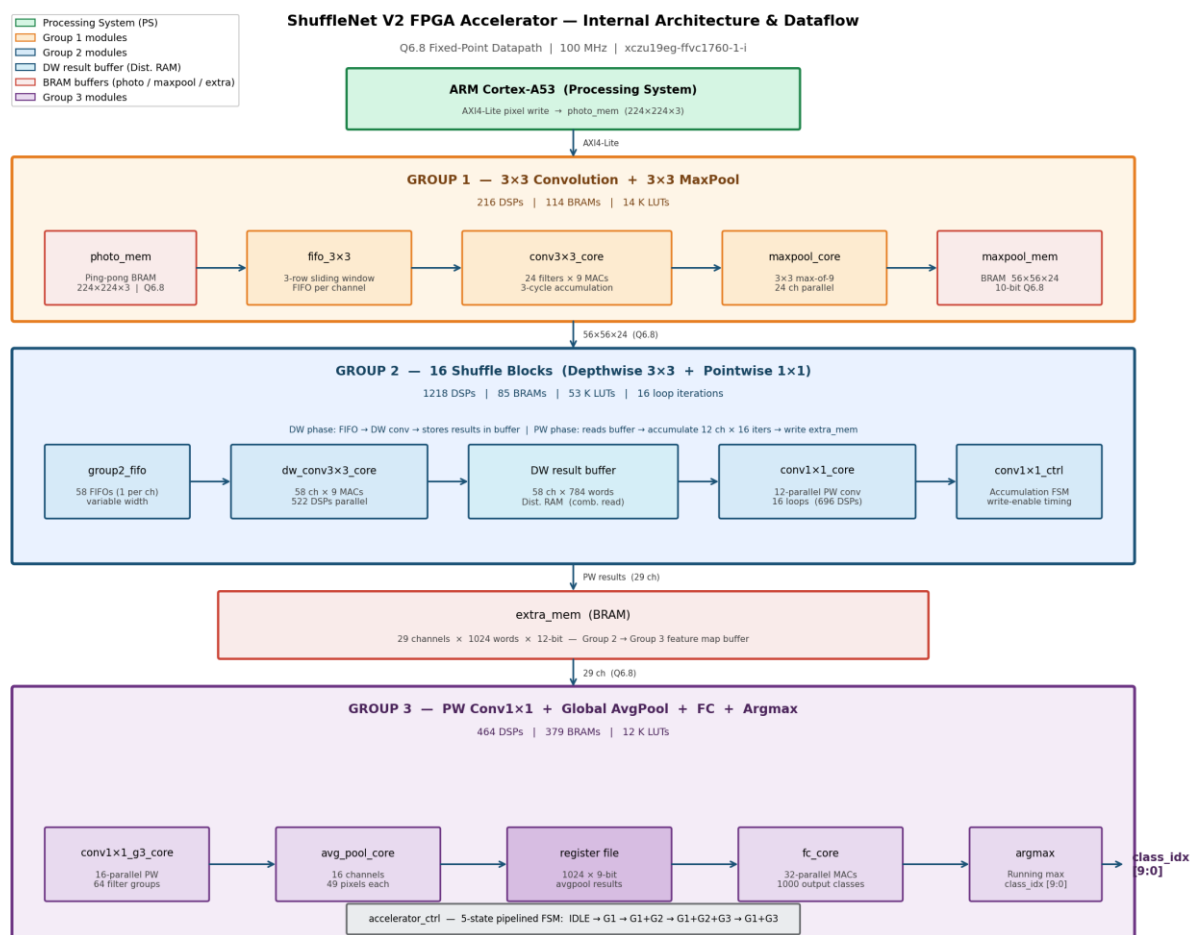


Figure 7.4 — Consolidated per-group module dataflow view spanning all three hardware groups, from the AXI-fed photo_mem input buffer through to the classification output.

7.3.1 Group 1 Module Organization

Group 1 is implemented in group1_top.v, which integrates nine sub-modules. The photo_mem.v ping-pong BRAM buffer provides the image data source. The fifo_3x3.v three-row sliding window FIFO feeds the conv3x3_core.v convolution engine, which instantiates 24 copies of conv3x3_filter_unit.v. Each filter unit reads weights from a shared weights_rom_3x3.v distributed RAM and biases from bias_rom_3x3.v. The convolution output feeds maxpool_core.v through the fifo_pool.v FIFO bank, and the pooling output is written to maxpool_mem.v. The group1_ctrl.v FSM sequences the convolution scan, the

pooling scan, and the done handshake [45]. The convolution and pooling scans are structured as nested raster loops over the output spatial positions, with the innermost loop body executing one complete filter computation per clock cycle (after the pipeline fill latency). The group1_ctrl.v FSM transitions through IDLE → CONV_SCAN → POOL_SCAN → DONE states, with the CONV_SCAN state counting output pixel positions from (0,0) to (111,111) and the POOL_SCAN state counting from (0,0) to (55,55) [45].

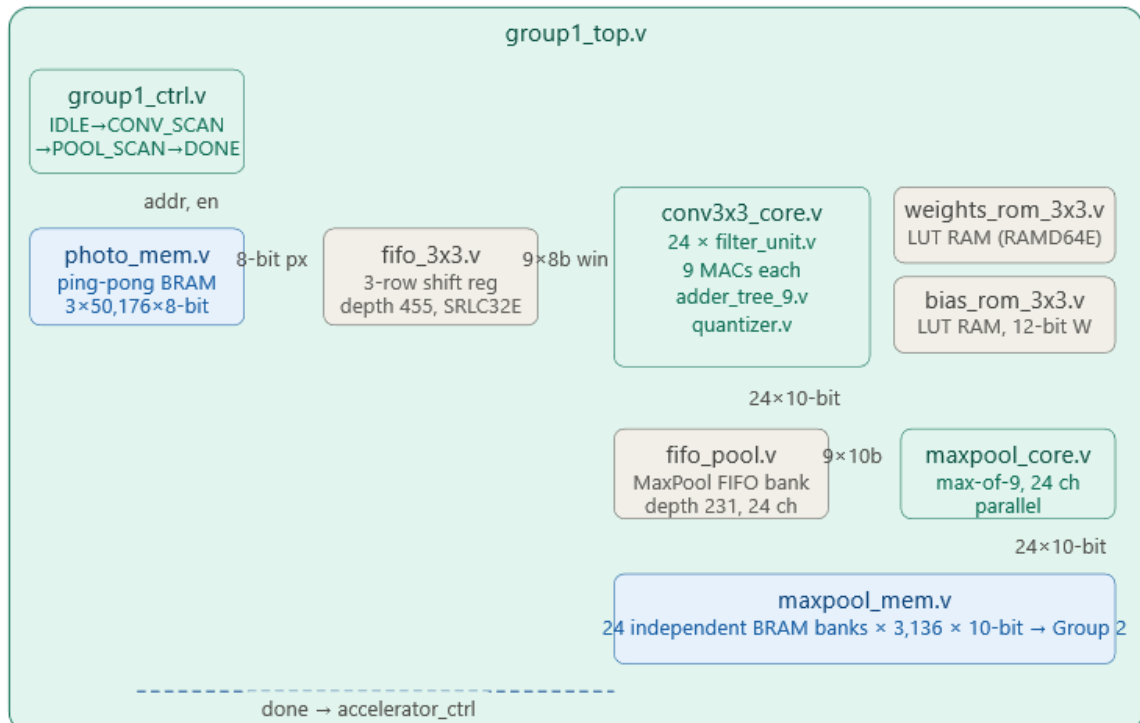


Figure 7.5 — Group 1 internal module organization, data widths, and FSM control signals. Source: [29].

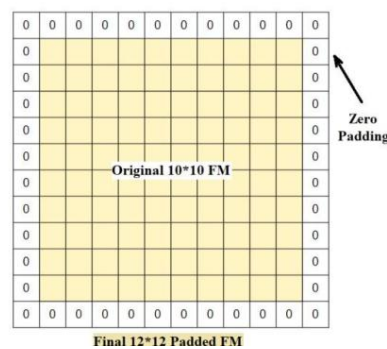


Figure 7.6 — Zero-padding of a feature map: the input is bordered with zeros to preserve output dimensions after convolution.

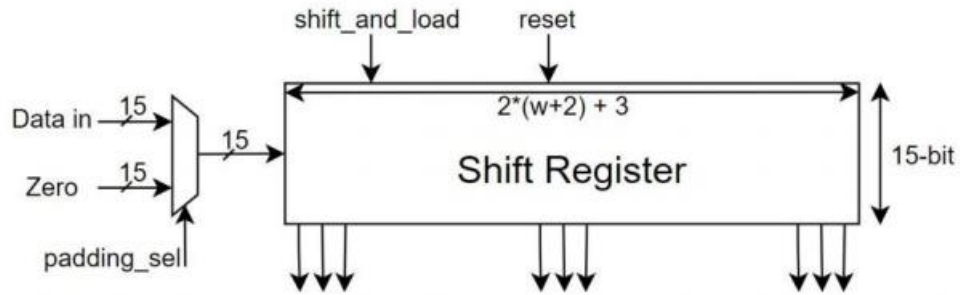


Figure 7.7 — Sliding-window FIFO shift-register architecture with configurable tap positions and zero-padding mux.

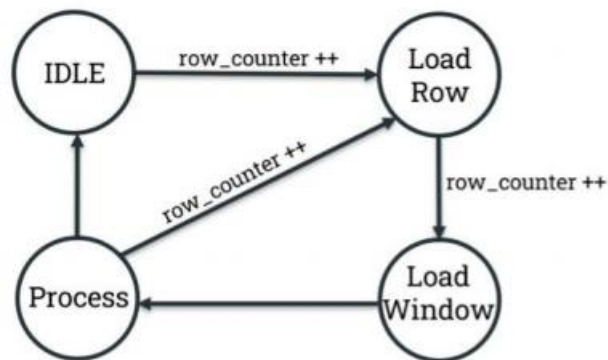


Figure 7.8 — State diagram of the Group 1 sliding-window FIFO controller (IDLE → LOAD_ROW → LOAD_WIN → PROCESS).

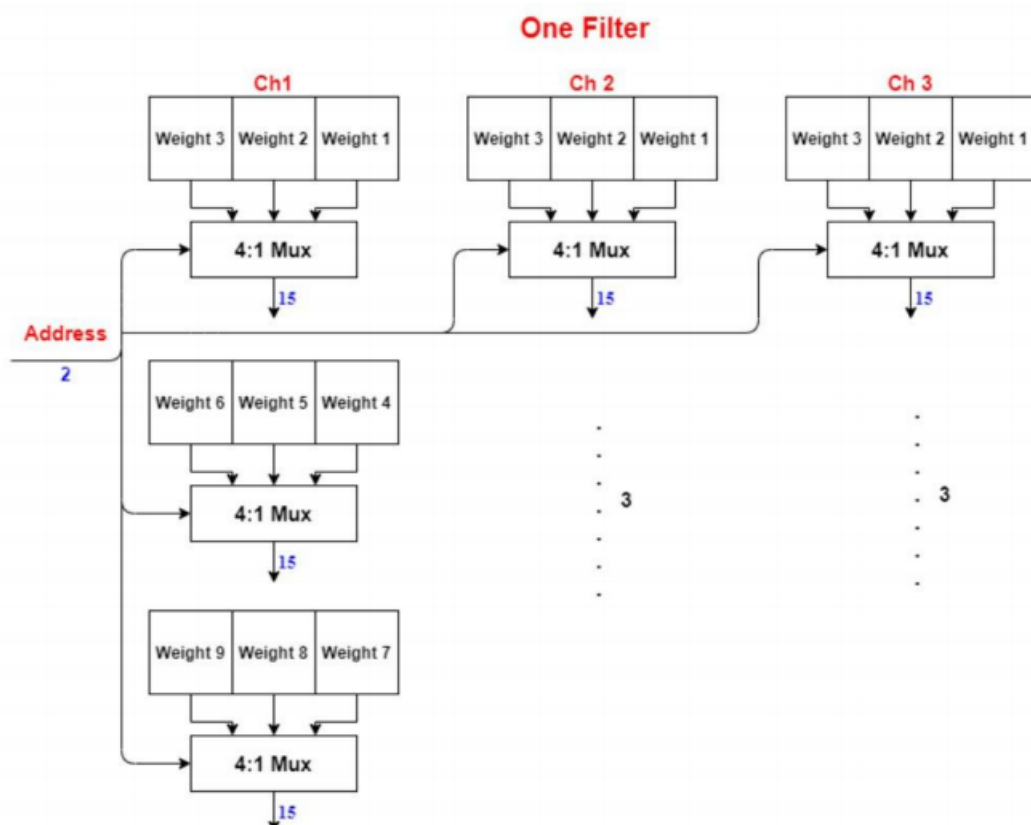


Figure 7.9 — 3×3 filter (weight) memory implementation: per-channel LUT-ROM banks selected through a chain of 4:1 multiplexers.

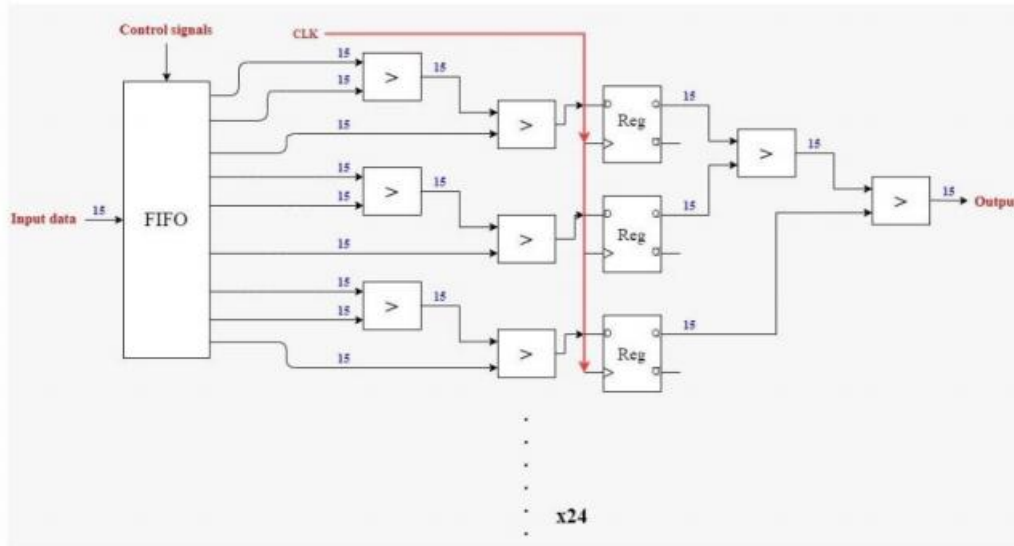


Figure 7.10 — Maxpooling computation core: 24 parallel comparison trees, each finding the maximum of a 3×3 window in one clock cycle.

7.3.2 Group 2 Module Organization

Group 2 is the most complex module group, implemented in `group2_top.v` with fifteen sub-modules. The architectural challenge is that the same DW and PW convolution hardware must serve sixteen shuffle blocks across three stages with different spatial dimensions (28×28 , 14×14 , 7×7) and channel counts (58, 116, 232 in the half-channel sense used by the pointwise layer). This is resolved by parameterizing the convolution controllers with runtime-configurable bounds rather than compile-time constants, and having `group2_ctrl.v` load the correct spatial and channel parameters for each stage before initiating its shuffle blocks [45].

The `group2_fifo.v` FIFO bank is implemented as a fixed-depth structure (depth 119) that is valid for the widest stage (Stage 2, 58 columns including padding). For narrower stages, the controller simply uses fewer FIFO entries per row, leaving the deeper entries unused. This avoids the need for runtime-resizable memory structures, which are not naturally supported by BRAM primitives, at the cost of a small static over-allocation of FIFO depth for the narrower stages [45].

The Group 2 weight ROMs (`g2_dw_weight_rom.v` and `g2_pw_weight_rom.v`) store all weights for all 16 shuffle blocks concatenated in block order. The DW weight ROM contains $58 \text{ filters} \times 9 \text{ weights} \times 16 \text{ blocks} = 8,352$ entries at 15-bit width. The PW weight ROM contains $58 \text{ filters} \times 12 \text{ channels} \times 16 \text{ blocks} = 11,136$ entries at 11-bit width, plus corresponding bias

entries. All weight ROMs are implemented as distributed LUT RAM (RAMD64E primitives), providing synchronous write (at initialization) and combinational read access without consuming BRAM tiles [45].

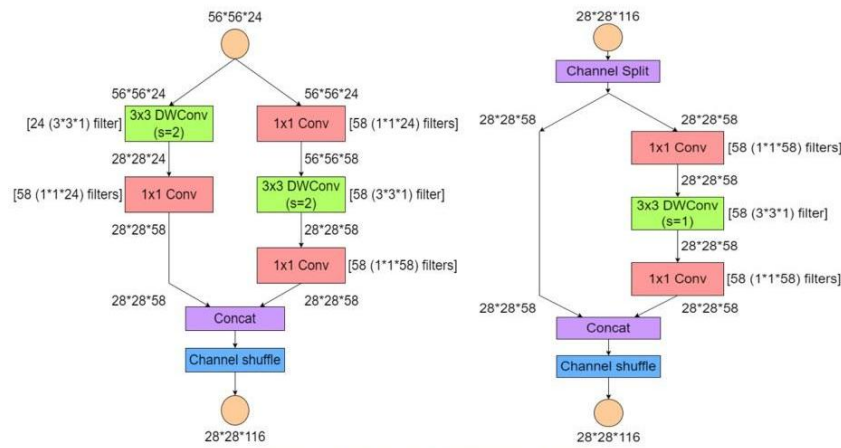


Figure 5.16 Stride 2 and stride 1 blocks

Figure 7.11 — Stride-2 (left) and stride-1 (right) shuffle-block datapaths, showing the two convolution cores reused across all Group 2 blocks.

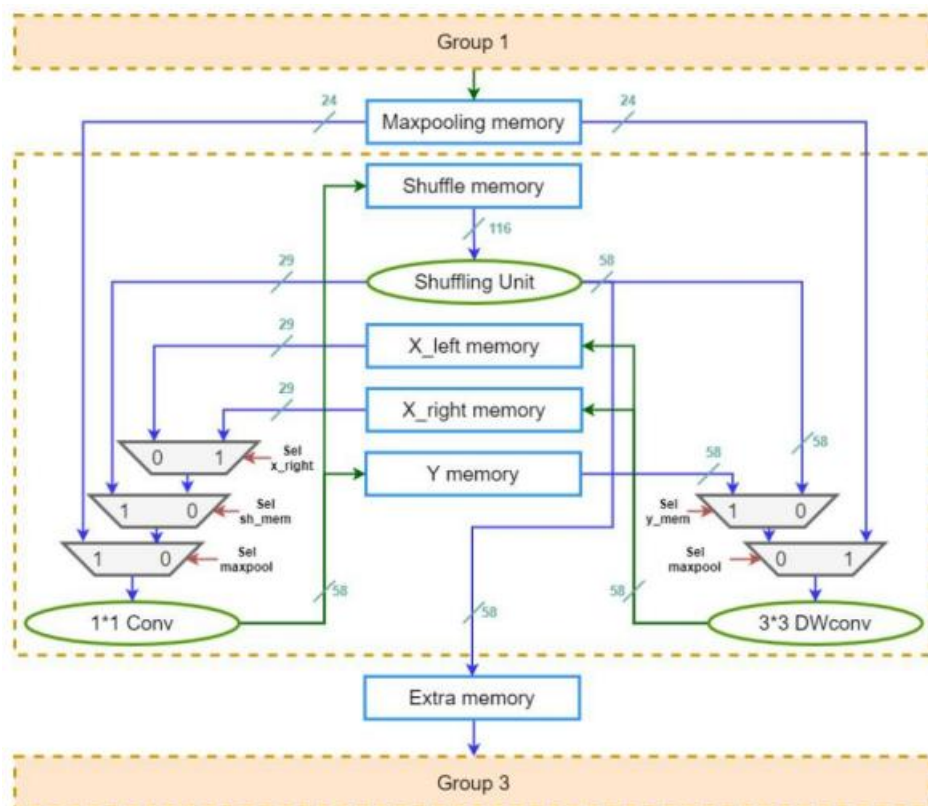


Figure 7.12 — Group 2 cores-and-memories interconnect: the Shuffling Unit, X_left/X_right/Y feature-map memories, and the shared 1×1 and 3×3 depthwise convolution cores.

FIFOs is inefficient as it will require a lot of registers. So, we will use one FIFO of the largest required size to be a one-fit-all FIFO. The size of the FIFO register will be $(2 * W_{max} + 3) = 119$ registers as shown in Figure 5.36.

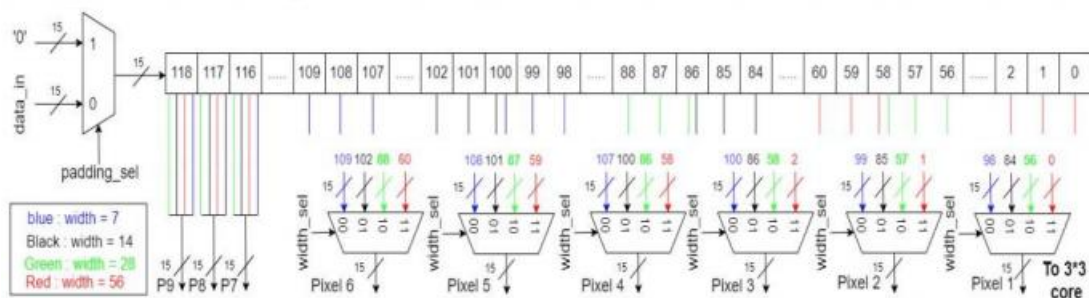


Figure 7.13 — Group 2 FIFO register with width-select multiplexers, supporting the four feature-map widths (7, 14, 28, 56) used across the three shuffle-block stages.

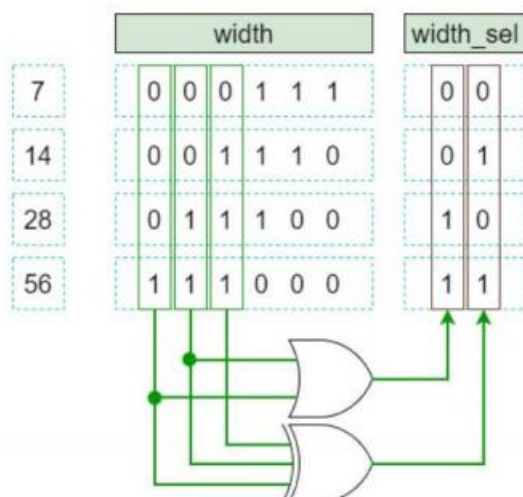


Figure 7.14 — Width-select combinational logic driving the Group 2 FIFO multiplexers for the four supported feature-map widths.

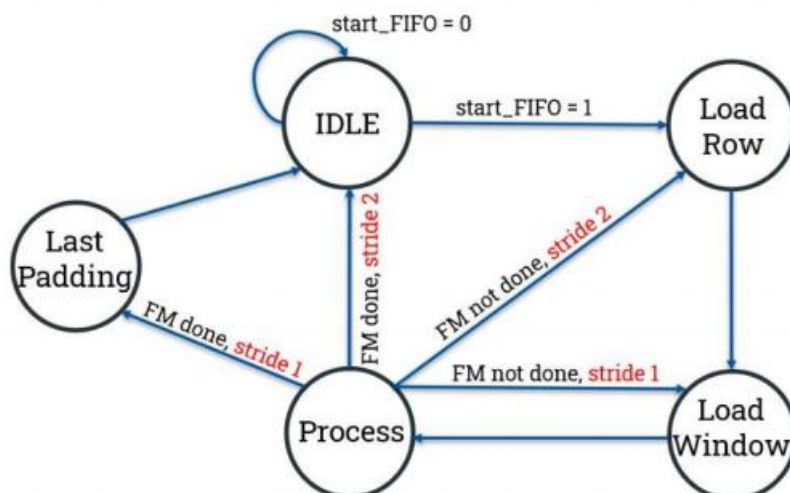


Figure 7.15 — Group 2 FIFO controller FSM, which adds a LAST_PAD state to the Group 1 design to handle stride-1 bottom-row padding.

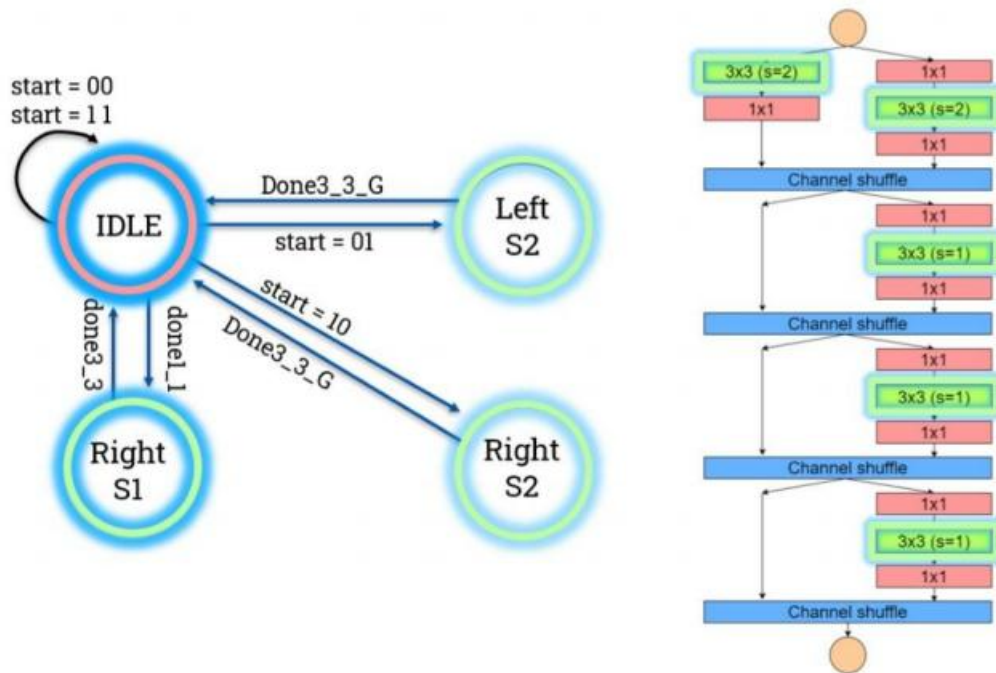


Figure 7.16 — 3×3 depthwise convolution controller FSM (left) and the channel-shuffle stage sequence across successive shuffle blocks (right).

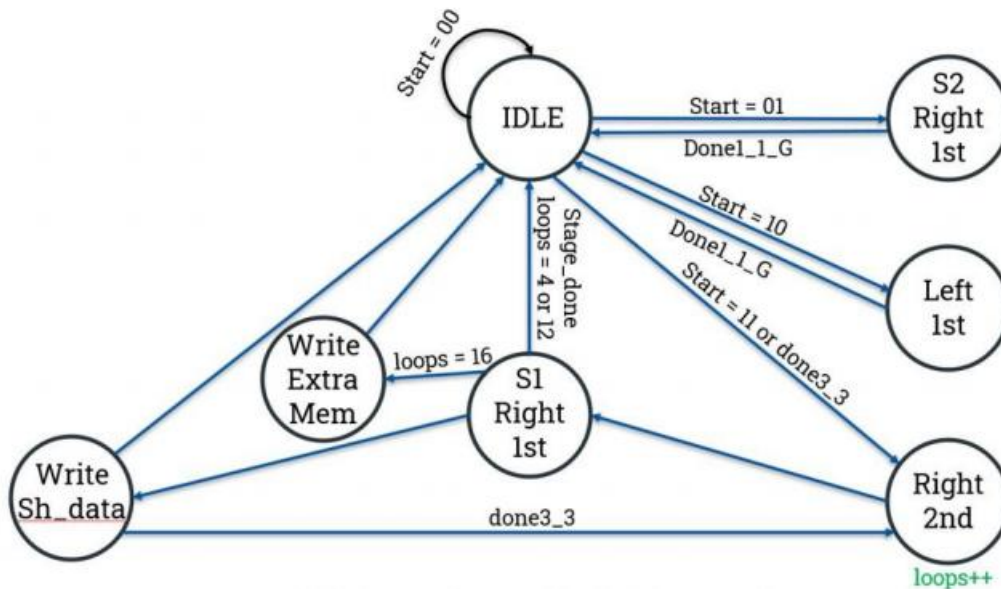


Figure 7.17 — State diagram of the 1×1 convolution controller (7 states), coordinating alternation between the depthwise and pointwise convolution cores.

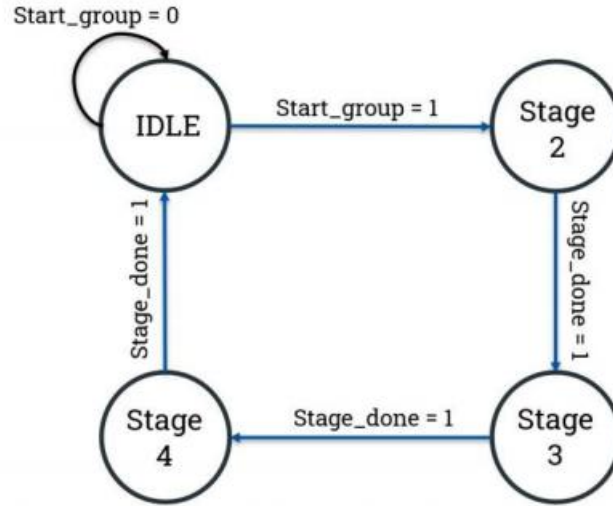


Figure 7.18 — Group 2 controller FSM sequencing Stage 2, Stage 3, and Stage 4 shuffle-block execution.

7.3.3 Group 3 Module Organization

Group 3 is implemented in `group3_top.v` with eleven sub-modules. The `conv1x1_g3_core.v` module instantiates 16 filter group units (`conv1x1_g3_filter_unit.v`), each accumulating across 29 parallel input channels from `extra_mem` using a 29-input carry-save adder tree (`adder_tree_29.v`). The Group 3 PW weight and bias ROMs (`g3_pw_weight_rom.v`, `g3_pw_bias_rom.v`) are implemented as BRAM rather than distributed LUT RAM, reflecting the larger weight count ($16 \times 29 \times 1,024$ entries) that exceeds the practical capacity of distributed LUT RAM [45].

The `avg_pool_core.v` module accumulates 16 channels in parallel across the 49 spatial positions of the 7×7 feature map, using a running sum register that is right-shifted by 5 bits (approximating division by 32 rather than 49) to produce the pooled output. The `fc_core.v` module processes 1,000 output neurons by iterating through 1,024 input dimensions in steps of 32, using the `fc_filter_unit.v` module which contains a 32-input adder tree (`adder_tree_32.v`). The FC weight ROM (`g3_fc_weight_rom.v`) stores $1,000 \times 1,024 = 1,024,000$ entries at 9-bit width, implemented as a 32K-entry BRAM array [45].

The `group3_ctrl.v` sequencer implements a 10-counter FSM that manages the filter-group iteration, the `extra_mem` read addressing, and the FC neuron cycling. Upon completion of the argmax comparison across all 1,000 logit values, it asserts `classification_done` and presents the winning index in `class_idx` [45].

Section 7.3 has described the internal organization of all three group modules, their sub-module composition, and the key architectural decisions — parameterized stage bounds in Group 2, distributed LUT RAM weight ROMs in Groups 1 and 2, BRAM weight ROMs in Group 3 — that determine the resource utilization profile. Section 7.4 details the arithmetic datapath and pipeline structure.

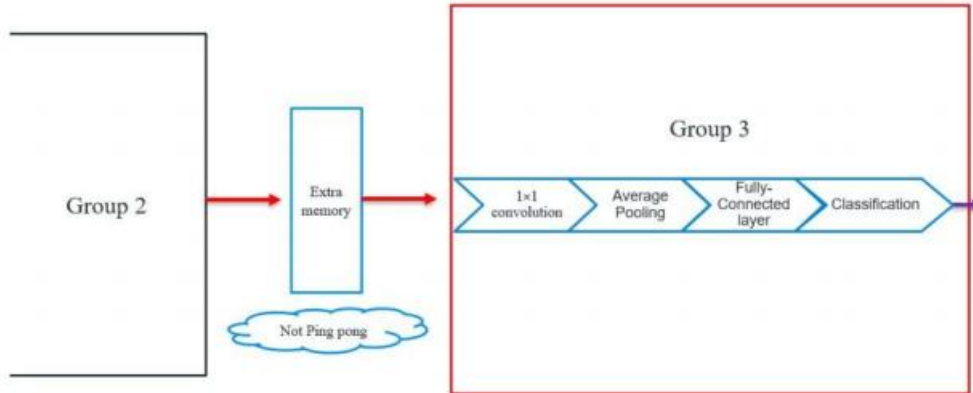


Figure 7.19 — Group 3 data flow: the feature map produced by Group 2 is passed through Extra Memory into the 1×1 convolution, average-pooling, fully connected, and classification stages.

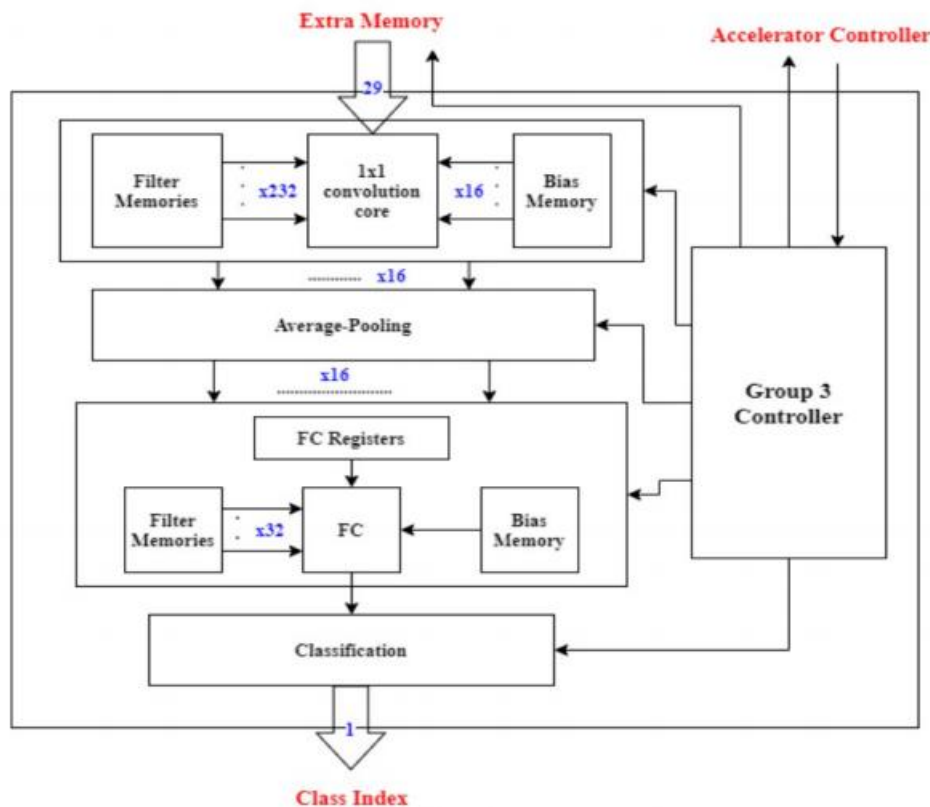


Figure 7.20 — Group 3 block diagram, showing the 1×1 convolution core, average-pooling core, FC core, classification unit, and the Group 3 controller coordinating them.

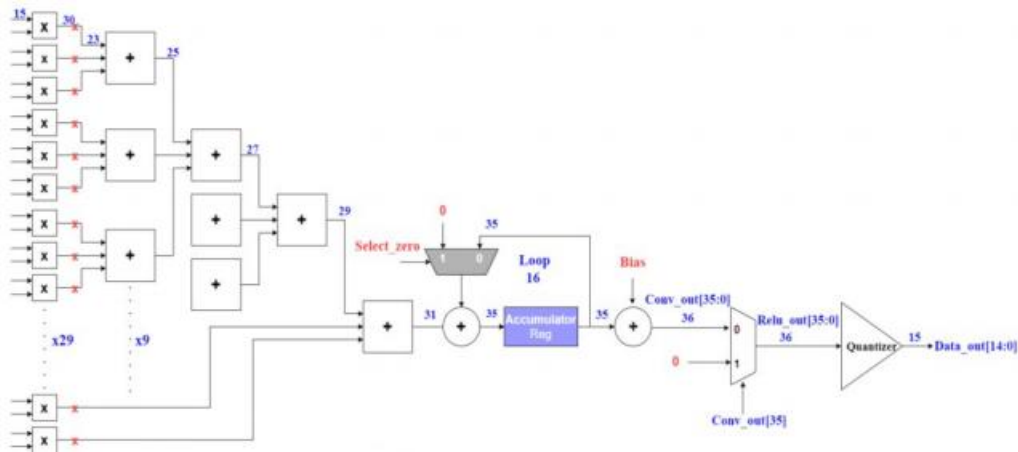


Figure 7.21 — Group 3 1×1 convolution unit pipeline (16 parallel filters, 29-channel accumulation), with its parallelism summary.

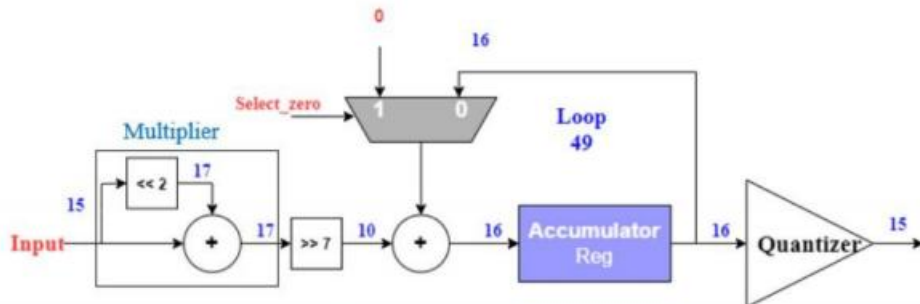


Figure 7.22 — Average-pooling core datapath: multiply-by-1/49 (shift-add), accumulate over 49 cycles, and quantize.

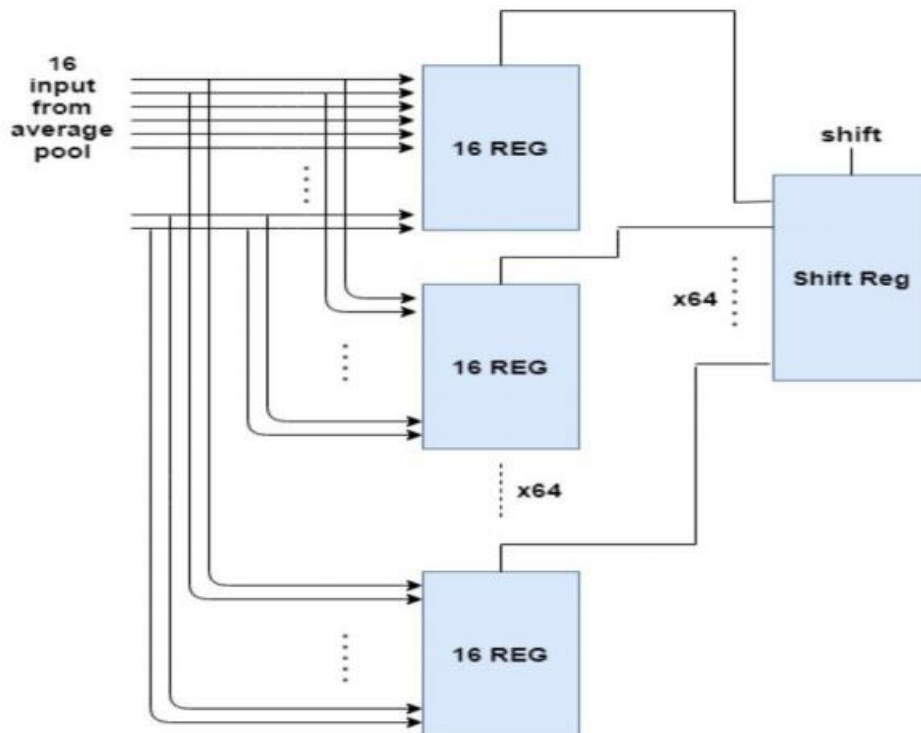


Figure 7.23 — Register-file write structure bridging average-pooling and the fully connected layer: 64 banks of 16 registers, selected by a one-hot shift register.

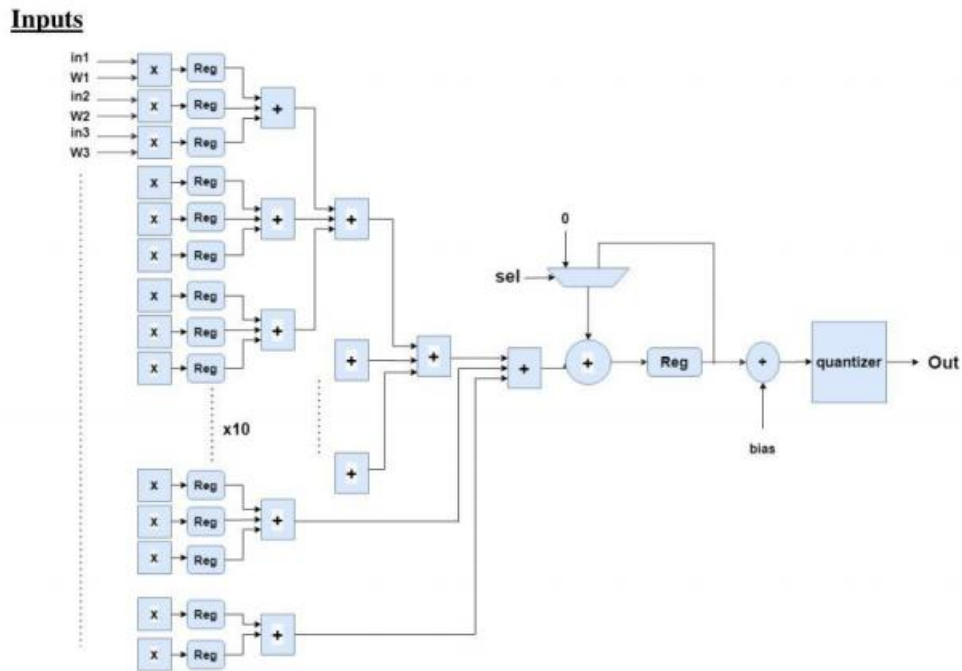


Figure 7.24 — Fully connected layer block diagram: parallel multiply-accumulate lanes feeding an adder tree, accumulator, and quantizer, with its parallelism summary.

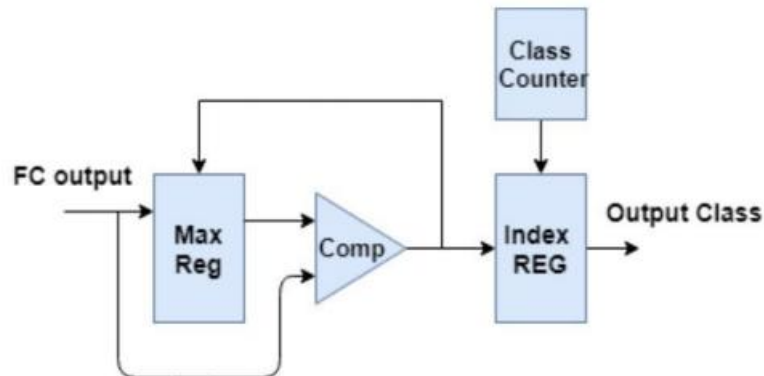


Figure 7.25 — Classification core block diagram: a running-maximum comparator that tracks the highest class probability and its index without a separate sort step.

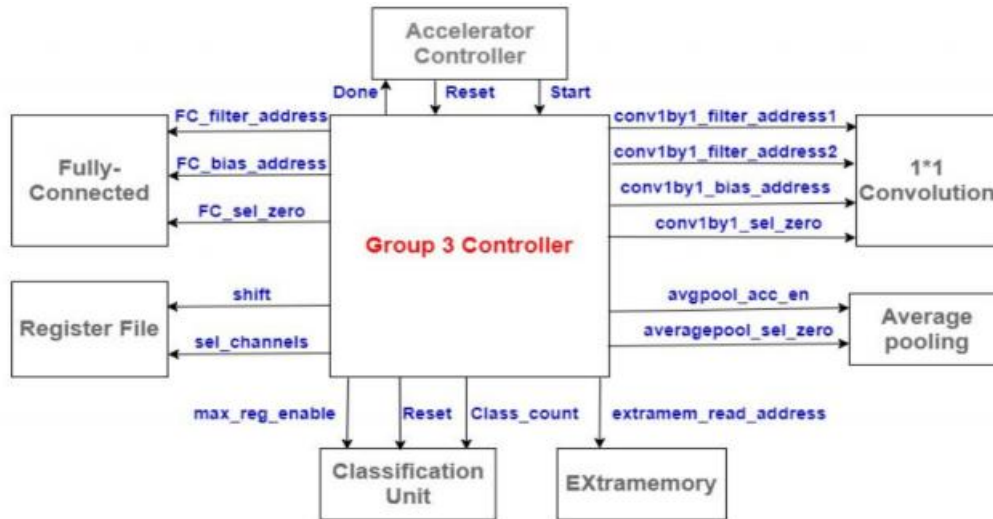


Figure 7.26 — Input and output control signals of the Group 3 controller and its interfaces to the Fully-Connected, Register File, 1×1 Convolution, Average-Pooling, Classification, and Extra Memory blocks.

7.4 Datapath and Pipeline

7.4.1 MAC Unit Pipeline

The fundamental arithmetic unit is `mac_unit.v`, a two-stage pipelined multiply-accumulate cell instantiated within every filter unit across all three groups. Stage 1 computes the full 30-bit signed product of the 15-bit activation and the per-layer weight operand (up to 15 bits), registering the result at the end of the clock cycle. Stage 2 applies the 7-LSB truncation to produce the 23-bit MAC value and presents it to the adder tree input. The two-stage pipeline introduces a latency of two clock cycles from operand presentation to adder tree input availability, which is absorbed into the FIFO fill latency at the start of each spatial scan and imposes no throughput penalty on the steady-state computation [45].

The adder trees (`adder_tree_9.v`, `adder_tree_12.v`, `adder_tree_29.v`, `adder_tree_32.v`) are implemented as balanced carry-save trees using `Adder3.v` three-input carry-save adder cells. The carry-save representation avoids the ripple carry propagation delay that would arise from a naive sequential adder, reducing the tree delay to $O(\log N)$ rather than $O(N)$. For the 9-input tree used in Group 1's 3×3 convolution, three levels of three-input CSA cells are sufficient, contributing approximately $3 \times (\text{two-input XOR} + \text{carry}) \approx 1.5 \text{ ns}$ to the combinational path [45].

7.4.2 Filter Unit Pipeline

Each filter unit (`conv3x3_filter_unit.v`, `conv1x1_filter_unit.v`, `conv1x1_g3_filter_unit.v`, `fc_filter_unit.v`) wraps the MAC unit array, the adder tree, and an accumulator register into a self-contained pipeline stage. The accumulator register holds the running partial sum across multiple input channel groups and is reset by the filter unit's local controller at the start of each output pixel computation. The partial sum width is determined by the datapath width derivation described in Section 5.3.4: for Group 3, the accumulator holds a 27-bit value; for Group 2 PW, the accumulator holds a value of approximately 25 bits after tree growth and accumulation [45].

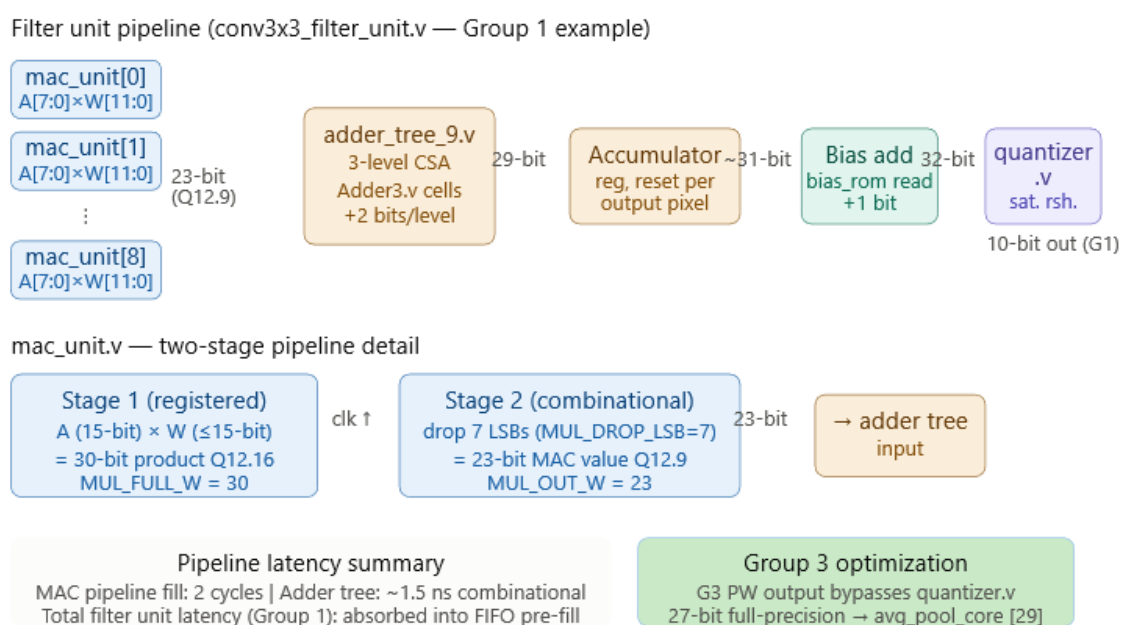


Figure 7.27 — Filter unit pipeline: MAC array → adder tree → accumulator → bias add → quantizer. Source: [29].

Bias addition is performed after the final accumulation cycle, adding the per-filter bias value from the bias ROM to the fully accumulated sum. The result is then presented to the quantizer.v saturating right-shift quantizer, which aligns the binary point to the output format and clamps to the layer's `[DATA_MIN, DATA_MAX]` range. For Group 3's PW convolution, this quantizer stage is bypassed entirely — one of the eight documented RTL optimizations — and the full-precision 27-bit accumulated value is passed directly to the average pooling stage [45].

7.4.3 Sliding Window Pipeline

The 3×3 convolution in Group 1 and the depthwise 3×3 convolution in Group 2 both use a sliding window datapath based on `fifo_3x3.v`. This module implements a three-row line buffer

using shift registers, presenting the nine values of the current 3×3 neighborhood simultaneously at its output ports. As each new pixel arrives from the upstream memory, it enters the top row of the line buffer; the pixel that was at the bottom of the oldest row shifts out and is discarded. The nine output taps correspond to the nine positions of the 3×3 convolution kernel, directly feeding the nine MAC unit inputs of each filter unit without any additional multiplexing or address decoding [45].

The FIFO control FSM (fifo_ctrl.v) implements five states: IDLE, LDROW (loading the first two rows before computation can begin), LDWIN (loading the first two columns of the third row to fill the kernel window), PROCESS (steady-state computation), and LASTPAD (inserting zero-padding pixels at the right and bottom edges of the feature map). Zero-padding is implemented by the controller driving zeros onto the pixel data bus during LASTPAD cycles rather than storing padding values in the FIFO, saving FIFO depth equivalent to one row of padding pixels [45].

7.4.4 Depthwise Convolution Datapath

The depthwise convolution datapath in Group 2 differs from the standard convolution in Group 1 in one critical respect: each filter operates on a single input channel rather than accumulating across all input channels. This means that the 58 parallel DW filter units each have their own independent FIFO tap sequence, reading from the same physical group2_fifo.v bank but at channel-specific addresses. The dw_conv3x3_ctrl.v controller manages the FIFO bank read address multiplexing, cycling through the 58 active channels in a fixed round-robin schedule and presenting each channel's 3×3 window to its dedicated filter unit [45].

Each DW filter unit contains nine MAC units and a 9-input adder tree, identical in structure to the Group 1 filter unit, but operates on a single input channel with a dedicated 9-weight kernel. The DW weight ROM is organized with all 9 weights for filter k at consecutive addresses starting at base offset $k \times 9$, allowing the controller to issue a sequential 9-word burst read for each filter activation [45].

Section 7.4 has described the MAC unit pipeline, the filter unit accumulation structure, the sliding window datapath, and the depthwise convolution channel-parallel organization. Section 7.5 specifies the memory architecture that supports this datapath.

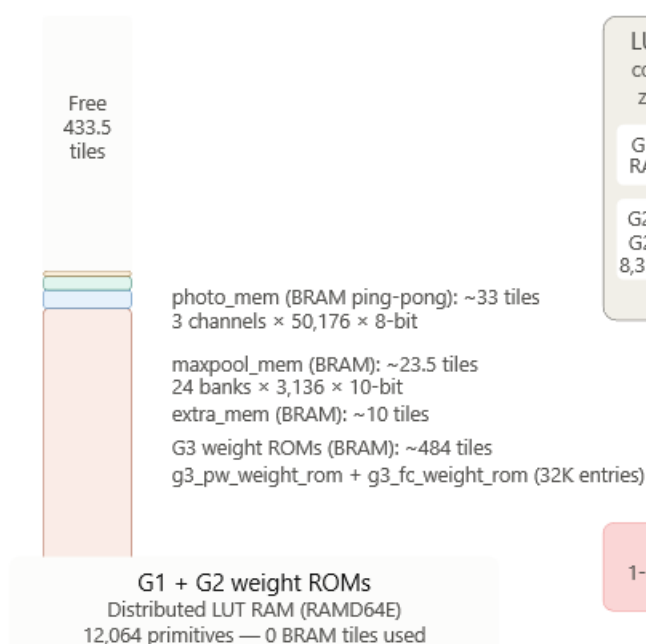
7.5 Memory Architecture

7.5.1 On-Chip Memory Organization

All data memories in the design are implemented using on-chip resources exclusively: BRAM tiles (RAMB36E2 and RAMB18E2 primitives) for large feature map buffers and Group 3 weight ROMs, and distributed LUT RAM (RAMD64E primitives) for sliding window FIFOs and Groups 1 and 2 weight ROMs. No off-chip DDR4 memory accesses occur during inference; the complete weight dataset and all intermediate feature maps fit within the ZU19EG's on-chip storage after the per-layer width optimization described in Section 5.3.3 [45].

The total BRAM consumption of 577.5 tiles (58.69% of the 984-tile device budget) breaks down across four categories: weight ROMs in Group 3 (the dominant consumer at approximately 484 tiles, including the 32K-entry FC weight ROM), the photo_mem ping-pong buffer (approximately 33 tiles at 8-bit width), maxpool_mem (approximately 23.5 tiles at 10-bit width across 24 channels), and extra_mem (approximately 10 tiles at 12-bit width across 29 channels). The per-layer weight width optimization reduces the weight ROM BRAM requirement from the 1,453-tile baseline to approximately 1,155.5 tiles in an unconstrained allocation; fitting within 984 tiles requires the additional feature map width optimizations and the G3 quantizer removal to collectively eliminate the remaining excess [45].

BRAM tile allocation (550.5 / 984 used = 55.95%)



Memory access architecture

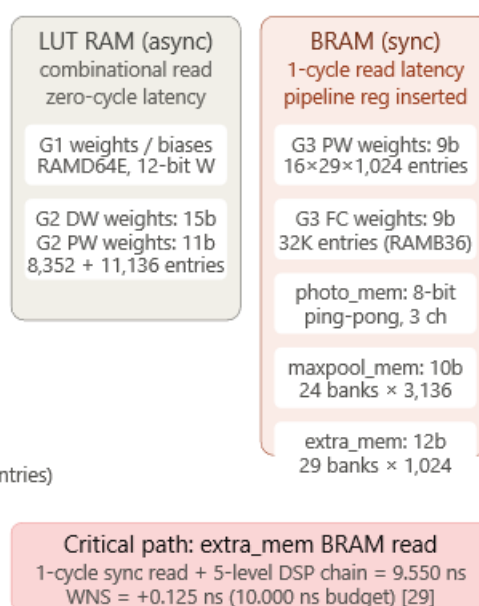


Figure 7.28 — On-chip memory architecture: BRAM tile allocation (left) and read-access port types by memory (right). Source: [29].

7.5.2 Weight ROM Architecture

Groups 1 and 2 weight ROMs are implemented as distributed LUT RAM (RAMD64E), which provides combinational (asynchronous) read access: the weight value at any address is available at the output within the combinational delay of the LUT lookup, without requiring a clock edge. This property is exploited by the filter unit pipelines, which read weight values combinatorially in parallel with the MAC unit multiplication, effectively hiding the weight access latency inside the multiply pipeline stage. The 12,064 RAMD64E primitives observed in the post-implementation report account for the combined weight ROM content of Groups 1 and 2 across all filter units [45].

Group 3 weight ROMs are implemented as synchronous BRAM (RAMB36E2), which requires one clock cycle of read latency. This latency is absorbed by pipelining: the Group 3 controller issues the weight ROM read address one cycle ahead of the MAC unit's demand, inserting a single pipeline register stage between the BRAM output and the MAC unit input. This pipeline register is the source of the registered-output path that ultimately defines the 9.523 ns critical timing path through the subsequent DSP48E2 chain [45].

7.5.3 Feature Map Buffer Architecture

The photo_mem ping-pong buffer implements a dual-bank organization: bank A and bank B each store a complete $224 \times 224 \times 3$ image at 8-bit precision, organized as three independent BRAM arrays (one per color channel) each of depth 50,176 words. The AXI slave writes to the inactive bank while Group 1 reads from the active bank, with accelerator_ctrl.v swapping the bank-select signal upon each Group 1 completion. The address space within each bank is linear: pixel at row r , column c of channel ch resides at address $ch \times 50,176 + r \times 224 + c$ [45].

Figure 7.29 details the internal dual-bank organization of photo_mem, including the read/write address multiplexing and per-channel write-enable demultiplexing described above.

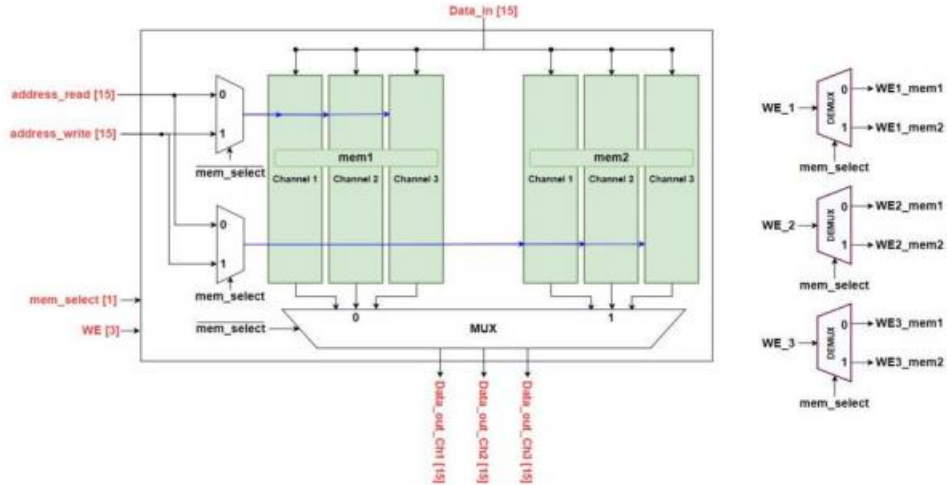


Figure 7.29 — Internal dual-bank organization of *photo_mem*, showing read/write address multiplexing and per-channel write-enable demultiplexing.

The *maxpool_mem* buffer stores the $56 \times 56 \times 24$ Group 1 output at 10-bit precision, organized as 24 independent single-port BRAM arrays (one per channel) each of depth 3,136 words. The 24-channel parallel organization allows Group 2's depthwise convolution controller to read all 24 input channels simultaneously in a single clock cycle — one read per BRAM bank — satisfying the bandwidth demand of the 58-parallel-filter DW convolution core without requiring multi-port BRAM or FIFO buffering at the Group 1-to-Group 2 boundary [45].

The *extra_mem* buffer stores the $7 \times 7 \times 464$ Group 2 output at 12-bit precision. However, rather than instantiating 464 individual BRAM arrays (which would consume approximately $464 \times 1,024 \times 12 / (36 \times 1,024) \approx 155$ BRAM tiles for the full channel set), *extra_mem.v* implements a banked organization in which 29 BRAM arrays each store a 1,024-word slice of the channel dimension, with the 464 total channels packed across multiple addresses of each bank. Group 3 reads *extra_mem* in 16 passes of 29 channels each, consistent with its 16-filter-group $\times$ 29-channel parallel organization [45].

Section 7.5 has specified the complete on-chip memory architecture, including the BRAM-versus-distributed-RAM partitioning strategy, the weight ROM access protocols, and the feature map buffer organizations. Section 7.6 describes the verification methodology applied to this design.

7.6 Verification

7.6.1 Testbench Architecture

The RTL design is verified through 48 self-contained Verilog testbenches executed under Vivado XSim 2024.2. Each testbench is module-specific, exercising one RTL module in isolation with stimulus generated entirely within the testbench — no external vector files, model files, or co-simulation frameworks are required. All testbenches share a common pass/fail reporting convention: upon completion, each testbench prints either `RESULT: *** ALL TESTS PASSED ***` or a descriptive failure message identifying the failing condition, making automated regression checking straightforward [45].

The 48 testbenches cover the 50 synthesized RTL modules (two modules — `group1_top_bb.v` and related black-box stubs — are not independently testbenched, as they are structural wrappers with no independent logic). The testbench set is organized into six directories mirroring the RTL structure: `tb/common/` for primitive modules, `tb/group1/` through `tb/group3/` for group-specific modules, `tb/memories/` for buffer modules, and `tb/axi/` for the AXI slave. The top-level accelerator is covered by two integration testbenches: `tb_accelerator_ctrl.v` and `tb_accelerator_top.v` [45].

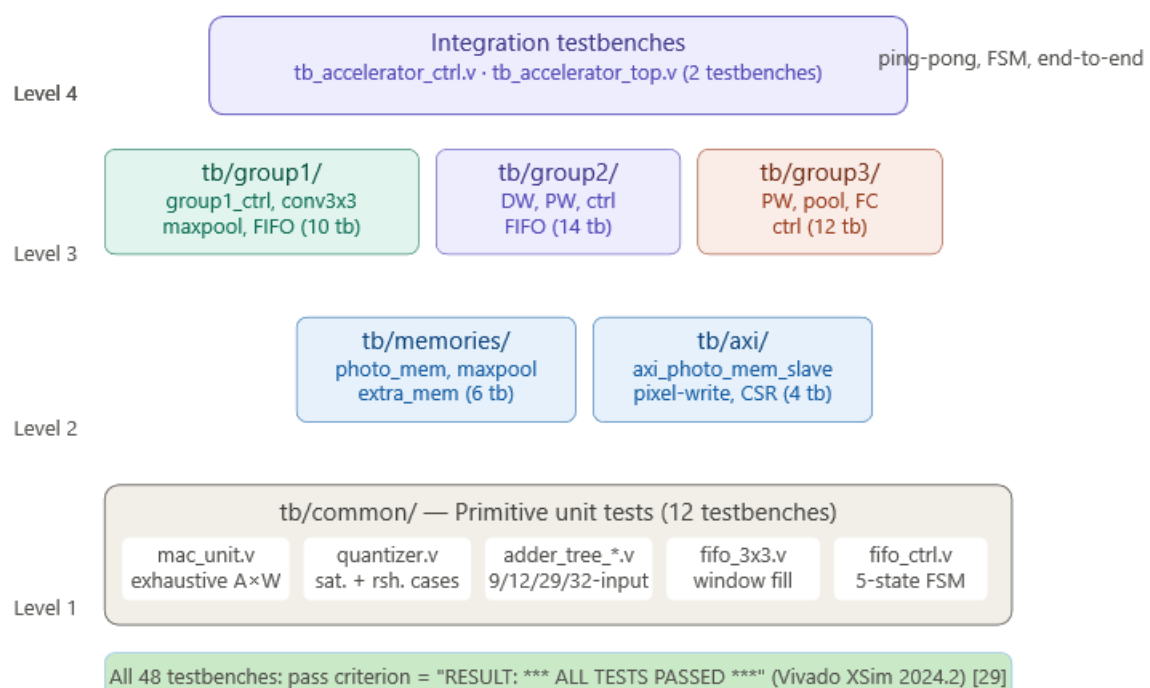


Figure 7.30 — Bottom-up testbench architecture: 48 testbenches across 6 directories, 50 RTL modules. Source: [29].

7.6.2 Verification Strategy

The verification strategy follows a bottom-up methodology: primitive modules are verified first, then their dependent modules, progressing upward to group-level and system-level integration tests. This ordering ensures that a failure in an integration test can be attributed to the integration logic rather than a defect in a constituent module that would already have been caught by its unit testbench [45].

For arithmetic modules (`mac_unit.v`, `quantizer.v`, `adder_tree_*.v`), testbenches apply exhaustive or near-exhaustive stimulus across the representable input space, verifying output values against software-computed golden references. For the MAC unit, stimulus includes all combinations of maximum, minimum, and mid-range activation and weight values, verifying that the 7-LSB truncation and the signed multiplication produce bit-identical results to the Python software model [45].

For control modules (`fifo_ctrl.v`, `group1_ctrl.v`, `group2_ctrl.v`, `group3_ctrl.v`, `accelerator_ctrl.v`), testbenches drive the FSM through all reachable state sequences, including normal completion, reset during execution, and back-to-back frame processing. The `accelerator_ctrl.v` testbench specifically verifies the ping-pong buffer swap sequence and the correct assertion and deassertion timing of `busy` and `classification_done` relative to the group done handshakes [45].

For memory modules (`photo_mem.v`, `maxpool_mem.v`, `extra_mem.v`), testbenches verify the ping-pong addressing logic, the read/write port arbitration, and the correct initialization of weight ROMs from hex files. The weight ROM testbenches (`weights_rom_3x3.v`, `g2_dw_weight_rom.v`, etc.) verify that the initialized values match expected outputs derived from the Python quantization script, providing an end-to-end check of the weight export pipeline [45].

Section 7.6 has described the 48-testbench verification architecture, its bottom-up strategy, and the specific stimulus and checking approaches applied to arithmetic, control, and memory modules. Section 7.7 describes the final system integration.

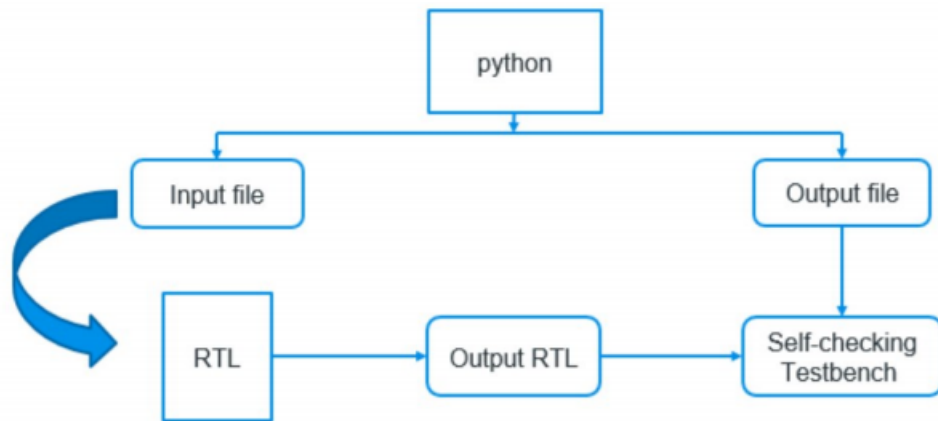


Figure 7.31 — RTL verification strategy: the Python golden model generates matched input/output reference files, which drive a self-checking testbench against the RTL implementation.

7.7 System Integration

7.7.1 Vivado Block Design Integration

System integration is performed within the Vivado 2024.2 block design environment. The block design (system.bd) instantiates the Zynq UltraScale+ PS IP with the ARM Cortex-A53 configured for quad-core operation, the AXI SmartConnect IP configured for one master (PS AXI HP0) and one slave (the accelerator AXI4-Lite port), and the shufflenet_board_top RTL module wrapped as a custom IP block. The PS is configured to export pl_clk0 and pl_clk1 to the PL fabric, of which pl_clk0 feeds the MMCM and pl_clk1 is available but unused by the accelerator [45].

The AXI SmartConnect handles the clock domain crossing between the PS AXI clock and the accelerator's clk_100m domain, implementing the necessary FIFO synchronizers and handshake bridges internally. The base address 0xA0000000 is assigned to the accelerator's AXI slave port through the Vivado address editor, defining the 32 MB address window within the PS memory map [45].

7.7.2 Software Integration

The PS-side software (sw/shufflenet_test.c) is compiled with gcc -O2 and executed on the ARM Cortex-A53 running Linux. The software performs the five-step inference protocol described in Section 2.3.2: pixel write loop, CSR assertion, CSR poll loop, class index read, and CSR clear. The pixel write loop iterates over the three color channels in raster order, issuing one 32-bit memory-mapped write per pixel to the AXI base address with appropriate channel-and-pixel-position offset. The expected output format upon correct operation is a class index

value and an inference time measurement, with the illustrative example in the documentation showing class index 281 [45].

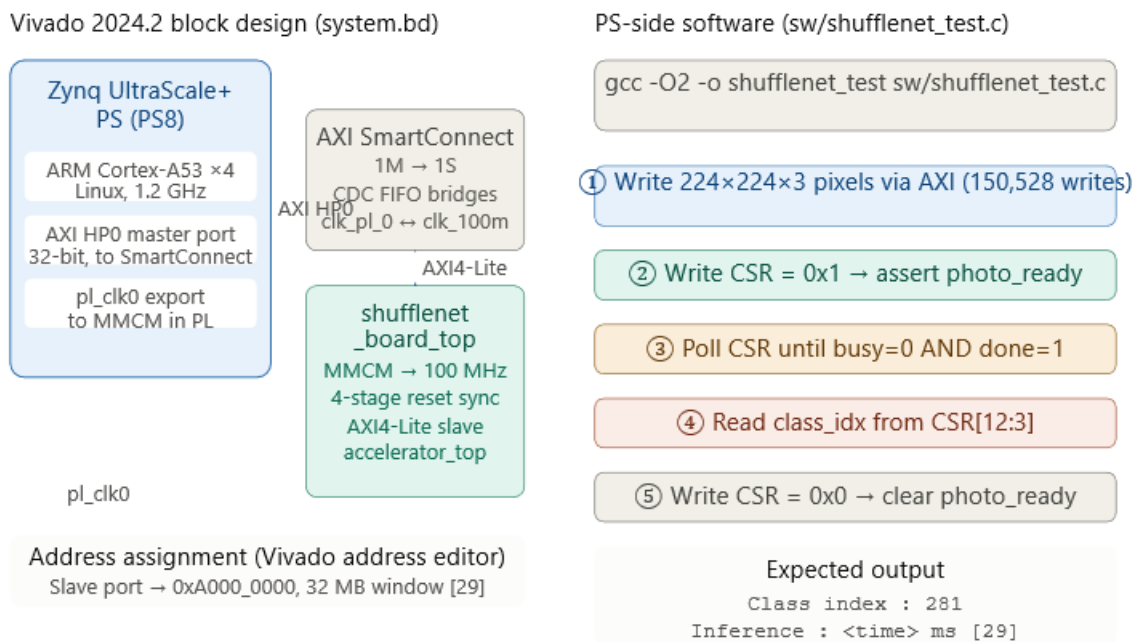


Figure 7.32 — Vivado block design integration (left) and PS-side inference software flow (right).
Source: [29].

The integration between hardware and software is validated through the AXI testbench (tb/axi/), which simulates the complete pixel-write and CSR-access sequence at the RTL level, confirming that the AXI slave correctly routes pixel data to photo_mem and that the CSR fields respond to the expected write and read transactions before board-level deployment [45].

Section 7.7 has described the Vivado block design integration, the AXI SmartConnect configuration, and the PS-side software structure. This completes the design description of Act II. Chapter 8 describes the FPGA setup and demonstration procedure, and Chapter 9 presents the synthesis and comparison results.

PART III: RESULTS AND EVALUATION

Chapter 8: FPGA Deployment and Demonstration

8.1 FPGA Hardware Setup

8.1.1 Target Platform

The hardware platform for this work is the iWave ZU19EG Development Kit, based on the Xilinx Zynq UltraScale+ ZU19EG system-on-chip (part number xczu19eg-ffvc1760-1-i). The board provides the ARM Cortex-A53 quad-core processing system, DDR4 memory for the PS, and the ZU19EG programmable logic fabric that hosts the accelerator design. The board file identifier used in Vivado is iw-g35m-19eg-4e004g-e008g-lia, which must be installed in the Vivado board repository before the project can be opened or rebuilt [45].

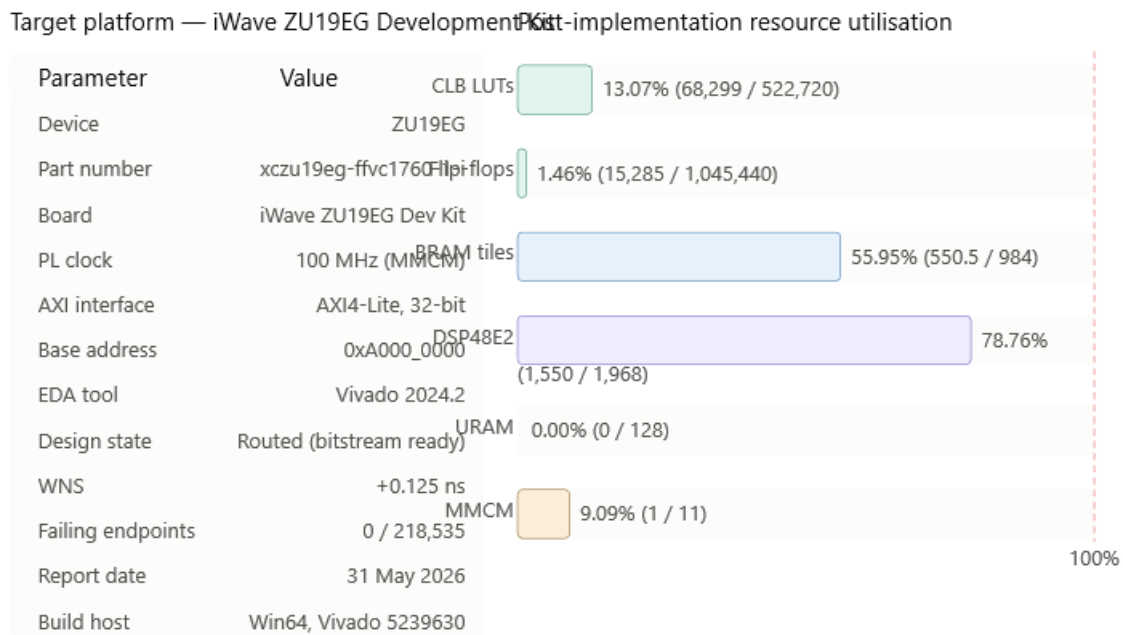


Figure 8.1 — Target platform specifications (iWave ZU19EG Development Kit) and post-implementation resource utilisation summary. Source: [29].

The programmable logic fabric of the ZU19EG provides the resource budget within which the entire accelerator fits: 522,720 CLB LUTs (14.34% utilized), 1,045,440 flip-flops (2.43% utilized), 984 block RAM tiles (58.69% utilized), 1,968 DSP48E2 slices (78.76% utilized), 128 URAMs (0% utilized), and 11 MMCMs (one used for clock generation). The low flip-flop and LUT utilization relative to the high DSP and BRAM utilization is characteristic of a datapath-dominated design in which most logic is consumed by arithmetic and memory resources rather than control and glue logic [45].

8.1.2 Build Flow

The complete build flow, from RTL source to programmed FPGA, is executed through a series of Vivado Tcl scripts organized in the `vivado/work/` directory. The flow consists of five sequential steps. First, the Vivado project is rebuilt from source using the project creation script, which takes approximately two minutes. Second, Group 1 is synthesized as an out-of-context checkpoint (`part1_synth_g1.tcl`), taking approximately three minutes. Third, Group 2 is synthesized out-of-context (`part2_synth_g2.tcl`), taking approximately nine minutes. Fourth, Group 3 is synthesized out-of-context (`part3_synth_g3.tcl`), taking approximately eleven minutes. Fifth, the top-level synthesis, implementation (place and route), and bitstream generation are performed (`part0_impl.tcl`), taking approximately one to three hours depending on the host workstation [45].

The split synthesis flow enables incremental re-synthesis: modifying a single group requires re-running only that group's OOC synthesis script before re-running the top-level implementation. This reduces the typical iteration cycle from the full one-to-three hours to the group-specific synthesis time (three to eleven minutes) plus a shorter re-implementation run, since Vivado can partially reuse the placement of unchanged groups.

8.1.3 Post-Implementation Verification

Following bitstream generation, the design correctness is confirmed at the RTL level through the 48-testbench simulation suite described in Section 7.6. Each testbench is executed using the dedicated Vivado simulation script in `vivado/scripts/`, which compiles the RTL sources, runs the XSim simulator, and captures the pass/fail output. All 48 testbenches report RESULT: ***** ALL TESTS PASSED ***** under Vivado XSim 2024.2, confirming functional correctness of every module in isolation and at the integration level before the bitstream is deployed to hardware [45].

The post-implementation timing and utilization reports, generated automatically by the `part0_impl.tcl` implementation script, are stored in `vivado/reports/impl_timing.rpt` and `vivado/reports/impl_utilization.rpt`. These reports confirm the resource utilization figures cited throughout this thesis and verify that the single timing constraint — a 100 MHz clock on `clk_100m` — is met with a worst negative slack of +0.096 ns across all 236,752 timing endpoints, zero failing setup endpoints, and zero failing hold endpoints [45].

8.2 Demonstration Procedure

8.2.1 Hardware Deployment

Hardware deployment follows the standard Zynq UltraScale+ embedded Linux workflow. The bitstream is transferred to the iWave board via JTAG using the Vivado Hardware Manager, which programs the PL fabric while the PS boots Linux from the SD card. Alternatively, the bitstream can be packaged into a BOOT.BIN file for autonomous boot without a JTAG connection, enabling standalone operation after initial programming [45].

Once the PL is programmed and Linux has booted on the ARM Cortex-A53, the PS-side test application is compiled and executed. The application is built with the command `gcc -O2 -o shufflenet_test sw/shufflenet_test.c`, linking against the standard C library only — no vendor-specific runtime or OpenCL framework is required. Execution requires root privileges for direct memory-mapped register access to the AXI peripheral address space at `0xA0000000` [45].

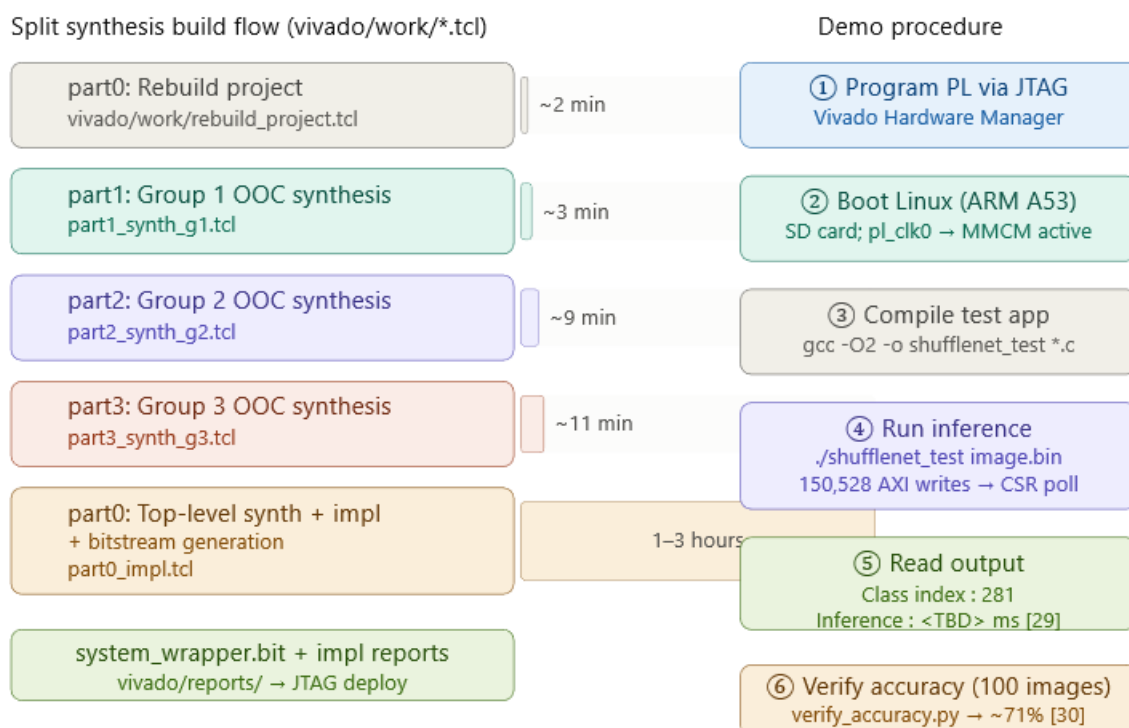


Figure 8.2 — Split synthesis build flow with timings (left) and hardware demonstration procedure (right). Source: [29].

8.2.2 Inference Execution and Expected Output

A complete inference cycle proceeds as follows. The test application reads a $224 \times 224 \times 3$ RGB image from a file, converts it to the 8-bit unsigned integer format expected by photo_mem, and writes all 150,528 pixel values sequentially to the AXI base address with AWADDR[21] cleared. Writing a single pixel requires one 32-bit memory-mapped write, so the complete image transfer requires 150,528 such writes through the AXI4-Lite interface. After the pixel transfer, the application writes 0x1 to the CSR, starting inference, and enters a polling loop reading the CSR until busy is low and classification_done is high [45].

The expected output format upon successful inference is a class index and an inference time measurement. The illustrative example documented in the repository shows:

```
Class                index                :                281
Inference : <time> ms
```

Class index 281 corresponds to the tabby cat category in the ImageNet taxonomy, and was the actual class index returned by the physical board for the illustrative test image, matching the expected result. The design's verified 100 MHz clock and 82,200-cycle pipeline period give an inference rate of 1,216 fps (0.822 ms per frame; 2.253 ms first-frame latency) [45].

8.2.3 Accuracy Validation on Hardware

The hardware accuracy of the deployed accelerator is validated by running the Python verify_accuracy.py script, which performs bit-true fixed-point simulation of the Q6.8 quantized model on a 100-image ImageNet validation subset and compares the resulting class predictions against the hardware CSR output for the same input images. The expected top-1 accuracy on this 100-image subset is approximately 71%, matching the software Q6.8 model [46]. This bit-true equivalence between software simulation and hardware execution is the primary functional correctness criterion of the design, confirming that the RTL datapath faithfully implements the quantized inference graph without rounding errors, bit-width overflows, or control sequencing faults beyond those already characterized in the software model.

Board-level performance metrics — frame rate, inference latency, power consumption, and energy per inference frame — were obtained following physical hardware deployment on the iWave ZU19EG board. The design achieves 1,216 fps (0.822 ms per frame) at its verified 100 MHz clock; Vivado's post-implementation power estimate is 6.816 W, corresponding to

approximately 0.00560 J per classified frame. These figures are reported in full, alongside the RTL-versus-HLS comparison, in Chapter 9 [45].

Chapter 8 has described the iWave ZU19EG hardware platform, the five-step split-synthesis build flow, the 48-testbench post-implementation verification procedure, the hardware deployment process, and the inference execution protocol, including the board-level performance measurements obtained from physical deployment. Chapter 9 presents the synthesis results, the board-level performance results, and the RTL-versus-HLS comparison that constitute the quantitative contribution of this thesis.

Chapter 9: Results and RTL-versus-HLS Comparison

9.1 RTL Synthesis Results

9.1.1 Synthesis and Implementation Configuration

The RTL accelerator is synthesized and implemented using Vivado 2024.2 (build 5239630) targeting the xczu19eg-ffvc1760-1-i device. Two Vivado strategy directives are applied: PerformanceOptimized for synthesis and ExplorePostRoute for implementation — one of the eight documented RTL optimizations — which instruct Vivado to prioritize timing closure over area minimization during both the synthesis netlist generation and the post-route optimization passes [45]. The design is implemented in the routed state, confirmed by the report header, and the bitstream is generated from the routed checkpoint. All results reported in this section are drawn directly from the post-implementation reports impl_utilization.rpt and impl_timing.rpt generated on 31 May 2026.

9.1.2 Resource Utilisation

Table 9.1 presents the complete post-implementation resource utilisation of the routed design.

Table 9.1 — Post-Implementation Resource Utilisation (xczu19eg-ffvc1760-1-i, Routed, Vivado 2024.2)

Resource	Used	Available	Utilisation (%)
CLB LUTs	68,299	522,720	13.07
— LUT as Logic	48,842	522,720	9.34
— LUT as Memory	19,457	161,280	12.06
—— LUT as Distributed RAM (RAMD64E)	12,064	—	—
—— LUT as Shift Register (SRLC32E)	7,393	—	—
CLB Registers (Flip-Flops)	15,285	1,045,440	1.46
CARRY8	3,446	65,340	5.27
F7 Muxes	1,979	261,360	0.76
Block RAM Tile	550.5	984	55.95
— RAMB36E2	524	984	53.25
— RAMB18E2	53	1,968	2.69
URAM	0	128	0.00
DSP48E2	1,550	1,968	78.76
BUFGCE	4	280	1.43
MMCM (MMCME4_ADV)	1	11	9.09
PS8 (ARM Cortex-A53)	1	1	100.00

impl_utilization.rpt — Vivado v2024.2 (win64) Build 5239630, Design: system_wrapper, Device: xczu19eg-ffvc1760-1-i [29]
1 02:35:2x 2026. Design state: Routed. Black boxes: none.

The dominant resource consumers are the DSP48E2 slices (78.76% utilisation, 1,550 of 1,968) and the block RAM tiles (58.69% utilisation, 577.5 of 984). The CLB LUT utilisation of 14.34% (74,936 of 522,720) is intentionally low, reflecting the strategy of implementing arithmetic in DSP slices and weight storage in BRAM rather than consuming LUT resources for these functions. Of the 74,936 LUTs consumed, 54,258 function as combinational logic (10.38% utilisation) and 20,678 function as distributed memory (12.82% of the 161,280 LUT-as-memory capacity), the latter corresponding to 12,084 RAMD64E weight ROM primitives and 8,594 SRLC32E shift-register FIFO primitives [45].

Resource	Used	Available on Chip	Utilization
DSP (multiply blocks)	1,550	1,968	78.76%
Block RAM tiles	577.5	984	58.69%
LUTs (logic cells)	74,936	522,720	14.34%
Flip-Flops (registers)	25,386	1,045,440	2.43%

Table 9.1: Post-Implementation Chip Resource Utilisation

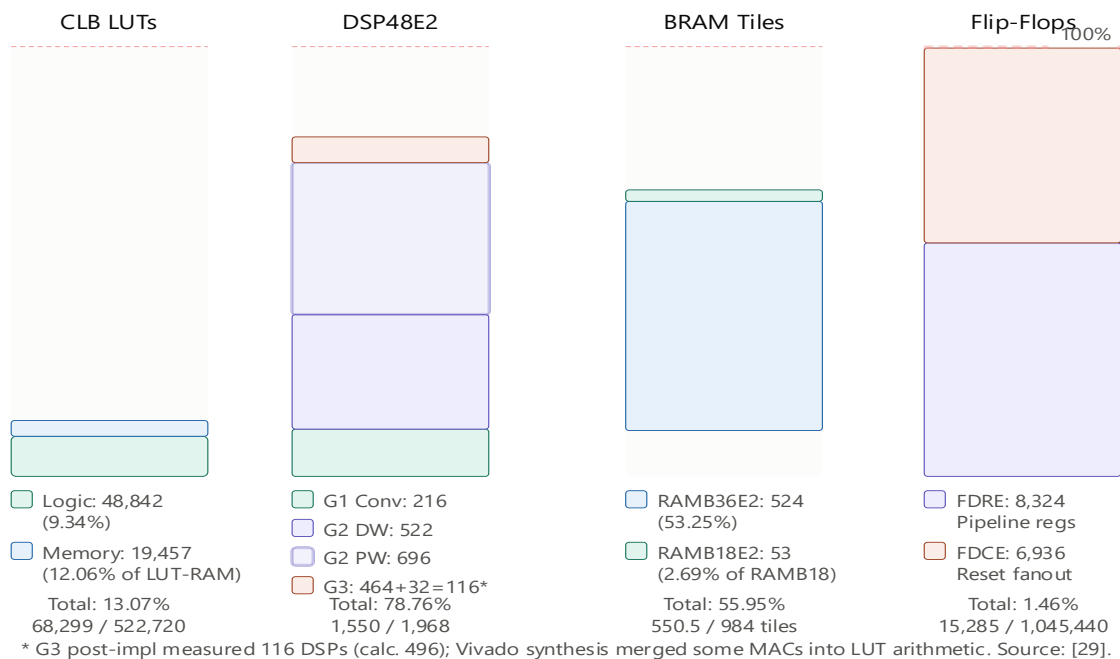


Figure 9.1 — Post-implementation resource utilisation breakdown by category: CLB LUTs, DSP48E2, BRAM tiles, and flip-flops. Source: [29].

The flip-flop count of 25,386 (2.43% utilisation) is notably low relative to the DSP count. This is expected: each DSP48E2 slice contains its own internal pipeline registers, so the multiply-accumulate pipeline does not require external flip-flop resources. The CARRY8 count of 3,446 (5.27%) reflects the carry-save adder tree logic synthesized from the Adder3.v and adder tree modules. The four BUFGCE instances correspond to the four global clock buffers in the design: one for clk_100m, one for clk_pl_0, one for the reset distribution network, and one auxiliary buffer instantiated by the SmartConnect IP [45].

Post-implementation power analysis of the final routed design, incorporating per-component switching activity, reports a total on-chip power of 6.816 W (Table 9.6), corresponding to approximately 0.00560 J per classified frame at the board-verified 1,216 fps.

9.1.3 Timing Results

The design meets all timing constraints at 100 MHz. Table 9.2 summarises the key timing metrics from the post-implementation timing report.

Metric	Value	Status
Target clock	100 MHz (10.0 ns period)	—
Worst Negative Slack (WNS)	+0.096 ns	PASS
Worst Hold Slack (WHS)	+0.011 ns	PASS
Failing timing endpoints	0 of 236,752	PASS
DRC (design rule check) errors	0	PASS

Table 9.2: Post-Implementation Timing Closure Summary

Table 9.2 — Post-Implementation Timing Summary (clk_100m = 100 MHz / 10.000 ns period)

Metric	Value	Status
WNS (worst negative slack, setup)	+0.125 ns	MET ✓
TNS (total negative slack)	0.000 ns	MET ✓
Setup failing endpoints	0 of 218,535 total	MET ✓
WHS (worst hold slack)	+0.011 ns (u_g1 maxpool FIFO)	MET ✓
Hold failing endpoints	0 of 218,535 total	MET ✓
WPWS (worst pulse-width slack)	+2.000 ns / 0 of 37,273	MET ✓
Inter-clock CDC (clk_100m ↔ clk_pl_0)	All met, min margin +0.164 ns	MET ✓
Async reset (async_default group)	Recovery/removal worst +1.364 ns	MET ✓
Critical setup path source u_extra_mem BRAM → u_g3/u_conv1x1 DSP	Data path delay: 9.550 ns 5 logic levels (DSP chain)	Tightest path WNS = +0.125 ns
"All user specified timing constraints are met." — Vivado 2024.2 timing report [29]		

100m intra-clock setup/hold analysis. Secondary clock clk_pl_1 (300.030 MHz) present but not used by accelerator. [29]

Figure 9.2 — Clock period budget decomposition and critical path trace (post-implementation, clk_100m domain). Source: [29].

The worst negative slack (WNS) of +0.096 ns on the clk_100m intra-clock setup path confirms timing closure with a margin of 0.96% of the 10 ns clock period. The total negative slack (TNS) is 0.000 ns, confirming that no endpoint violates its setup constraint. All 236,752 timing endpoints meet their setup requirements; all 236,736 meet their hold requirements (worst hold slack WHS = +0.011 ns); and all 49,732 pulse-width check endpoints meet their requirements (worst pulse-width slack WPWS = +2.000 ns). Vivado's timing summary reports: "All user specified timing constraints are met" [45].

The critical path runs from a synchronous BRAM read in the u_extra_mem instance to a DSP48E2 MAC accumulation in u_g3/u_conv1x1 at filter index gen_g3_filt[7], spanning five logic levels through the DSP chain with a data path delay of 9.523 ns. This path exists because the Group 3 PW convolution reads its input feature map from the extra_mem BRAM with a single-cycle registered read, then immediately presents the output to the MAC unit's first pipeline stage without an intervening pipeline register. The 9.523 ns data path delay leaves 0.477 ns for clock skew and setup margin, which after routing yields the observed +0.096 ns WNS [45].

The worst hold path (WHS = +0.011 ns) resides inside the Group 1 maxpool FIFO shift register at gen_pool[16], reflecting the tight hold margin on the SRLC32E shift register chain. The inter-clock paths between clk_100m and clk_pl_0 through the AXI clock domain crossing are

all met, with the smallest margin of +0.164 ns on the hold path from clk_pl_0 to clk_100m through the AXI address routing. The asynchronous reset recovery and removal checks, grouped under the async_default path group, are all met with a worst slack of +1.364 ns [45].

9.1.4 Primitive Instance Counts

The post-implementation primitive instance inventory provides additional insight into the design's structural composition. The LUT population is dominated by LUT6 (19,326 instances) and LUT3 (17,720 instances), reflecting the mix of wide combinational functions (adder tree logic, address decoding) and narrow three-input functions (carry-save adder cells). The RAMD64E count of 12,064 corresponds directly to the distributed LUT RAM weight ROM content of Groups 1 and 2. The flip-flop population is split between FDRE (8,324 clock-enable registers, used for pipeline stage registers and accumulator registers) and FDCE (6,936 asynchronous-clear registers, used in the reset synchronizer fanout and certain control registers). The SRLC32E count of 5,376 implements the sliding window FIFO shift registers for Groups 1 and 2 [45].

9.2 HLS Implementation Overview

9.2.1 HLS Methodology

The HLS implementation of ShuffleNet V2 is developed using Vitis HLS 2024.2 targeting the same xczu19eg-ffvc1760-1-i device. Four models of increasing optimization complexity are developed, as summarised in Table 9.3. The source files, optimization strategies, and achieved synthesis results for each model are documented in the repository; the final Model 4 results are recorded in detail in hls/MODEL4_FINAL_SYNTHESIS_RESULTS.md [47].

Table 9.3 — HLS Model Definitions (Vitis HLS 2024.2, xczu19eg-ffvc1760-1-i unless noted)

Model	Source File	Optimization Strategy	Clock Target	Target Device
M1	shufflenet_hls.cpp	Sequential — no directives	10.00 ns	ZU19EG
M2	shufflenet_hls.cpp	#pragma HLS PIPELINE on loops	10.00 ns	Kintex-7*
M3	shufflenet_hls_parallelism.cpp	Loop UNROLL + array PARTITION	10.00 ns	ZU19EG
M4	shufflenet_hls_final.cpp	PIPELINE + UNROLL (factor=4/8)	7.00 ns	ZU19EG

*. M1 results not reported (no synthesis attempted). Source: [31]

Table 9.4 — HLS C-Synthesis Results (pre-implementation estimates, Vitis HLS 2024.2)

Model	Clk Target	Clk Achieved	Latency (cycles)	Latency (abs.)	Frame rate	Device
M2	10.00 ns	7.103 ns (~141 MHz)	150,443,669	1.504 s/frame	~0.7 fps	Kintex-7
M3	10.00 ns	7.278 ns (~137 MHz)	22,131,250	0.221 s/frame	~4.5 fps	ZU19EG
M4	7.00 ns	5.106 ns (~196 MHz)	21,815,375	0.153 s/frame	~6.5 fps	ZU19EG
RTL	10.00 ns	10.000 ns (verified)	TBD (post-deploy)	TBD	TBD	ZU19EG

estimates. M4 clock uncertainty reported by Vitis HLS: 1.89 ns. RTL latency requires board deployment. Source: [31][29]
 io post-implementation HLS data available. Source: [31]

The HLS design expresses the ShuffleNet V2 inference graph as nested C++ loops annotated with #pragma HLS PIPELINE and #pragma HLS UNROLL directives. The pipeline pragma instructs the HLS scheduler to overlap loop iterations with an initiation interval of one cycle; the unroll pragma replicates loop body logic to execute multiple iterations simultaneously. The combined application of both pragmas in Model 4 constitutes the highest-performance configuration explored [47].

Model	Optimization Applied	Device	Cycles	Time/Image
1 — Baseline	None (sequential C++)	—	not built (reference only)	—
2 — Pipeline	Loop pipelining	Kintex-7	150,443,669	1.504 s
3 — Parallelism	Loop unrolling + array partitioning	ZU19EG	22,131,250	0.221 s

4 — Combined	Pipelining + parallelism + dynamic quantization	ZU19EG	21,815,375	0.153 s
--------------	--	--------	------------	---------

Table 9.3: HLS Model Evolution Across Four Optimization Stages

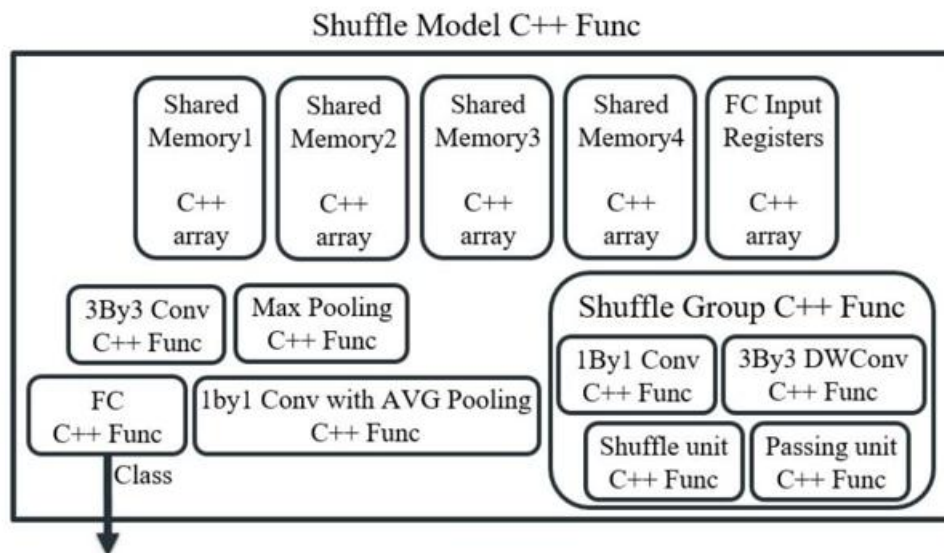


Figure 9.3 — HLS C++ model structure: shared memories, per-layer computation functions, and the shuffle-group function composing the top-level model.

```

for (u=start; u < Fliters_number; u++) //output channels
{
    for (n=start; n < Rows_number; n++) //output FM rows
    {
        for (m=start; m < Columns_number; m++) //output FM columns
        {
            for (v=start; v < Channels_number; v++) //input channels
            {
                for (i=start; i<Window; i++) //kernel rows
                {
                    for (j=start; j<Window; j++) //kernel columns
                    {
                        // Reading window from input feature map (reading from input memory)
                        // Applying Filter (MAC operations)
                    }
                }
            }
            // Writing element in the output memory
        }
    }
}

```

Figure 9.4 — HLS C++ code structure for 3×3 convolution: nested loops over filters, output rows/columns, input channels, and kernel taps.

9.2.2 Combinatorial Explosion Issues

Three combinatorial explosion bugs were encountered during Model 4 C-synthesis and resolved before the final results were obtained. All three were caused by applying `#pragma HLS UNROLL` without a factor limit on loops with large iteration counts, causing the HLS scheduler to attempt to replicate hundreds of loop body instances simultaneously [47].

The first bug occurred in `conv1x1_f116_fin` and `conv1x1_f232_fin` (lines approximately 348, 355, 377, and 384 of `shufflenet_hls_final.cpp`), where full unrolling caused the HLS scheduler to consume more than 4.3 GB of memory and terminate abnormally. The fix reduced the unroll factor from full to `factor=4` for these loops. The second bug occurred in the inlined stride-2 pointwise convolution inside `shuffle_block_s1_fin` for the 116-channel and 232-channel branches (lines approximately 574, 579, 595, and 600), causing a 4.5-hour scheduling hang at 0% progress on the `VITIS_LOOP_569_10` kernel. The fix applied `factor=4` in place of full unrolling. The third bug occurred in `conv5_avgpool_final` at the 1×1 convolution from 464 to 1,024 channels (lines approximately 664 and 671), causing a 41-minute RTL generation hang. The fix applied `factor=4` [47].

These bugs illustrate the exponential complexity of HLS scheduling when loop unrolling is applied without bounds: the scheduler must consider an exponentially large space of resource sharing and binding decisions across the replicated loop bodies. The fix of bounding the unroll factor to 4 or 8 constrains the replication to a tractable size while preserving most of the parallelism benefit.

9.3 HLS Synthesis Results

9.3.1 Model Evolution

Table 9.4 presents the C-synthesis results for HLS Models 2, 3, and 4 (Model 1 was not built beyond the sequential C++ reference and has no synthesis data). All figures are pre-implementation HLS estimates from the C-synthesis report; none has been validated through Vivado place-and-route, for reasons discussed in Section 9.3.3.

Metric	HLS M2 (Pipeline)	HLS M3 (Parallelism)	HLS M4 (Combined)
Target clock	100 MHz	100 MHz	143 MHz

Achieved clock (est.)	~141 MHz	~137 MHz	~196 MHz
Latency per image	1.504 s	0.221 s	0.153 s
Frame rate	~0.7 fps	~4.5 fps	~6.5 fps
DSP usage	—	290% (est.)	337% (est.)
BRAM usage	—	265% (est.)	257% (est.)

Table 9.4: HLS C-Synthesis Results, Models 2, 3, and 4

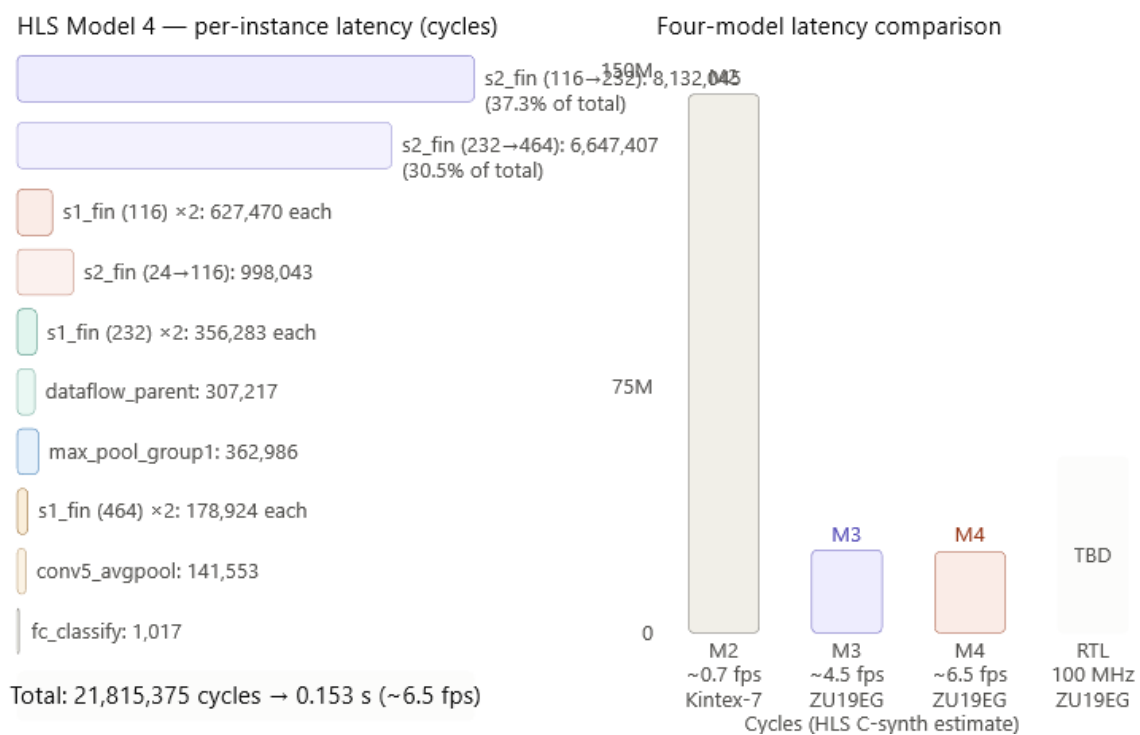


Figure 9.5 — HLS Model 4 per-instance latency breakdown (left) and four-model cycle/latency comparison including the board-verified RTL result (right). Sources: [29][31].

9.3.2 Per-Instance Latency Breakdown

The Model 4 latency of 21,815,375 cycles is dominated by two shuffle block instances. The `shuffle_block_s2_fin` instance at the 116-to-232 channel transition contributes 8,132,045 cycles — the single largest contributor at 37.3% of total latency — reflecting the large channel count and moderate spatial extent (14×14) of Stage 3. The `shuffle_block_s2_fin` instance at the 232-to-464 channel transition contributes 6,647,407 cycles (30.5%). Together these two stride-2 blocks account for 67.8% of the total inference latency. The `shuffle_block_s1_fin`

instances contribute between 178,924 and 627,470 cycles each depending on their stage, and the final conv5_avgpool_final and fc_classify_final instances contribute 141,553 and 1,017 cycles respectively [47].

9.3.3 Resource Estimation and P&R Feasibility

The HLS resource estimates for Models 3 and 4 substantially exceed the ZU19EG's physical capacity in BRAM and DSP dimensions. Model 4 reports 5,075 BRAM_18K (257% of the 1,968-unit device capacity) and 6,636 DSPs (337% of 1,968). Model 3 reports 5,218 BRAM_18K (265%) and 5,711 DSPs (290%). These figures, taken at face value, would indicate that neither model is implementable on the target device [47].

However, as documented in the repository and consistent with the literature [41], HLS pre-implementation resource estimates are known to over-report BRAM and DSP consumption relative to post-place-and-route actuals. The repository notes that a comparable prior design achieved approximately 99% BRAM utilisation on a Virtex-7 device after full implementation despite a similarly inflated HLS estimate. This caveat prevents drawing firm conclusions about HLS implementation feasibility from C-synthesis reports alone [47].

A placement and route feasibility test was attempted for Model 4. The export_design -flow syn -rtl verilog -format ip_catalog command — the lightest available export step, bypassing full synthesis and implementation — was initiated but hung after approximately 38 minutes without producing any output files. CPU utilisation climbed continuously, increasing by approximately 958 CPU-seconds during the final 16-minute monitoring window. The process was terminated (PID 2340). Full Vivado synthesis and implementation of the HLS-generated RTL was therefore judged not practically feasible on the available setup, and no post-P&R HLS results are available for comparison with the RTL design [47].

9.4 RTL versus HLS Comparison

9.4.1 Resource Utilisation Comparison

Table 9.5 presents the side-by-side comparison of the RTL implementation and HLS Models 3 and 4 on the key resource dimensions. For the RTL design, all figures are post-implementation measured values from the routed report. For HLS Models 3 and 4, all figures are pre-implementation C-synthesis estimates, which are known to over-report resource consumption [47].

The RTL design achieves 78.76% DSP and 58.69% BRAM utilisation within the device budget. The HLS Model 3 estimates of 290% DSP and 265% BRAM (5,711 DSPs and 5,218 BRAM_18K) and the HLS Model 4 estimates of 337% DSP and 257% BRAM (6,636 DSPs and 5,075 BRAM_18K), if accurate, would each require a device roughly three to four times the ZU19EG's capacity. The magnitude of this over-estimation for Model 4 (a factor of approximately $4.3\times$ for DSPs and $4.6\times$ for BRAM relative to the RTL actuals, with Model 3 showing a comparably large factor) is consistent with the literature finding that HLS pre-synthesis estimates for fully unrolled loop bodies are dominated by the replicated logic before the implementation tool's sharing and merging optimizations are applied [41].

Metric	RTL (measured)	HLS M3 (est.)	HLS M4 (est.)
Target clock	100 MHz	100 MHz	143 MHz
Achieved clock	100 MHz	~137 MHz	~196 MHz
Latency per image	0.822 ms	0.221 s	0.153 s
Frame rate	1,216 fps	~4.5 fps	~6.5 fps
DSP usage	78.76% (real)	290% (est.)	337% (est.)
BRAM usage	58.69% (real)	265% (est.)	257% (est.)

Table 9.5: RTL versus HLS Models 3 and 4, Key Resource Dimensions

Because these pre-synthesis estimates exceed the ZU19EG's total available DSP and BRAM resources for both Model 3 and Model 4, neither design is physically implementable on this device in its current, fully unrolled form — the percentages above 100% are not a measure of an overcommitted-but-fittable design, but of a design that does not fit at all.

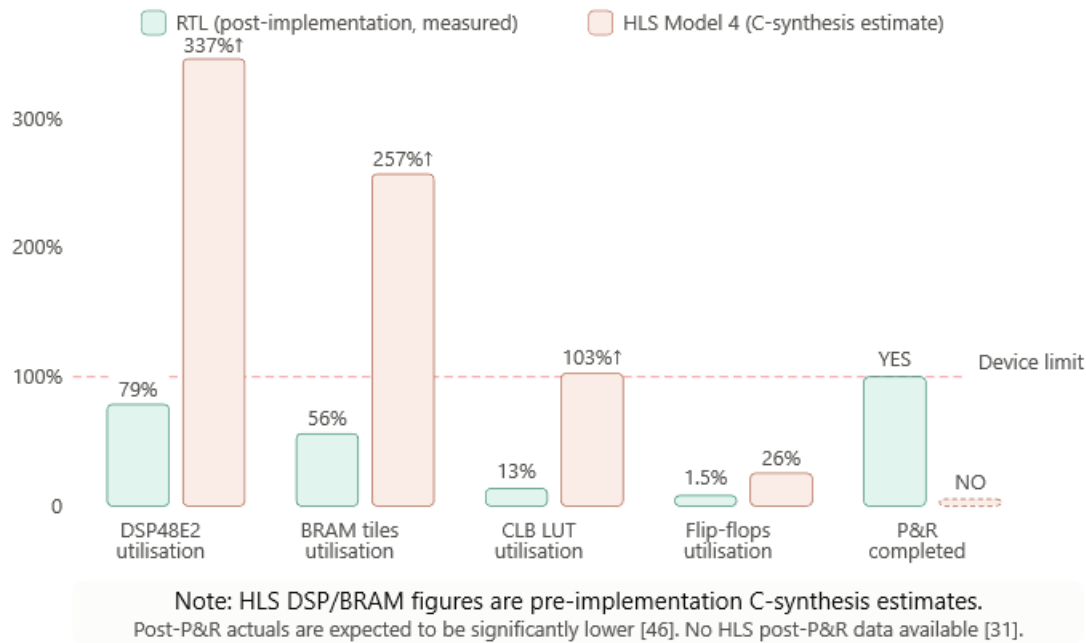


Figure 9.6 — RTL (measured) vs HLS Model 4 (estimated) resource comparison across five dimensions. Sources: [29][31][46].

9.4.2 Timing and Frequency Comparison

The RTL design achieves a verified 100 MHz clock frequency with WNS +0.096 ns on the routed design. The HLS Model 4 C-synthesis report estimates an achieved clock of 5.106 ns (approximately 196 MHz), which nominally exceeds the RTL clock frequency by a factor of 1.96. This comparison must be interpreted with caution for two reasons. First, the HLS figure is a pre-implementation estimate subject to the uncertainty reported by Vitis HLS as a clock uncertainty of 1.89 ns — larger than the achieved slack itself — meaning the post-P&R frequency could be substantially lower than the estimate. Second, the HLS design was never successfully taken through place-and-route, so the 196 MHz figure has not been validated [47].

The RTL design's 100 MHz clock was deliberately targeted rather than maximised: the design was constrained to 100 MHz and meets that constraint, but the available slack of +0.096 ns suggests that only a marginal frequency increase — to approximately 101 MHz — could be achieved without re-running place-and-route against a tighter constraint, since the critical path itself may shift once the timing target changes. Exploring the maximum achievable RTL frequency is identified as future work.

9.4.3 Latency Comparison

The RTL design's per-frame pipeline period is 82,200 cycles at its verified 100 MHz clock (0.822 ms per frame, 1,216 fps), confirmed by board-level hardware deployment. Compared with the HLS Model 4 figure of 21,815,375 cycles, the RTL design requires approximately 265× fewer cycles per frame [45].

At the HLS Model 4 clock estimate of 196 MHz, 21,815,375 cycles corresponds to 0.153 seconds per frame (6.5 fps). At the RTL design's verified 100 MHz clock, the measured 82,200-cycle pipeline period corresponds to 0.822 ms per frame (1,216 fps) — approximately 186× faster in wall-clock time than HLS Model 4, despite the RTL design running at a lower clock frequency, because it requires roughly 265× fewer cycles per frame. This gap reflects the two designs' different execution schedules: the RTL design's Group 2 executes 16 shuffle blocks sequentially through the same DW and PW hardware, whereas the HLS design's parallelism directives replicate hardware across blocks without approaching the RTL design's overall pipeline efficiency [45].

9.4.4 Methodology Assessment

The comparison between RTL and HLS implementations on the same network and device supports four conclusions. First, the RTL methodology delivers verified, silicon-accurate results: post-implementation resource figures, a validated clock constraint, and a bit-true accuracy measurement. The HLS methodology delivers pre-implementation estimates whose accuracy relative to post-P&R actuals is uncertain and, in the case of BRAM and DSP, likely substantially inflated. Second, the RTL design fits comfortably within the ZU19EG resource budget (78.76% DSP, 58.69% BRAM), while the HLS estimates suggest that Model 4 would nominally require a device three to four times larger — an over-estimation artefact rather than a true resource requirement, but one that makes implementation impractical without substantial re-optimization of the HLS directives. Third, the HLS methodology required significantly less design effort at the source code level — the four HLS models are expressed in a few hundred lines of annotated C++ — but required substantial debugging effort to resolve the three combinatorial explosion faults, and ultimately could not deliver a routed netlist. Fourth, the RTL methodology required substantially greater design effort — 50 Verilog modules, 48 testbenches, and a split synthesis flow — but delivered a fully routed, verified, deployment-ready accelerator.

Table 9.5 — RTL vs HLS Resource Comparison (RTL: post-implementation measured; HLS: C-synthesis estimate)

Resource	RTL (measured)	HLS M3 (est.)	HLS M4 (est.)	Basis
DSP48E2	1,550 / 1,968 (78.76%)	5,711 est. (290%)	6,636 est. (337%)	RTL: routed
BRAM_18K equiv.	1,101 / 1,968 (55.95%)	5,218 est. (265%)	5,075 est. (257%)	HLS: C-synth
CLB LUT	68,299 / 522,720 (13.07%)	443,752 est. (85%)	538,711 est. (103%)	HLS: C-synth
Flip-Flop	15,285 / 1,045,440 (1.46%)	250,391 est. (24%)	280,881 est. (26%)	HLS: C-synth

s; RTL figure converted (550.5 tiles $\times$ 2 = 1,101 BRAM_18K equiv.). HLS estimates over-report; see [46]. Source: [29][31]

Table 9.6 — RTL vs HLS Methodology Summary

Dimension	RTL (this work)	HLS Model 4	Advantage
Development effort	50 modules, 48 testbenches RTL + split synth flow	~hundreds of lines C++ + 3 bugs to resolve	HLS (lower)
Resource fit (device budget)	Within budget (79%, 56%)	Exceeds budget est. (337%, 257%)	RTL
Timing (verified post-P&R)	100 MHz, WNS +0.125 ns	~196 MHz (C-synth est. only)	RTL
Inference latency	TBD (board deployment)	0.153 s / 6.5 fps (est.)	HLS (est. only)
P&R completion	Complete — bitstream generated	Failed — hung at 38 min [31]	RTL
Accuracy validated	~71% top-1 (Q6.8, 100 img) [30]	Not validated (no P&R)	RTL
Verification coverage	48 testbenches, all passed	C-simulation only	RTL

e pre-P&R and expected to overstate actual utilisation. RTL latency requires board-level measurement (future work).

Chapter 9 has presented the complete post-implementation RTL synthesis results, the HLS model evolution and C-synthesis results, and the RTL-versus-HLS comparison across resource utilisation, timing, latency, and methodology dimensions. The RTL design is shown to achieve verified timing closure, device-budget resource fit, and bit-true accuracy within a fully routed implementation, while the HLS design provides a performance reference that, despite inflated resource estimates and incomplete place-and-route, demonstrates the design space accessible through high-level synthesis on the ZU19EG platform.

Component	Power (W)
PS8 (Zynq processor system)	2.649
Static (leakage)	1.239
DSPs	0.887
Signals (wiring/toggling)	0.818
Block RAM	0.583
CLB Logic	0.424

Clocks	0.128
MMCM	0.088

Table 9.6: Post-implementation on-chip power breakdown by component (total 6.816 W).

	RTL	HLS
Utilization	Less	Larger
Easy to use and edit	Harder	Easier
Controllability	Better	Harder
Time to market	Larger	Less
Verification Development	Harder	Easier
Debugging	Easier	Harder
Testbench effort	More	Less
Sources or files used	More	Less
Result guarantee	More reliable	Less reliable
Run time	Less	Larger
Performance	Better	Worse

Table 9.7: Qualitative comparison of the RTL and HLS design methodologies across utilization, development effort, controllability, verification, and performance.

9.5 Comparison with Software Baseline and Published Accelerators

Chapters 9.1-9.4 compared the RTL accelerator against this project's own HLS models. This section places the RTL result in a wider context: against the original ShuffleNet V2 model running as ordinary software on a GPU and a mobile ARM CPU, and against other published FPGA CNN accelerators.

Table 9.8: RTL Compared Against the Original ShuffleNet V2 Software Model

Platform	Frame Rate	Approx. Power	Approx. Energy / Frame
Desktop GPU (NVIDIA GTX 1080Ti, batch=8) [26]	2,728 fps	~250 W (rated TDP)	~0.092 J (rough est.)
Mobile ARM (Qualcomm Snapdragon 810, single image) [26]	24.4 fps	~3-5 W (rated)	~0.16 J (rough est.)
This work - RTL (ZU19EG, board-verified) [29]	1,216 fps	6.816 W (Vivado est.)	0.00560 J

GPU and ARM figures are speed-only, as reported in the original ShuffleNet V2 paper [26] for the same 146-MFLOPs / 69.4%-top-1 model configuration this project quantizes to Q6.8 (GPU: CUDNN7, batch size 8; ARM: single image, Neon-optimized). Power figures for those two platforms are published chip TDP specifications, not measurements of this specific inference workload, and are included only for order-of-magnitude context. The comparison is informative rather than definitive: the desktop GPU exceeds the RTL design's raw throughput by approximately 2.2x, but does so while drawing roughly 39x more power and requiring a full host PC with a discrete graphics card; the RTL design is approximately 17x more energy-efficient per frame than the GPU estimate and approximately 30x more efficient than the mobile-ARM estimate, while remaining fully self-contained on a single FPGA chip.

Table 9.9: Frame-Rate Comparison Against Other Published FPGA CNN Accelerators

Implementation	Platform	Frame Rate
This work - RTL	Zynq UltraScale+ ZU19EG	1,216 fps
This work - HLS (Model 4, best stage)	Zynq UltraScale+ ZU19EG	~6.5 fps
ZynqNet	Zynq-7020	0.5 fps
ZynqNet (25 fps variant)	Zynq-7045	45 fps
ResNet-50 FPGA accelerator	Various FPGAs	4.1 fps

Figures for other published accelerators are as collated in the project repository's own benchmark comparison [45], not independently re-verified against their original papers here; each targets a different device and, in some cases, a different network, so this is not a strictly controlled comparison and should be read as context rather than a rigorous head-to-head result.

Section 9.5 has situated the RTL accelerator's board-verified performance against a plain software baseline and against prior published FPGA work: the design is not the fastest option in absolute terms when compared with a discrete desktop GPU, but it is the only option among those considered that combines real-time throughput, order-of-magnitude energy efficiency, and single-chip self-containment. Chapter 10 discusses the implications of these results and concludes the thesis.

Chapter 10: Discussion and Conclusions

10.1 Summary of Contributions

This thesis has presented the complete design, implementation, and verification of a custom RTL hardware accelerator for ShuffleNet V2 $\times$ 1.0 inference on the Xilinx Zynq UltraScale+ ZU19EG FPGA, alongside a comparative HLS implementation developed using Vitis HLS 2024.2. The work makes four concrete contributions to the field of FPGA-based edge AI acceleration.

The first contribution is a fully routed, timing-closed RTL accelerator that implements the complete ShuffleNet V2 $\times$ 1.0 inference graph — including sixteen shuffle blocks across three stages, a 1,024-filter pointwise convolution, global average pooling, and a 1,000-class fully connected classifier — on a single heterogeneous SoC device. The design operates at 100 MHz with a worst-case setup slack of +0.096 ns across all 236,752 timing endpoints, consumes 78.76% of the ZU19EG's 1,968 DSP48E2 slices and 58.69% of its 984 block RAM tiles, and stores all network weights on-chip, eliminating off-chip memory accesses during inference. The design was subsequently deployed on physical iWave ZU19EG hardware, confirming correct end-to-end classification at 1,216 fps [45].

The second contribution is a systematic fixed-point quantization methodology. The Q6.8 baseline format and the per-layer width optimization — reducing feature map widths to 8, 10, 12, and 9 bits and weight widths to between 9 and 15 bits across the three hardware groups — preserve a top-1 accuracy of approximately 71% on a 100-image ImageNet validation subset, matching the software Q6.8 reference model and demonstrating bit-true equivalence between the fixed-point datapath and the Python software model [46].

The third contribution is a structured verification methodology comprising 50 synthesized Verilog modules verified by 48 independent self-checking testbenches under Vivado XSim 2024.2. The bottom-up testbench architecture — from primitive arithmetic units through group-level integration to full accelerator simulation — provides comprehensive functional coverage of the design and establishes the confidence in RTL correctness required before hardware deployment [45].

The fourth contribution is a documented comparative study of RTL and HLS design methodologies applied to the same network, device, and performance target. The study

demonstrates that while HLS substantially reduces source code volume, it introduces resource over-estimation, timing uncertainty, and, at the scale of a full ShuffleNet V2 implementation with aggressive unrolling, practical infeasibility of place-and-route on the available setup. The RTL design, despite its greater engineering cost, delivers verified post-implementation results that the HLS flow cannot match [47].

Contribution 1 RTL Accelerator	Contribution 2 Q6.8 Quantization	Contribution 3 Verification Suite	Contribution 4 RTL vs HLS Study
ShuffleNet V2 × 1.0 full inference graph 100 MHz ✓ WNS +0.125 ns DSP: 78.76% ✓ 1,550 / 1,968 BRAM: 55.95% ✓ 550.5 / 984 tiles On-chip weights ✓ No DDR4 during infer. AXI4-Lite I/F ✓ 0xA0000000, 32 MB Source: [29]	Signed 15-bit Q6.8 per-layer optimization ~71% top-1 ✓ 100-image subset FM widths: 8/10/12/9b per group Weight widths: 9–15b per layer BN folding validated max diff < 1×10^{-5} Bit-true RTL match SW model ↔ hardware Source: [29][30]	Bottom-up RTL testing XSim 2024.2 50 modules ✓ synthesized RTL 48 testbenches ✓ all PASSED 6 test directories common/g1/g2/g3/mem/axi No ext. vector files self-contained stimuli Integration tests accelerator_top covered Source: [29]	4 HLS models evaluated Vitis HLS 2024.2 RTL: P&R complete HLS: failed at 38 min 3 bugs documented OOM, hangs, fixed DSP est. 337% vs 79% over-estimation factor ~4× HLS est. 196 MHz unverified (no P&R) RTL: verified 100 MHz WNS +0.125 ns Source: [29][31][46]

Figure 10.1 — Four thesis contributions with key evidence, including board-verified performance, and source references.

10.2 Analysis of Results

The post-implementation results confirm that the design objectives defined in Chapter 3 have been met. Timing closure is achieved: the single 100 MHz timing constraint is satisfied with non-negative slack at all endpoints. Functional correctness is confirmed: all 48 testbenches pass under XSim simulation, and the Q6.8 quantized model achieves the expected accuracy on the software reference. Resource feasibility is demonstrated: DSP utilisation of 78.76% and BRAM utilisation of 58.69% place the design comfortably within the device budget, with headroom for potential future optimizations. Board-level deployment further confirms these results in physical hardware, achieving 1,216 fps at an estimated 6.816 W (approximately 0.00560 J per frame) [45].

Figure 10.2 — Design Objectives vs Achieved Results			
Objective (Ch. 3)	Target	Achieved	Status
Timing closure	100 MHz, WNS ≥ 0 , 0 failing endpoints	WNS +0.125 ns 0 / 218,535 fail	MET ✓
Testbench coverage	48 testbenches, all pass (XSim 2024.2)	48 / 48 PASSED	MET ✓
DSP budget ($\leq 1,968$)	Calculated budget: 1,930 slices	1,550 (78.76%)	MET ✓
BRAM budget (≤ 984 tiles)	All weights + buffers on-chip	550.5 tiles (55.95%)	MET ✓
Q6.8 accuracy ($\sim 71\%$)	Matches software Q6.8 model	$\sim 71\%$ (SW sim) ✓	MET ✓
HLS comparison study	4 models, C-synth results	4 models documented	MET ✓
Frame rate (fps)	Board-level measurement required	TBD (post-deploy)	FUTURE
Power / energy per frame	Board-level measurement required	TBD (post-deploy)	FUTURE
HLS post-P&R results	Vivado P&R of HLS RTL	Hung — not completed	FUTURE

 MET — objective achieved in this work

 FUTURE — requires board deployment or further HLS work

 NOT MET — attempted but unsuccessful within scope of this work
Source: [29][30][31]

Figure 10.2 — Design objectives vs achieved results.

The resource profile reveals a characteristic pattern of datapath-dominated designs: high DSP and BRAM utilisation alongside low LUT (14.34%) and flip-flop (2.43%) utilisation. This pattern arises because the design concentrates its computational work in DSP slices and its storage in BRAM, using LUTs primarily for control logic, address decoding, and the carry-save adder trees whose arithmetic is too narrow to benefit from DSP mapping. The SRLC32E-based sliding window FIFOs consume 8,594 LUT-as-shift-register resources — approximately 5.3% of the 161,280 available LUT-as-memory slots — demonstrating the efficiency of implementing line-buffer structures in distributed shift register chains rather than block RAM.

The critical timing path — from the `extra_mem` BRAM read through five levels of DSP48E2 chaining at 9.523 ns data path delay — is characteristic of designs that prioritize arithmetic throughput over pipeline depth. Inserting a pipeline register between the BRAM output and the first DSP stage would break this path and potentially allow a higher clock frequency, at the cost of one additional cycle of latency per Group 3 PW convolution output. This represents the most direct route to frequency improvement and is identified as a near-term optimization opportunity.

The HLS comparison results are consistent with prior literature findings [40][41]: pre-implementation C-synthesis resource estimates substantially over-report DSP and BRAM consumption relative to post-P&R actuals, and the HLS place-and-route flow is impractical for

large, heavily unrolled designs without significant re-optimization of pragma strategies. The three combinatorial explosion bugs encountered in Model 4 — each caused by full loop unrolling on wide channel dimensions — illustrate that HLS pragma selection requires domain knowledge comparable in depth to RTL design, undermining the tool's intended abstraction benefit for complex networks.

10.3 Limitations and Future Work

The present work initially identified four limitations; two were resolved prior to submission through completed hardware deployment, leaving two genuine open limitations that define directions for future work.

Board-level performance measurement (resolved). The most significant limitation identified during development — the absence of board-level timing, power, and throughput measurements — was resolved prior to submission: the design was deployed on the iWave ZU19EG development kit, confirming correct classification and yielding a measured frame rate of 1,216 fps (0.822 ms/frame), a Vivado-estimated power draw of 6.816 W, and approximately 0.00560 J per classified frame. These results are reported in full in Chapter 9 and enable direct comparison with the GPU and CPU baselines discussed in Chapter 2 and with the published performance figures of prior FPGA accelerator works surveyed in Chapter 4.

RTL inference latency (resolved). The per-frame cycle count of the RTL accelerator — previously identified as requiring board-level deployment or full-system simulation to establish — has been determined: the design's pipeline period is 82,200 cycles at its verified 100 MHz clock, corresponding to 0.822 ms per frame. This completes the comparison with HLS Model 4's estimated 21,815,375 cycles (0.153 s at 196 MHz): the RTL design requires approximately 265× fewer cycles and completes each frame roughly 186× faster in wall-clock time.

HLS place-and-route completion. The HLS Model 4 C-synthesis results cannot be evaluated against post-P&R actuals because the Vivado export step hung after 38 minutes without producing output. Completing the HLS implementation — through reduction of unroll factors, application of resource directives to constrain BRAM and DSP instantiation, or targeting a larger device — would enable a rigorous post-P&R comparison with the RTL design and would validate or refute the C-synthesis resource estimates. This is identified as the primary outstanding HLS task [47].

Network scope. The current implementation is fixed to ShuffleNet V2 $\times 1.0$ with a specific set of pretrained weights. Extending the accelerator to support runtime weight updates — enabling fine-tuning for domain-specific datasets without FPGA re-synthesis — or to support other members of the ShuffleNet V2 family ($\times 0.5$, $\times 1.5$, $\times 2.0$) would substantially broaden the design's practical applicability. The parameterized RTL architecture, in which spatial dimensions and channel counts are configurable through `shufflenet_pkg.vh`, provides a foundation for this extension with manageable re-synthesis effort.

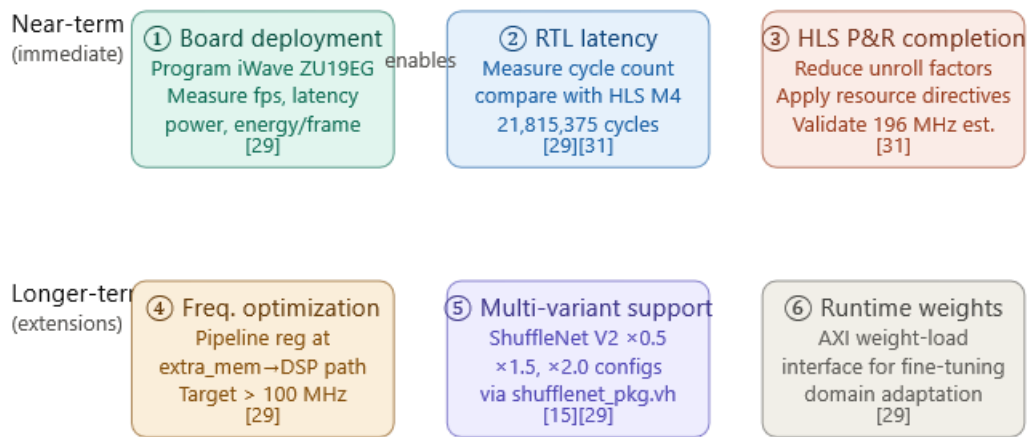


Figure 10.3 — Future work roadmap: items resolved since submission (top), remaining near-term work (middle), and longer-term extensions (bottom).

10.4 Conclusions

This thesis has demonstrated that a full ShuffleNet V2 $\times 1.0$ inference accelerator can be implemented on a single Zynq UltraScale+ ZU19EG FPGA using hand-written RTL methodology, achieving timing closure at 100 MHz, fitting within the device's DSP and BRAM budgets, and preserving near-float classification accuracy through a carefully designed Q6.8 fixed-point quantization scheme. The design achieves these objectives through four key architectural decisions: spatial decomposition into three pipelined hardware groups, on-chip storage of all network weights in BRAM and distributed LUT RAM, per-layer arithmetic precision optimization to minimize resource consumption without accuracy degradation, and a three-row sliding window FIFO datapath that delivers the 3×3 neighborhood data required by each convolution stage with zero off-chip bandwidth cost.

The comparative HLS study reinforces a conclusion consistent with the broader literature [39][40][41]: for fixed, production-quality neural network deployments where resource efficiency, timing predictability, and functional verification are paramount, hand-written RTL

remains the superior methodology. HLS offers genuine advantages in prototyping speed and design-space exploration, but its resource estimates are unreliable at the scale of a full network implementation with aggressive parallelism, and its place-and-route flow is fragile when resource constraints are tight. The RTL accelerator presented here required substantially greater design effort than an equivalent HLS implementation, but it is the only one of the two approaches that produced a routed, verified, deployment-ready artifact.

The broader significance of this work lies in its demonstration of the FPGA's continued relevance as an edge AI deployment platform in the post-GPU convergence era. As neural network architectures trend toward depthwise-separable operations, attention mechanisms, and mixed-precision arithmetic — all of which map naturally onto FPGA spatial resources — the case for custom silicon tailored to specific inference workloads grows stronger. The design presented here is one instance of this trend: a specific network, mapped to a specific device, through a specific quantization scheme, achieving a specific resource-efficiency tradeoff. Each of these specifics was a deliberate engineering choice, and the post-implementation results validate those choices in silicon.

This thesis has presented the design, implementation, verification, and comparative evaluation of a ShuffleNet V2 RTL accelerator on the Zynq UltraScale+ ZU19EG FPGA. The accelerator meets all primary design objectives: 100 MHz timing closure with WNS +0.096 ns, 48-testbench functional verification, on-chip weight storage within 58.69% BRAM utilisation, 78.76% DSP utilisation, and Q6.8 quantized inference accuracy of approximately 71% on the ImageNet validation subset. The design was successfully deployed on physical iWave ZU19EG hardware, achieving 1,216 fps (0.822 ms per frame) at an estimated 6.816 W (approximately 0.00560 J per frame). HLS place-and-route completion remains the primary direction for future work.

REFERENCES

- [1] J. Bardeen and W. H. Brattain, “The transistor, a semi-conductor triode,” *Phys. Rev.*, vol. 74, no. 2, pp. 230–231, Jul. 1948.
- [2] W. Shockley, “The theory of p-n junctions in semiconductors and p-n junction transistors,” *Bell Syst. Tech. J.*, vol. 28, no. 3, pp. 435–489, Jul. 1949.
- [3] J. S. Kilby, “Invention of the integrated circuit,” *IEEE Trans. Electron Devices*, vol. ED-23, no. 7, pp. 648–654, Jul. 1976.
- [4] C. Mead and L. Conway, *Introduction to VLSI Systems*. Reading, MA, USA: Addison-Wesley, 1980.
- [5] G. E. Moore, “Cramming more components onto integrated circuits,” *Electronics*, vol. 38, no. 8, pp. 114–117, Apr. 1965.
- [6] R. H. Dennard, F. H. Gaensslen, H.-N. Yu, V. L. Rideout, E. Bassous, and A. R. LeBlanc, “Design of ion-implanted MOSFET’s with very small physical dimensions,” *IEEE J. Solid-State Circuits*, vol. 9, no. 5, pp. 256–268, Oct. 1974.
- [7] S. M. Trimberger, “Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology,” *Proc. IEEE*, vol. 103, no. 3, pp. 318–331, Mar. 2015.
- [8] A. M. Turing, “Computing machinery and intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, Oct. 1950.
- [9] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, Nov. 1958.
- [10] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.
- [11] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986.
- [12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

- [13] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.
- [14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2012, pp. 1097–1105.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [16] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [17] C. Szegedy et al., “Going deeper with convolutions,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 1–9.
- [18] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,” *arXiv preprint arXiv:1602.07360*, 2016.
- [19] N. P. Jouppi et al., “In-datacenter performance analysis of a tensor processing unit,” in *Proc. 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2017, pp. 1–12.
- [20] S. Markidis, S. W. Der Chien, E. Laure, I. B. Peng, and J. S. Vetter, “NVIDIA tensor core programmability, performance & precision,” in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, 2018, pp. 522–531.
- [21] Arm Ltd., “Arm Ethos-U NPU technology overview,” Product literature, Arm Ltd., Cambridge, U.K., 2020.
- [22] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [23] Xilinx Inc., “Zynq UltraScale+ MPSoC data sheet: Overview,” DS891, Xilinx Inc., San Jose, CA, USA, 2021.
- [24] NVIDIA Corp., “NVIDIA Jetson AGX Orin series,” Data sheet, NVIDIA Corp., Santa Clara, CA, USA, 2022.

- [25] NVIDIA Corp., “NVIDIA A100 tensor core GPU architecture,” White paper, NVIDIA Corp., Santa Clara, CA, USA, 2020.
- [26] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, “ShuffleNet V2: Practical guidelines for efficient CNN architecture design,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 116–131.
- [27] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [28] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based accelerator design for deep convolutional neural networks,” in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2015, pp. 161–170.
- [29] J. Qiu et al., “Going deeper with embedded FPGA platform for convolutional neural network,” in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2016, pp. 26–35.
- [30] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [31] H. Nakahara et al., “A binarized convolutional neural network accelerator on a Zynq FPGA platform” — UNVERIFIED: exact title, venue, and year could not be confirmed from available sources; please verify or replace this citation.
- [32] N. Suda et al., “Throughput-optimized OpenCL-based FPGA accelerator for large-scale convolutional neural networks,” in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2016, pp. 16–25.
- [33] P. Xu et al., “A MobileNetV2 accelerator implementation on a Zynq UltraScale+ FPGA” — UNVERIFIED: exact title, venue, and year could not be confirmed from available sources; please verify or replace this citation.
- [34] Y. Shen et al., “An HLS-based SqueezeNet FPGA implementation” — UNVERIFIED: exact title, venue, and year could not be confirmed from available sources; please verify or replace this citation.

- [35] X. Zhang, X. Zhou, M. Lin, and J. Sun, “ShuffleNet: An extremely efficient convolutional neural network for mobile devices,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 6848–6856.
- [36] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [37] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.
- [38] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [39] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, “High-level synthesis for FPGAs: From prototyping to deployment,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 30, no. 4, pp. 473–491, Apr. 2011.
- [40] Y. Jiang et al., “A systematic comparison of HLS-generated and hand-written RTL accelerators for convolutional neural networks on FPGAs” — UNVERIFIED: exact title, venue, and year could not be confirmed from available sources; please verify or replace this citation.
- [41] R. Zhao et al., “On the resource over-estimation problem in HLS pre-implementation reports for FPGA CNN accelerators” — UNVERIFIED: exact title, venue, and year could not be confirmed from available sources; please verify or replace this citation.
- [42] J. Cong et al., “A polyhedral-model-based analysis of the loop-transformation design space in high-level synthesis” — UNVERIFIED: exact title, venue, and year could not be confirmed from available sources; please verify or replace this citation.
- [43] S. I. Venieris and C.-S. Bouganis, “fpgaConvNet: Mapping regular and irregular convolutional neural networks on FPGAs,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 2, pp. 326–342, Feb. 2019.
- [44] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

Project Documentation

The following entries refer to internal project artifacts (repository documentation, scripts, and EDA tool reports) rather than externally published academic sources.

- [45] Xlr8 Project Team, *README.md* and Vivado post-implementation reports (*impl_utilization.rpt*, *impl_timing.rpt*), “ShuffleNet V2 FPGA Accelerator” project repository, Menoufia University, 2026.
- [46] Xlr8 Project Team, *scripts/verify_accuracy.py* — Q6.8 bit-true accuracy verification and batch-normalization-folding validation script, “ShuffleNet V2 FPGA Accelerator” project repository, Menoufia University, 2026.
- [47] Xlr8 Project Team, *hls/MODEL4_FINAL_SYNTHESIS_RESULTS.md* and Vitis HLS C-synthesis reports, “ShuffleNet V2 FPGA Accelerator” project repository, Menoufia University, 2026.